

C# 基礎課程 01

Bill Chung
V2026.1 台北夏季班

請尊重講師的著作權及智慧財產權!

本課程之教材、程式碼等、僅供課程中學習用、請不要任意自行散佈、重製、分享，謝謝

關於我

- Bill Chung
- Microsoft MVP -- 2009 至今
- 部落格：[海角點部落](#)
- GitHub：<https://github.com/billchungiii>
- FB group：[微軟台灣論壇愛好者俱樂部](#)
- 專長：說故事
- 我是個臉盲

歡迎加入社群

- Tw MVC :
- <https://www.facebook.com/twmvc/>
- 無界開發者社群 :
<https://www.facebook.com/groups/UniversalDevelopers/>

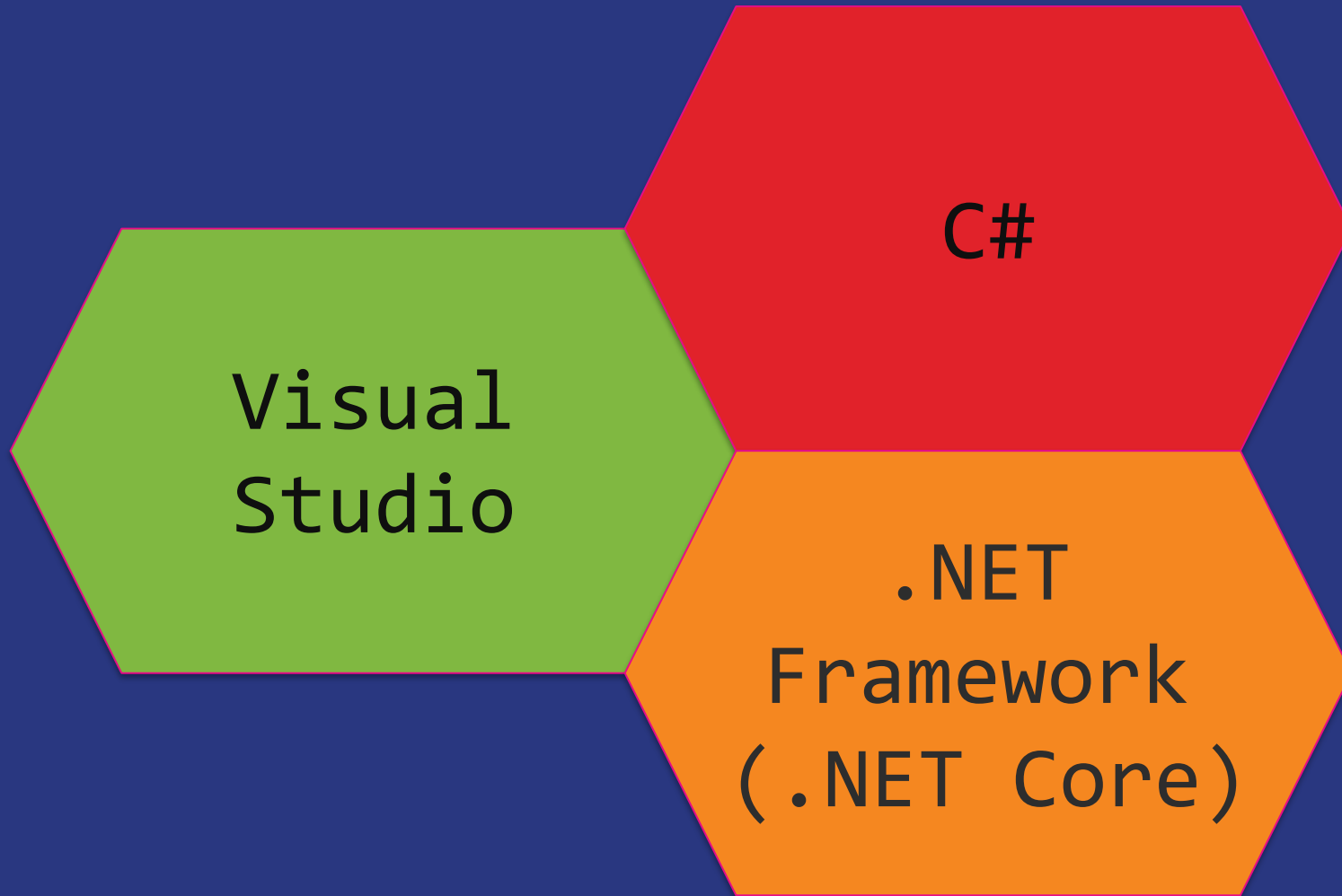
- 點部落：記錄你學習心得的好地方，
<https://dotblogs.com.tw/>
- SkillTree：未來你需要進修的時候，
<https://skilltree.my/>

期望你們能學到

- 程式語言的理論基礎
- 懂得搜尋並閱讀文件
- 使用 **AI** 輔助

工具概要

這個課程我們使用 Visual Studio 2026



年份	.NET Framework	.NET Core	.NET	C# 版本	備註
2002	1.0	-	-	1.0	初始版本
2003	1.1	-	-	1.2	-
2005	2.0	-	-	2.0	泛型支援
2006	3.0	-	-	2.0	WPF, WCF, WF
2007	3.5	-	-	3.0	LINQ
2010	4.0	-	-	4.0	Dynamic, Task
2012	4.5	-	-	5.0	async/await
2015	4.6	-	-	6.0	Roslyn 編譯器
2016	4.6.1	1.0	-	6.0	跨平台開始
2017	4.7	2.0	-	7.0	Pattern Matching
2018	4.7.2	2.1, 2.2	-	7.1, 7.2, 7.3	快速迭代
2019	4.8	3.0, 3.1 (LTS)	-	8.0	最後的 Framework
2020	-	-	5.0	9.0	統一平台
2021	-	-	6.0 (LTS)	10.0	長期支援
2022	-	-	7.0	11.0	-
2023	-	-	8.0 (LTS)	12.0	長期支援
2024	-	-	9.0	13.0	
2025	-	-	10.0	14.0	長期支援

Hi , Visual Studio

主要專案類型

- Web 應用程式
- 網路服務
- 遊戲開發
- 桌面應用程式
- 行動應用程式
- Cloud
- AI

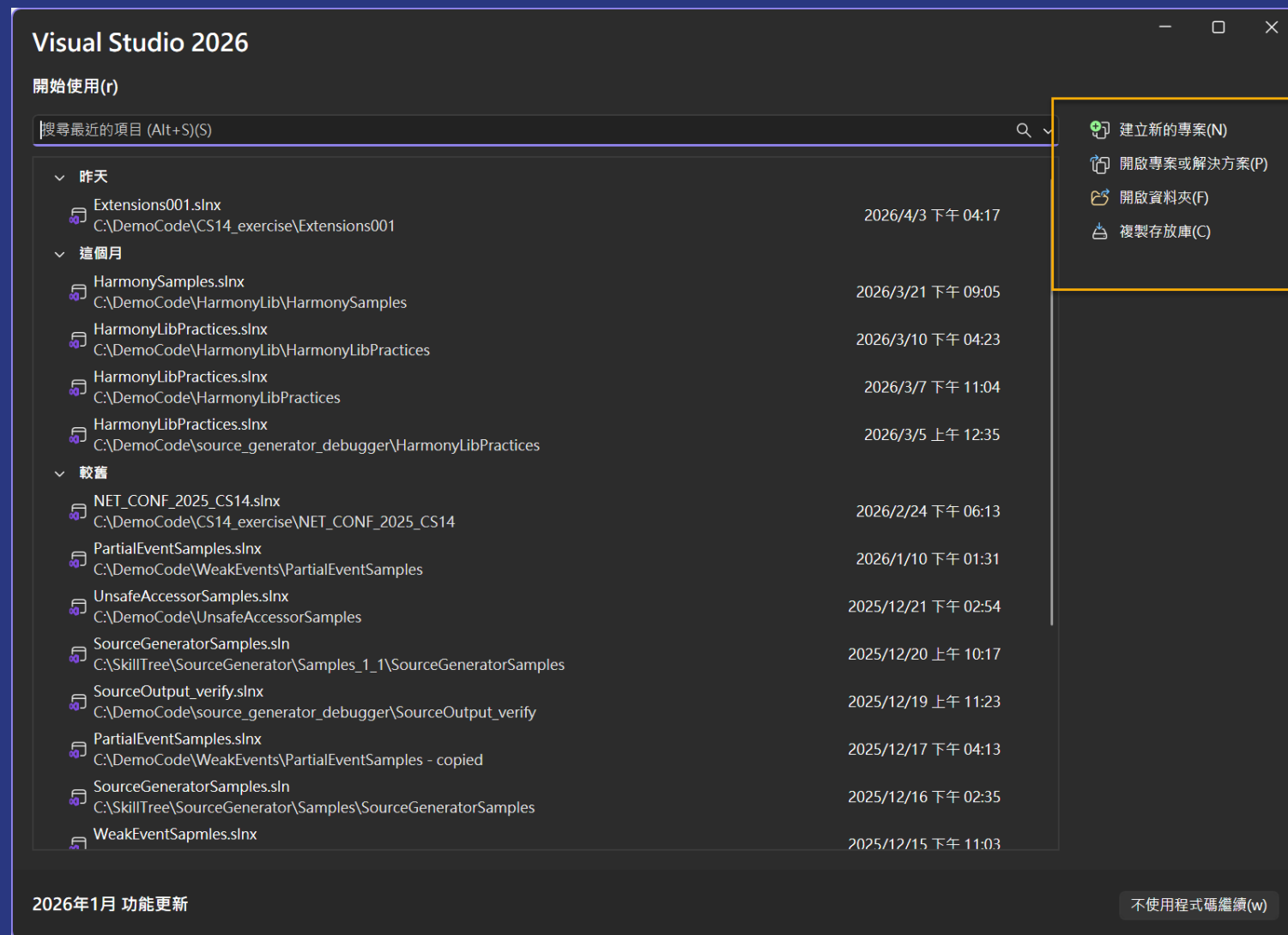
LAB 須知

- 請務必要按照步驟動手做 lab
- 不要心急，慢慢來
- 做 lab 不是複製貼上，請同時動腦。
- 當你想到打結的時候，請務必發問，不要把疑問留在教室裡。

LAB

使用 Visual Studio 新增專案

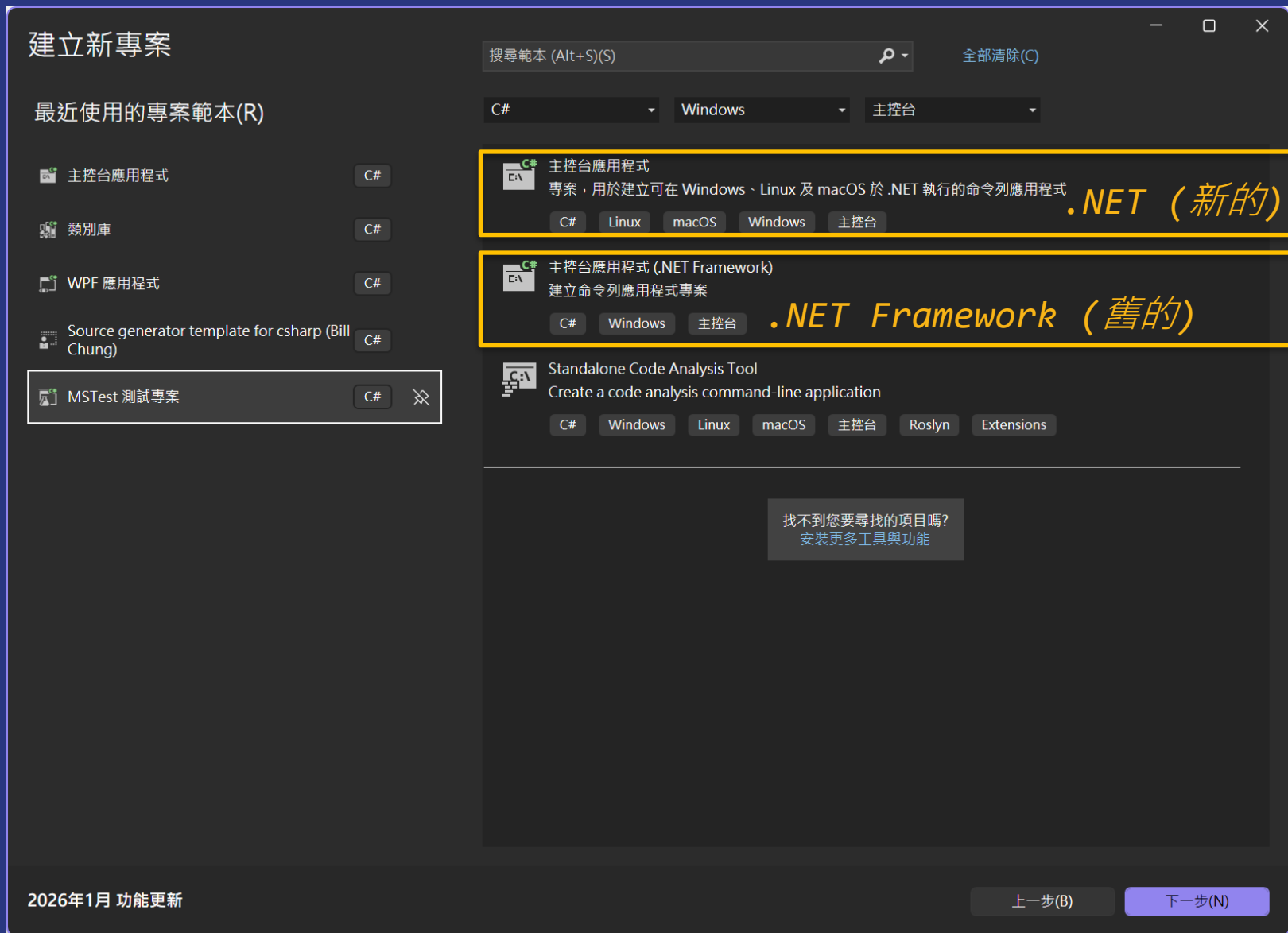
新增專案



過濾與搜尋專案範本



專案範本差異 (選 .NET 新的)



先確認範本是否正確

設定新的專案

主控台應用程式 C# Linux macOS Windows 主控台

專案名稱(I)
ConsoleApp1

位置(L)
C:\DemoCode\HarmonyLib

解決方案名稱(M) ⓘ
ConsoleApp1

☐ 將解決方案與專案置於相同目錄中(D)

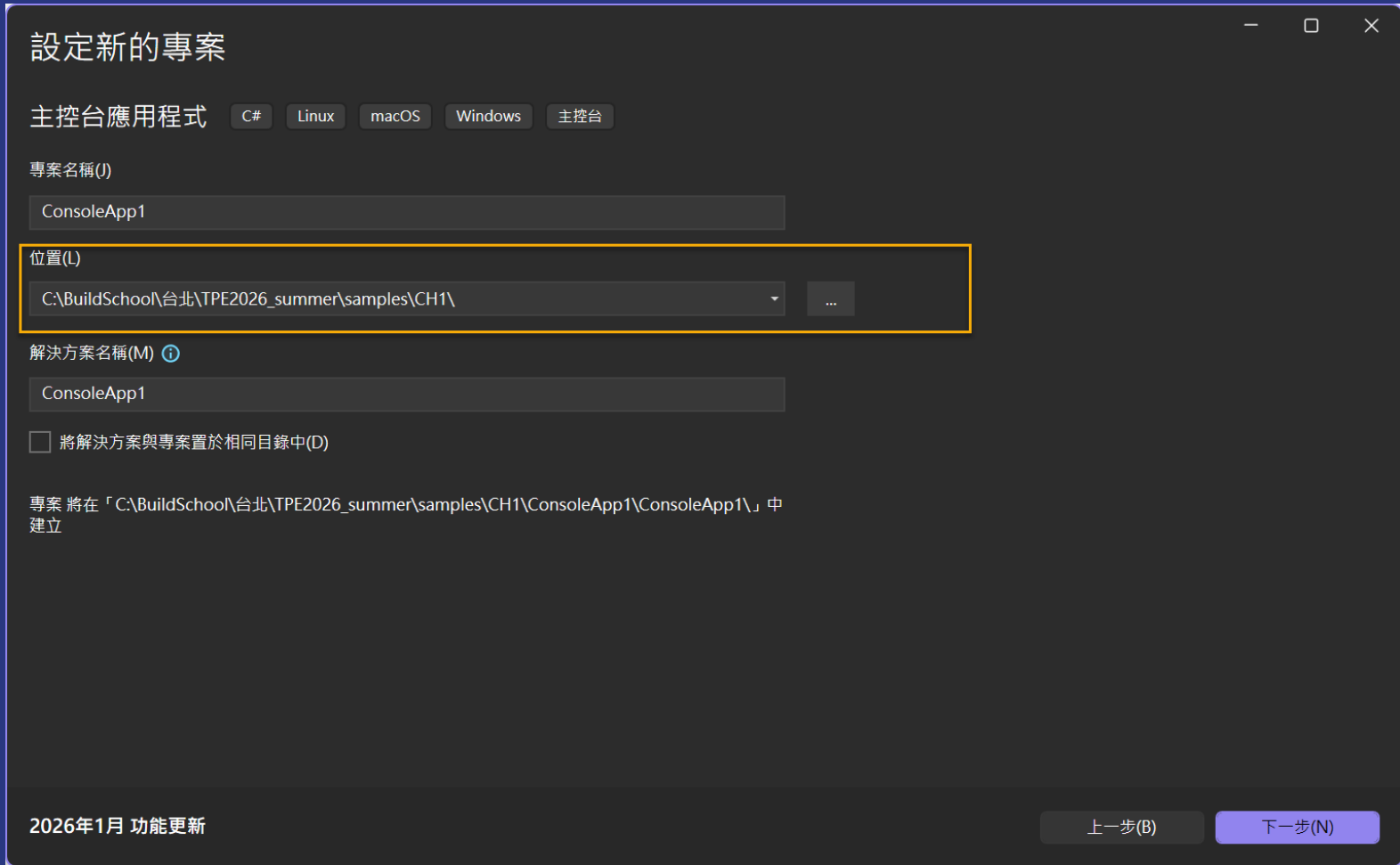
專案 將在「C:\DemoCode\HarmonyLib\ConsoleApp1\ConsoleApp1\」中建立

2026年1月 功能更新

上一步(B) 下一步(N)



選一個好目錄存放你的程式碼 (不要設定在桌面)



設定新的專案

主控台應用程式 C# Linux macOS Windows 主控台

專案名稱(I)
ConsoleApp1

位置(L)
C:\BuildSchool\台北\TPE2026_summer\samples\CH1\

解決方案名稱(M) ⓘ
ConsoleApp1

☐ 將解決方案與專案置於相同目錄中(D)

專案 將在「C:\BuildSchool\台北\TPE2026_summer\samples\CH1\ConsoleApp1\ConsoleApp1\」中
建立

2026年1月 功能更新

上一步(B) 下一步(N)

設定新的專案

主控台應用程式

C#

Linux

macOS

Windows

主控台

專案名稱(I)

ConsoleApp1

位置(L)

C:\BuildSchool\台北\TPE2026_summer\samples\CH1

解決方案名稱(M) ⓘ

MyFirstSolution

☐ 將解決方案與專案置於相同目錄中(D)

請勿勾選

專案 將在「C:\BuildSchool\台北\TPE2026_summer\samples\CH1\MyFirstSolution\ConsoleApp1\」中建立

解決方案名稱改為 MyFirstSolution

**一個方案可以擁有一個或多個專案

專案名稱和方案名稱未必要相同

2026年1月 功能更新

上一步(B)

下一步(N)

設定新的專案

主控台應用程式

C#

Linux

macOS

Windows

主控台

專案名稱(I)

MyFirstApp

位置(L)

C:\BuildSchool\台北\TPE2026_summer\samples\CH1

解決方案名稱(M) ⓘ

MyFirstSolution

☐ 將解決方案與專案置於相同目錄中(D)

專案 將在「C:\BuildSchool\台北\TPE2026_summer\samples\CH1\MyFirstSolution\MyFirstApp\」中
建立

設定專案名稱為 MyFirstApp

2026年1月 功能更新

上一步(B)

下一步(N)

設定新的專案

主控台應用程式

C#

Linux

macOS

Windows

主控台

專案名稱(I)

MyFirstApp

位置(L)

C:\BuildSchool\台北\TPE2026_summer\samples\CH1

解決方案名稱(M) ⓘ

MyFirstSolution

☐ 將解決方案與專案置於相同目錄中(D)

專案 將在「C:\BuildSchool\台北\TPE2026_summer\samples\CH1\MyFirstSolution\MyFirstApp\」中
建立

先檢查完上面的資訊是否正確後
按「下一步」

2026年1月 功能更新

上一步(B)

下一步(N)



其他資訊

主控台應用程式

C#

Linux

macOS

Windows

主控台

架構(F) ⓘ

.NET 10.0 (長期支援)

☐ 啟用容器支援(E) ⓘ

容器作業系統(O) ⓘ

Linux

容器組建類型 ⓘ

Dockerfile

☒ 不要使用最上層陳述式(T) ⓘ

☐ 啟用 native AOT 發佈 ⓘ

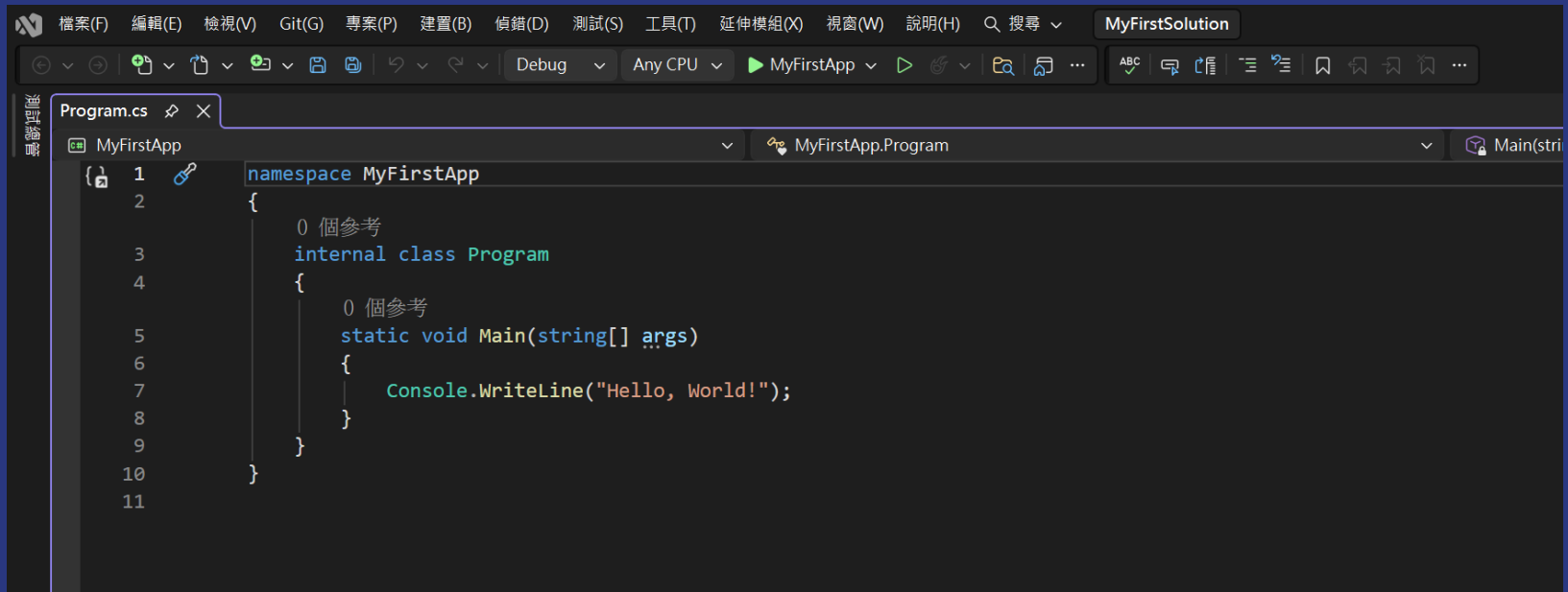
- (1) 選擇架構版本 .NET 10.0
 - (2) 請勿勾選
 - (3) 勾選「不要使用最上層陳述式」
 - (4) 請勿勾選
- 確認好資訊後按下「建立」

2026年1月 功能更新

上一步(B)

建立(C)

建立完成



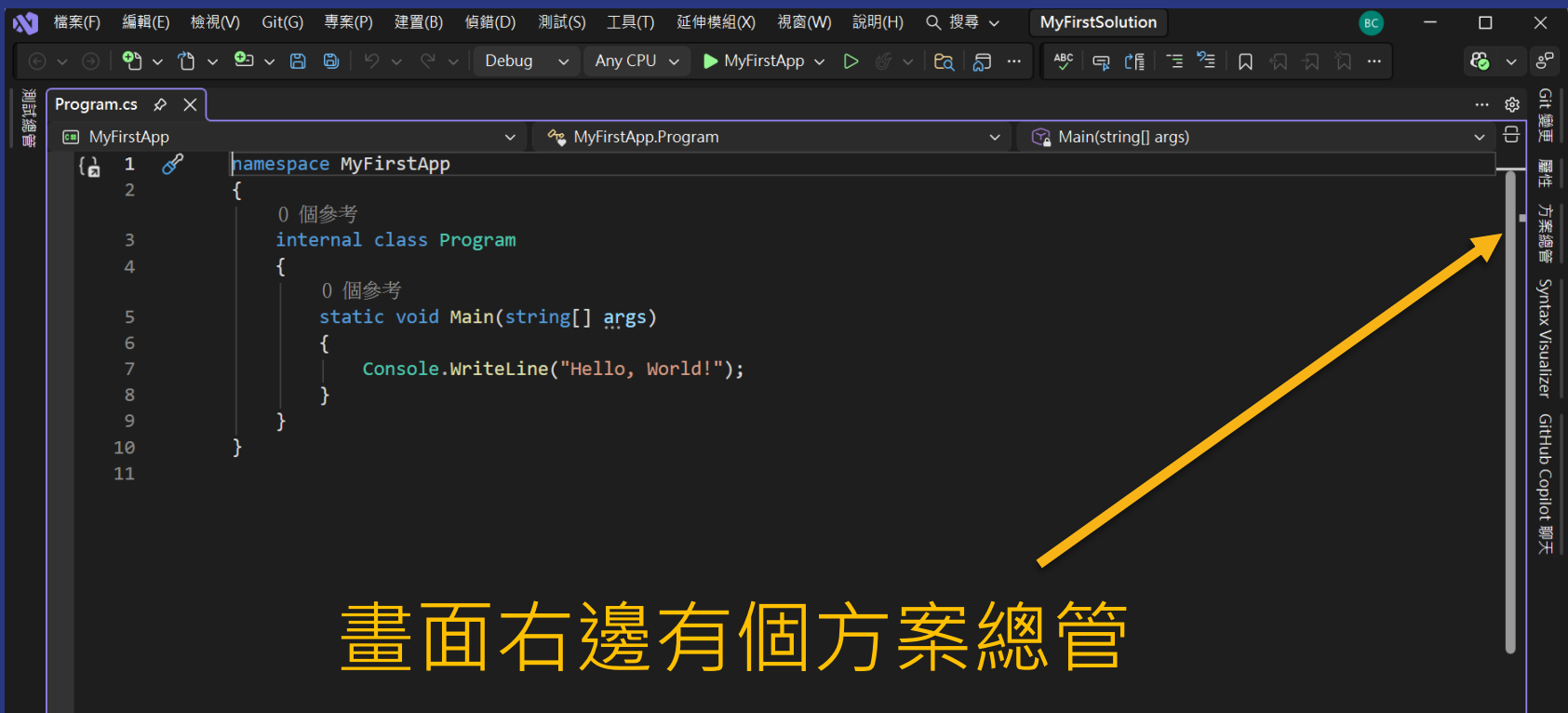
The screenshot shows the Visual Studio IDE with a C# project named "MyFirstApp". The code is written in "Program.cs" and is as follows:

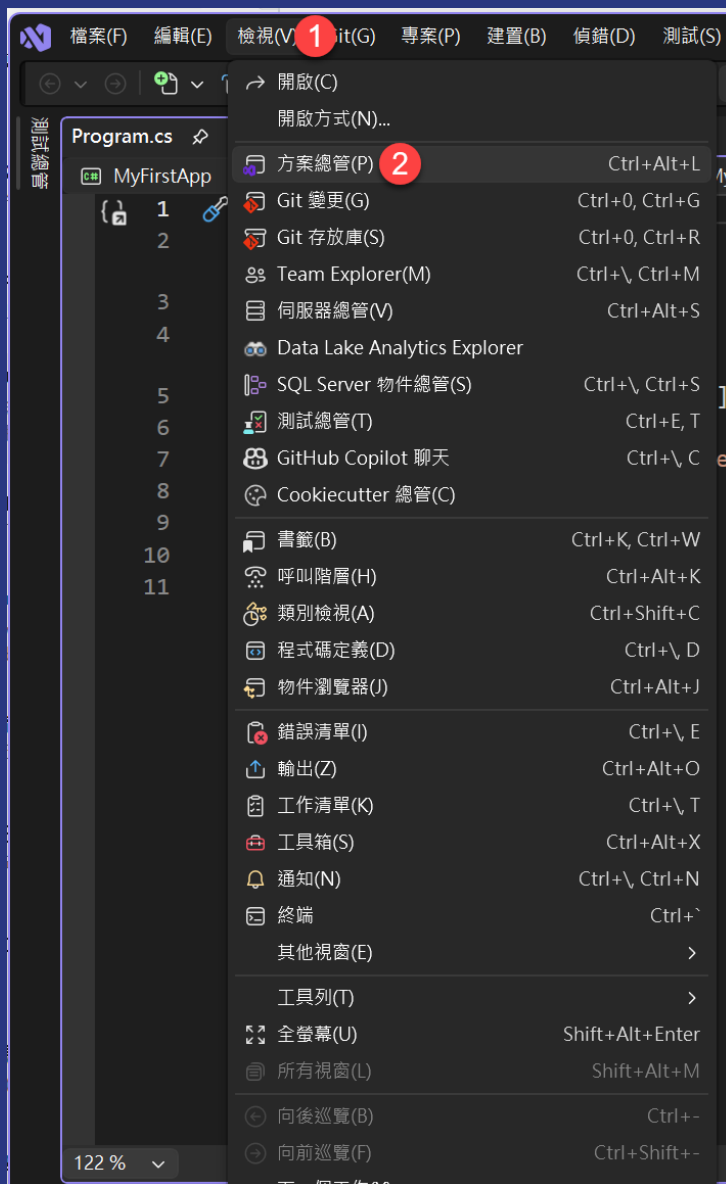
```
1 namespace MyFirstApp
2 {
3     0 個參考
4     internal class Program
5     {
6         0 個參考
7         static void Main(string[] args)
8         {
9             Console.WriteLine("Hello, World!");
10        }
11    }
```

The interface includes a menu bar at the top with options like "檔案(F)", "編輯(E)", "檢視(V)", "Git(G)", "專案(P)", "建置(B)", "偵錯(D)", "測試(S)", "工具(T)", "延伸模組(X)", "視窗(W)", and "說明(H)". Below the menu is a toolbar with icons for file operations, debugging, and running. The "Debug" dropdown is set to "Any CPU", and the "Run" button is highlighted. The solution explorer on the left shows the project structure, and the output window on the right is empty.

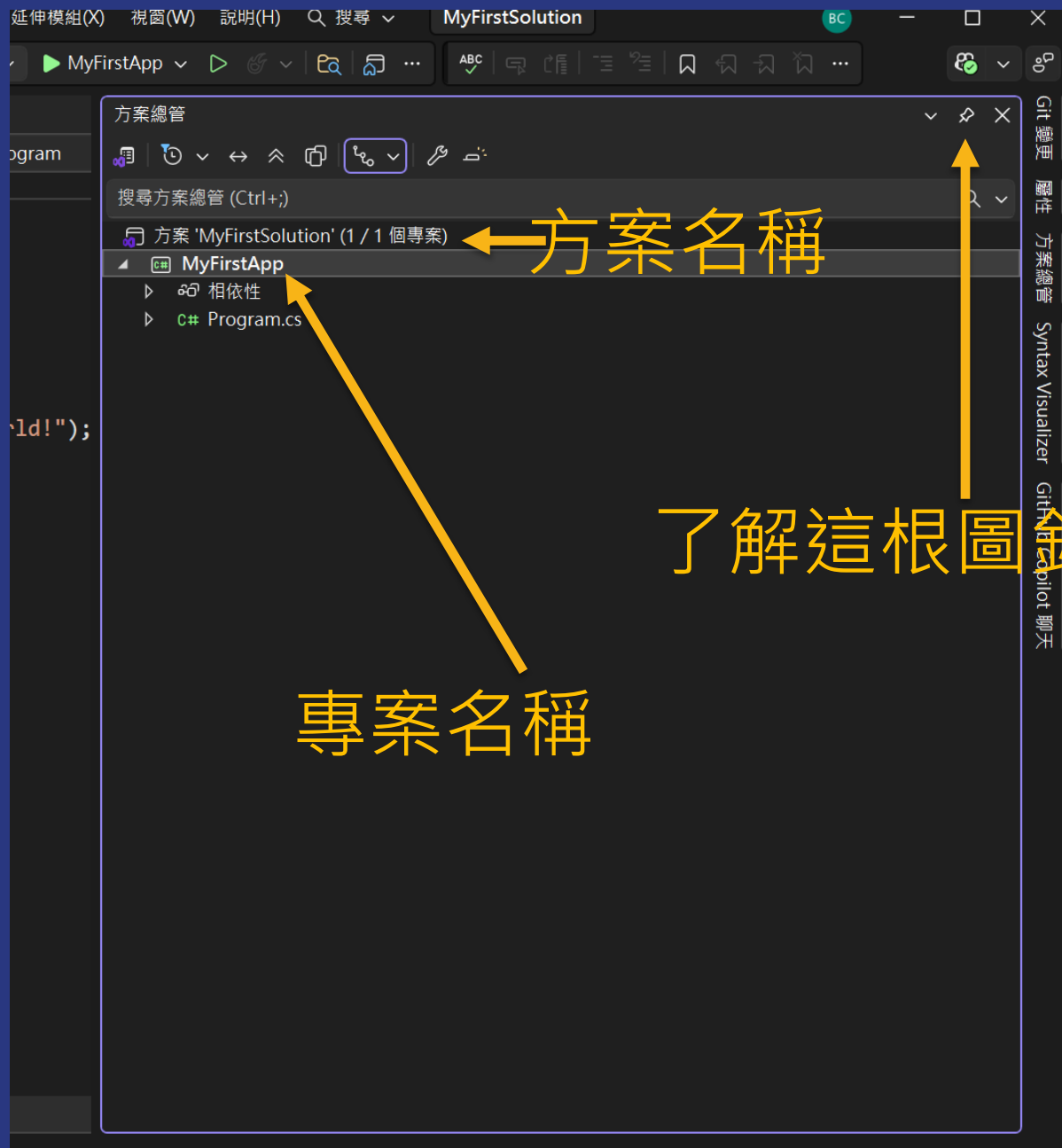
LAB

方案總管





如果沒有
找「檢視」
→「方案總管」



方案與專案

- 專案是一個可編譯的單位，包含：
 - 原始碼檔案（.cs）
 - 資源檔案
 - 設定檔
 - 參考的套件和組件
 - 編譯設定
- 一個專案會編譯成一個輸出檔案，例如：
 - 執行檔（.exe）
 - 類別庫（.dll）

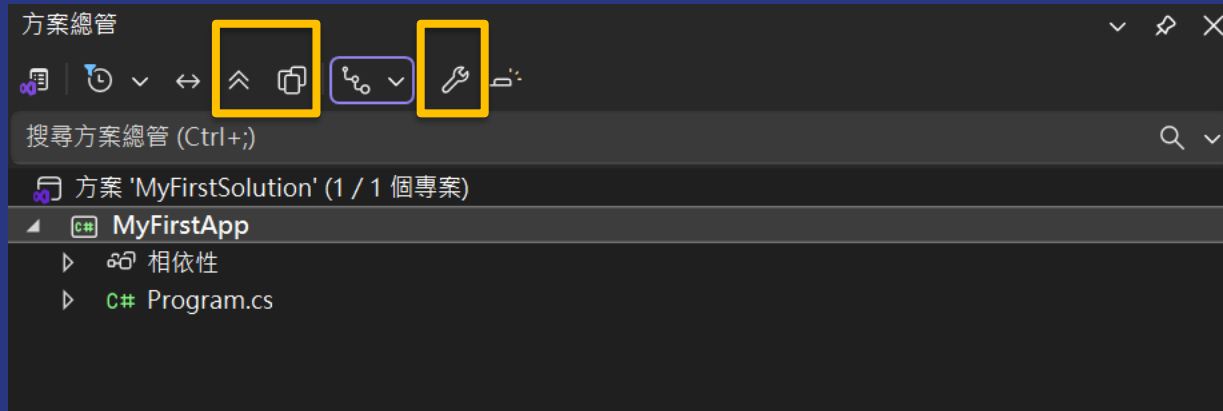
- **csproj** 檔案是 **XML** 格式，定義專案的結構和設定
 - 目標框架 (.NET 版本)
 - 專案類型 (Console App, Class Library, Web App 等)
 - 檔案清單
 - NuGet 套件參考
 - 類別庫參考
 - 專案間的參考關係

- 方案是一個容器，用來組織和管理多個相關的專案
 - 可以包含多個專案
 - 提供統一的編譯、偵錯環境
 - 管理專案間的相依性
 - 設定建置順序
- **slnx 檔案記錄 (VS2026 ~)**
 - 專案清單與路徑
 - 建置設定 (Debug/Release)
 - 專案間的關係
 - 簡化舊版 sln 不易修改與閱讀的問題

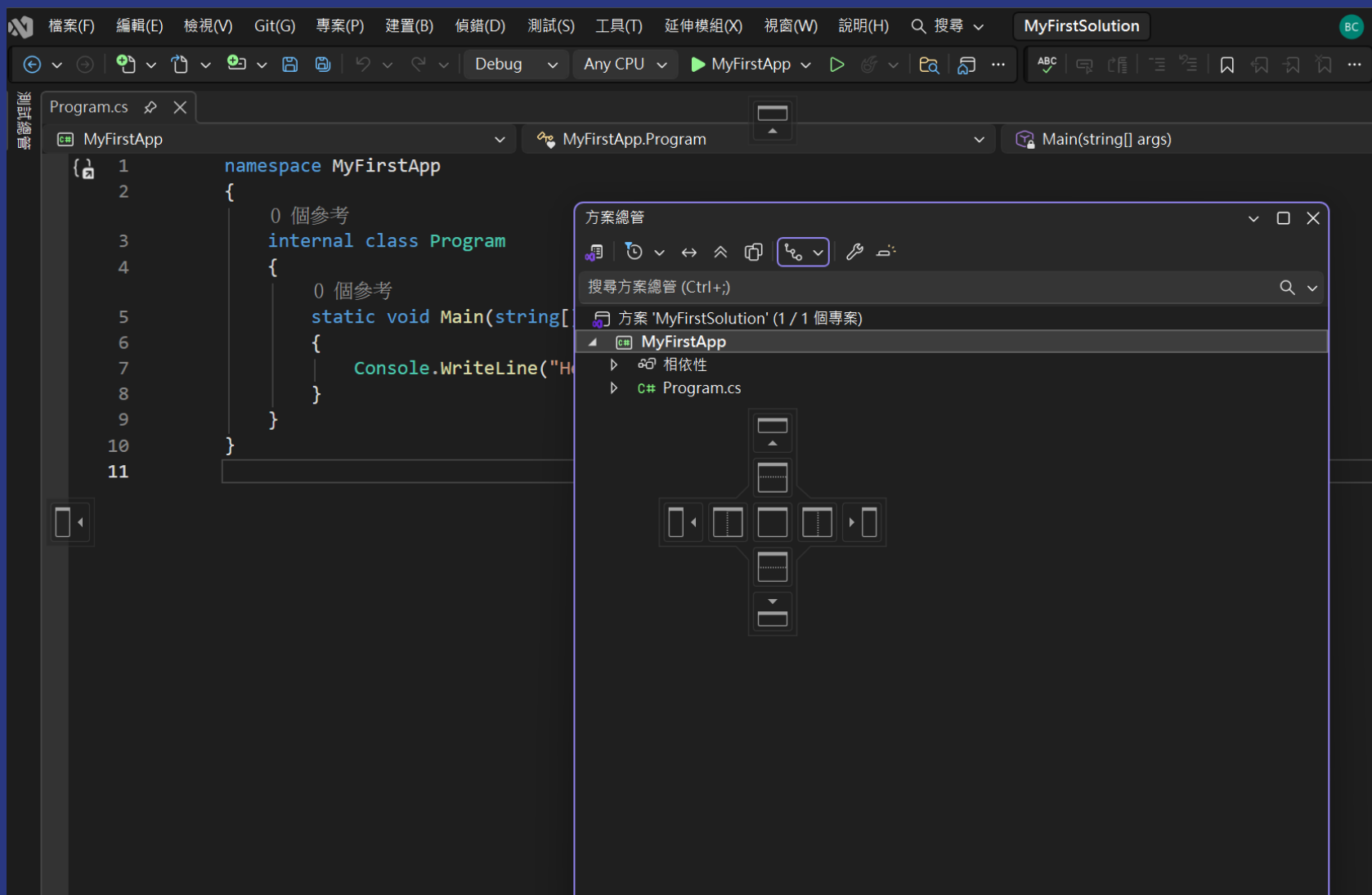
- sln 檔案記錄 (VS2003 ~ VS2022)

- 專案清單與路徑
- 建置設定 (Debug/Release)
- 專案間的關係

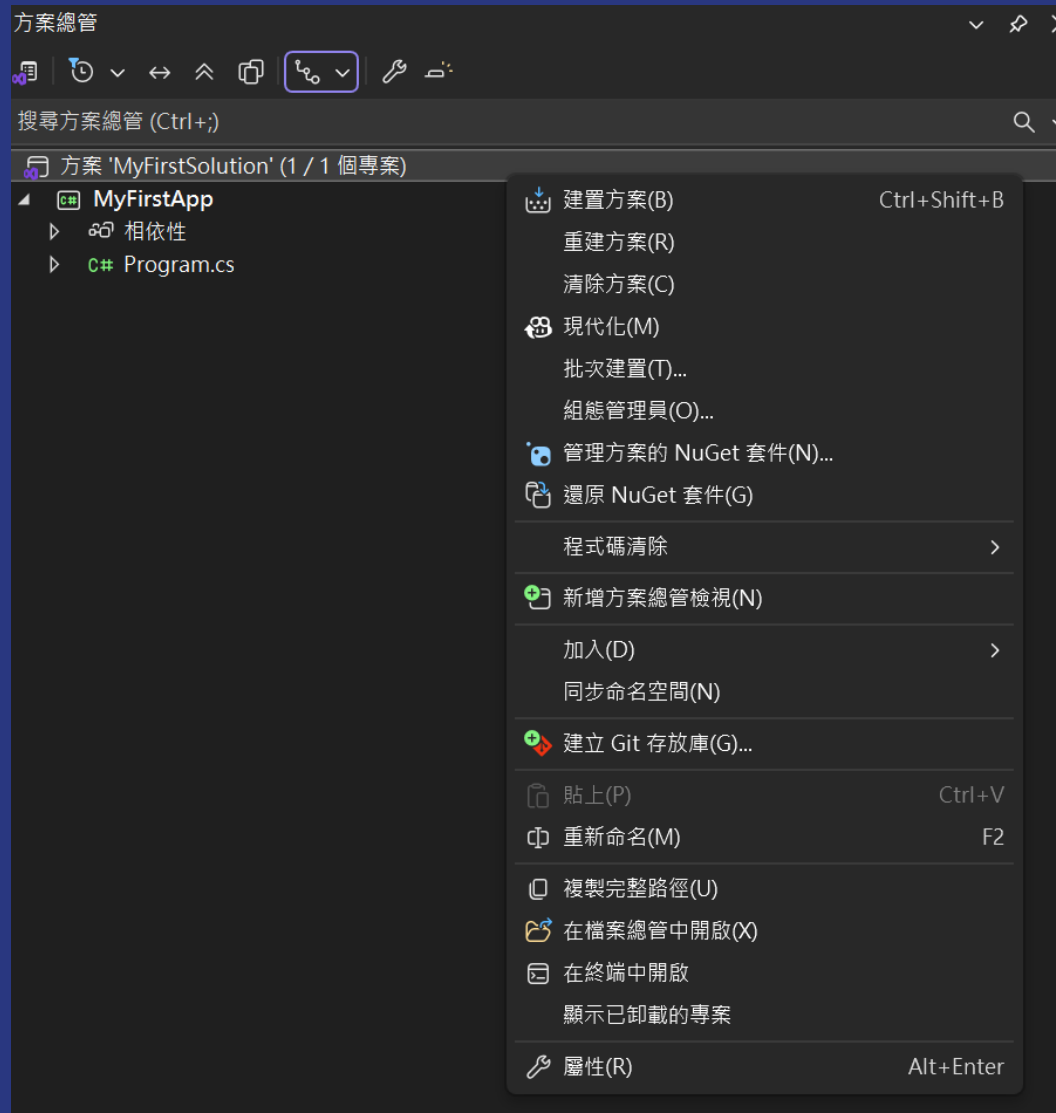
操作方案總管功能



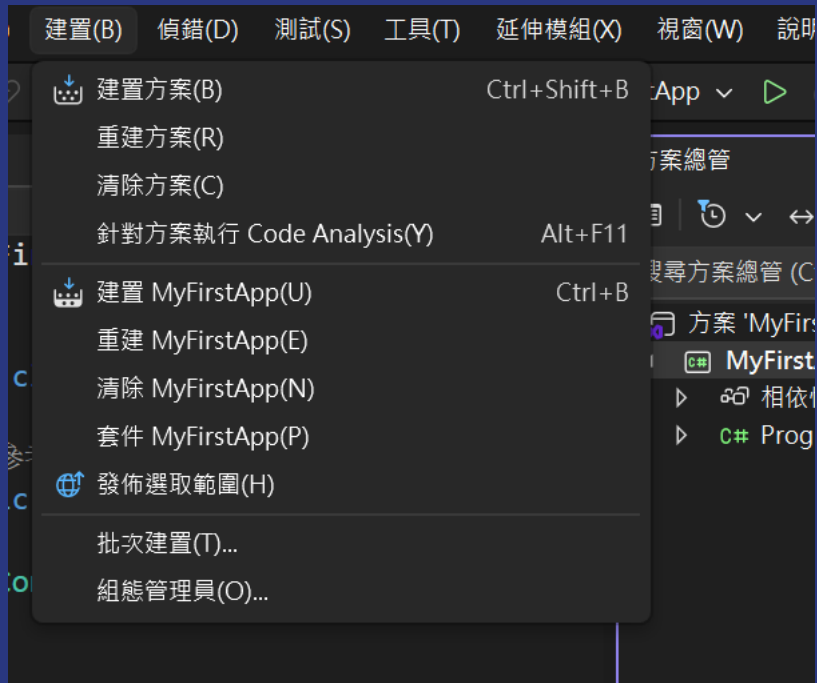
拖曳



開啟方案或專案的快顯功能表 (滑鼠右鍵)



建置方案或專案 (在方案總管使用快顯也可以)

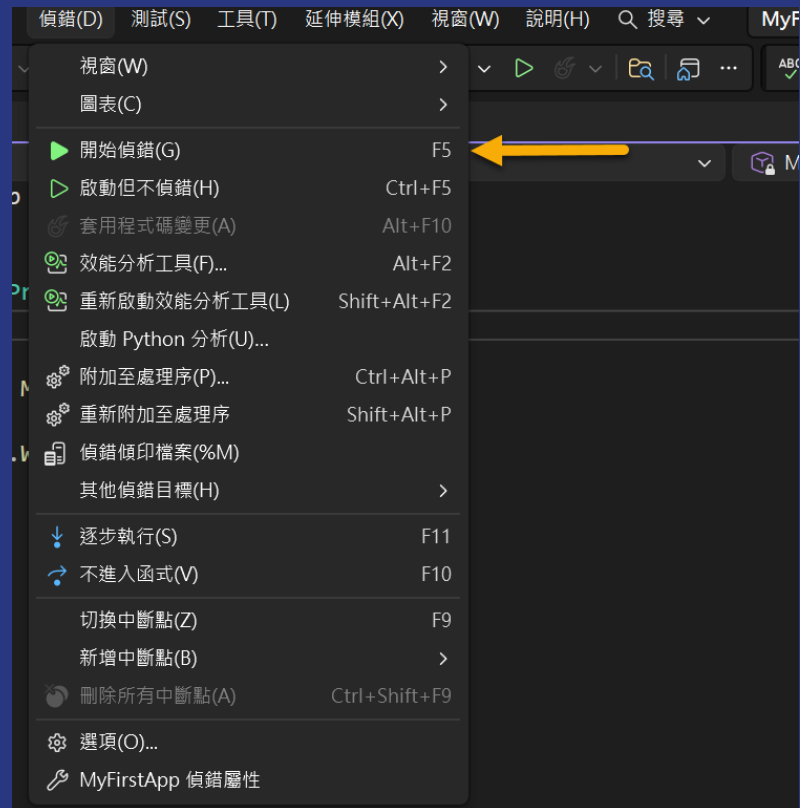


偵錯



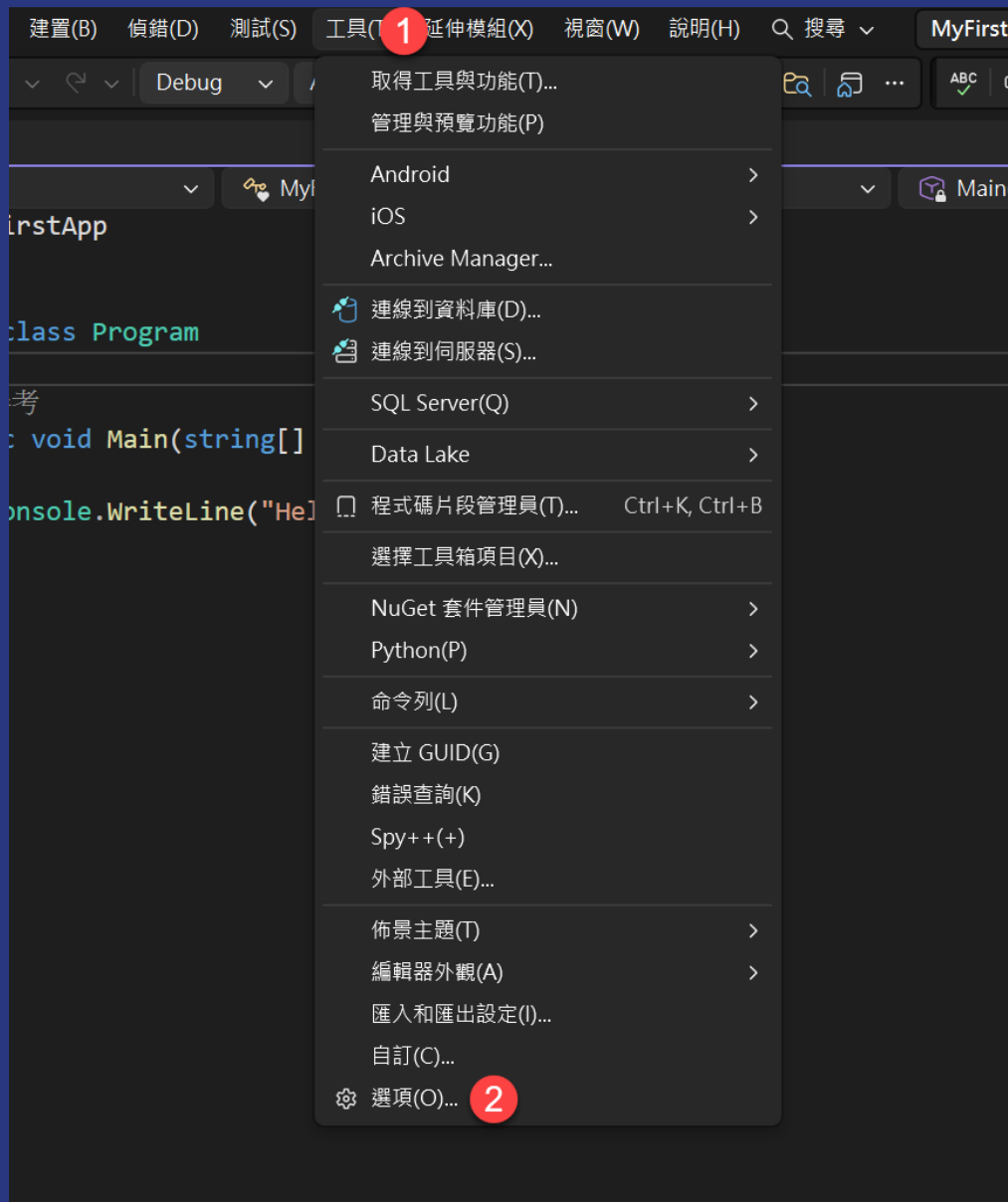
啟動但不偵錯

開始偵錯
(debug)



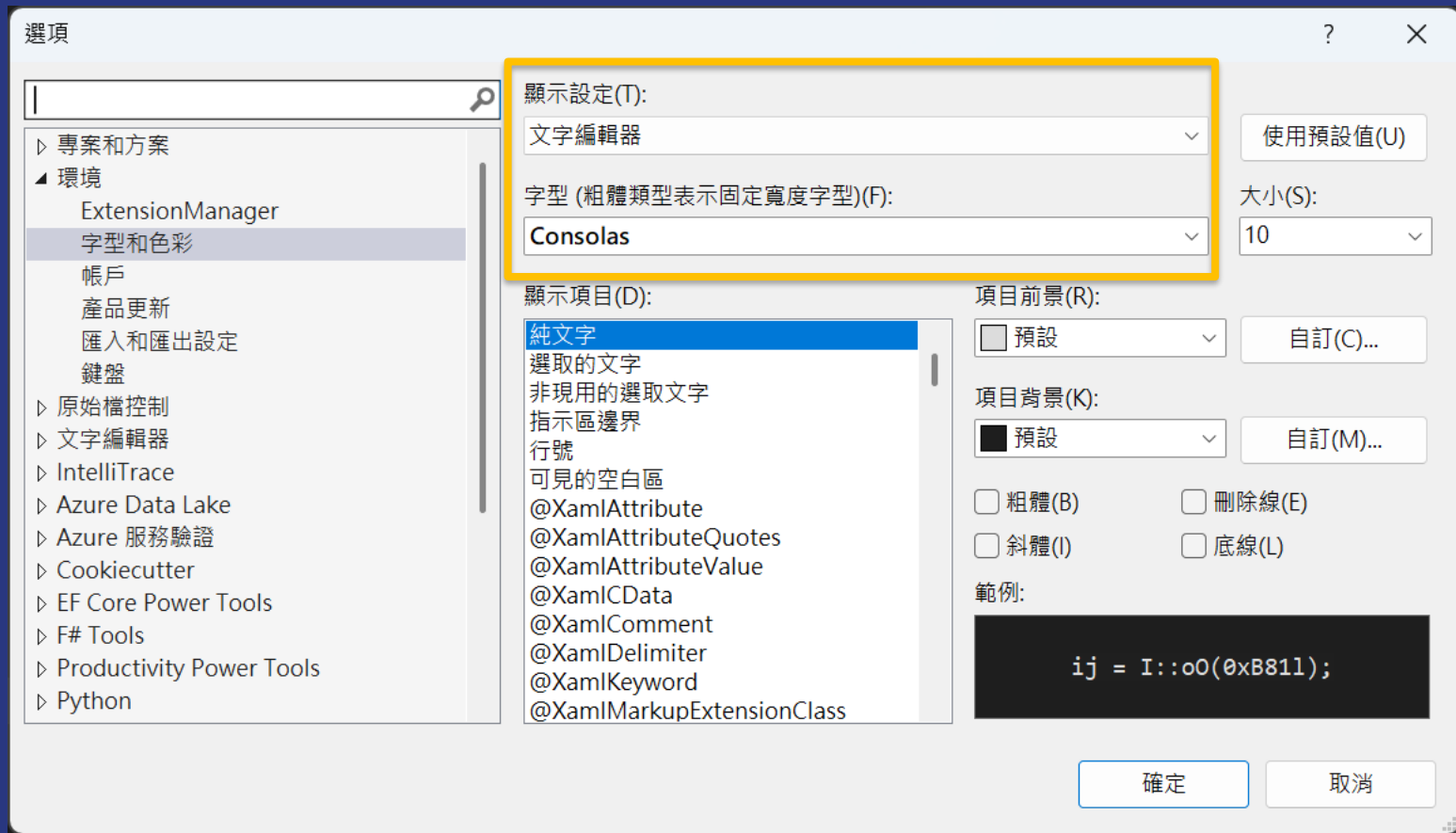
LAB

來改個好用的字型吧





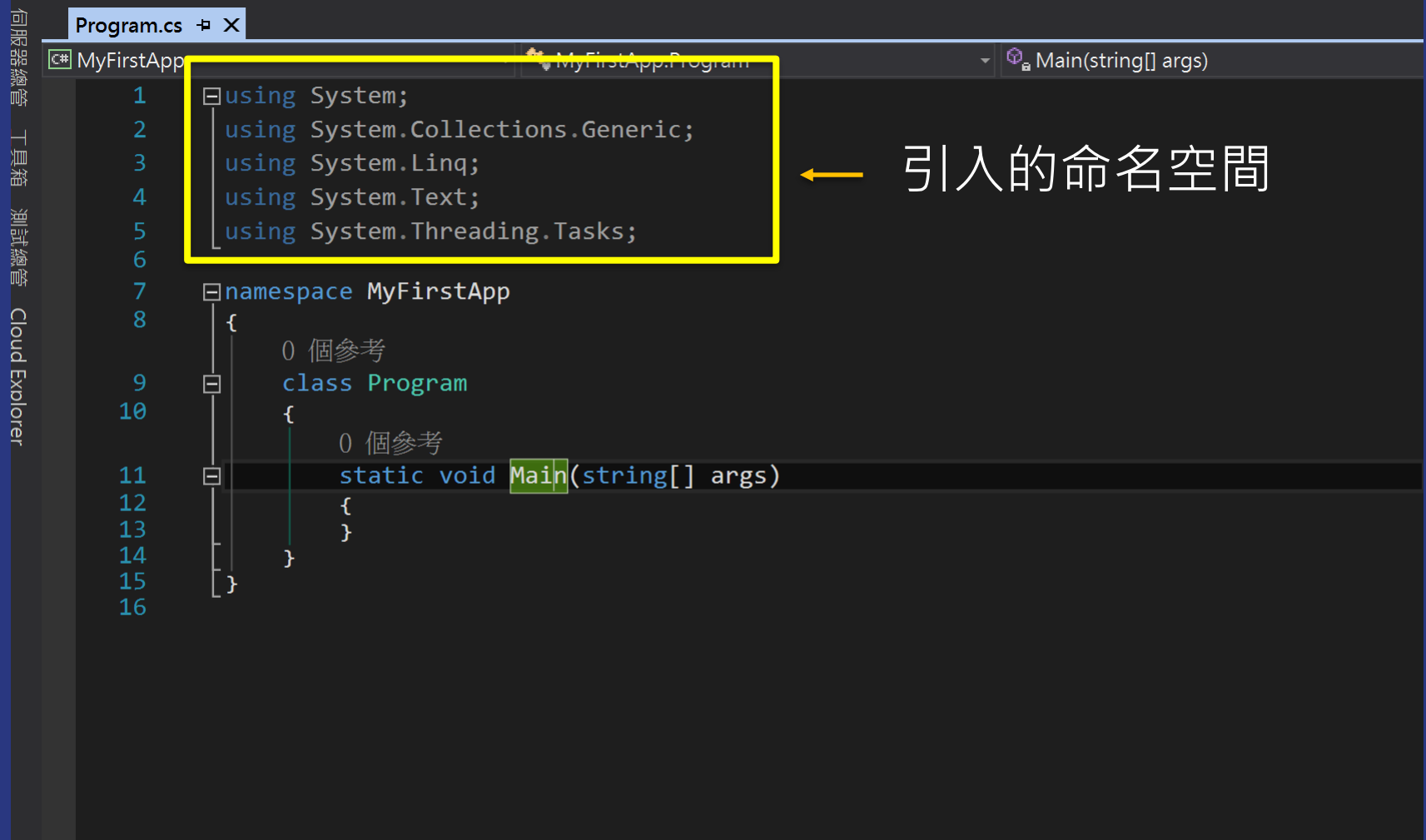
字型選 Consolas



LAB

認識程式碼檔案

本來應該是看到這樣



The screenshot shows a Visual Studio editor window with a C# file named `Program.cs`. The code is as follows:

```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace MyFirstApp
8 {
9     0 個參考
9     class Program
10    {
11        0 個參考
11        static void Main(string[] args)
12        {
13        }
14    }
15 }
16
```

A yellow rectangular box highlights the first five lines of code, which are the `using` statements for the `System`, `System.Collections.Generic`, `System.Linq`, `System.Text`, and `System.Threading.Tasks` namespaces. An orange arrow points from the Chinese text "引入的命名空間" (Introduced namespaces) to this box.

On the left side of the editor, there is a sidebar with the following text: 伺服器總管, 工具箱, 測試總管, Cloud Explorer.

但現在是這樣

```
Program.cs  X
MyFirstApp  MyFirstApp.Program  Main(string[] args)
1  namespace MyFirstApp
2  {
3      0 個參考
4      internal class Program
5      {
6          0 個參考
7          static void Main(string[] args)
8          {
9              Console.WriteLine("Hello, World!");
10         }
11     }
```

這差異怎麼來的？

- C# 10 新增一個功能 「global using」
 - 全域引用命名空間
 - 開啟這功能會有預設的命名空間被引用
 - 預設的內容會放在 專案\obj 下，
找到一個 「(專案名稱).GlobalUsings.g.cs」

讓我們把這功能關閉

- 在方案總管選擇專案 MyFirstApp
- 滑鼠左鍵雙擊，將會開啟 csproj
- 把 **ImplicitUsings** 設為 **disable**
- 把 **Nullable** 設為 **disable**

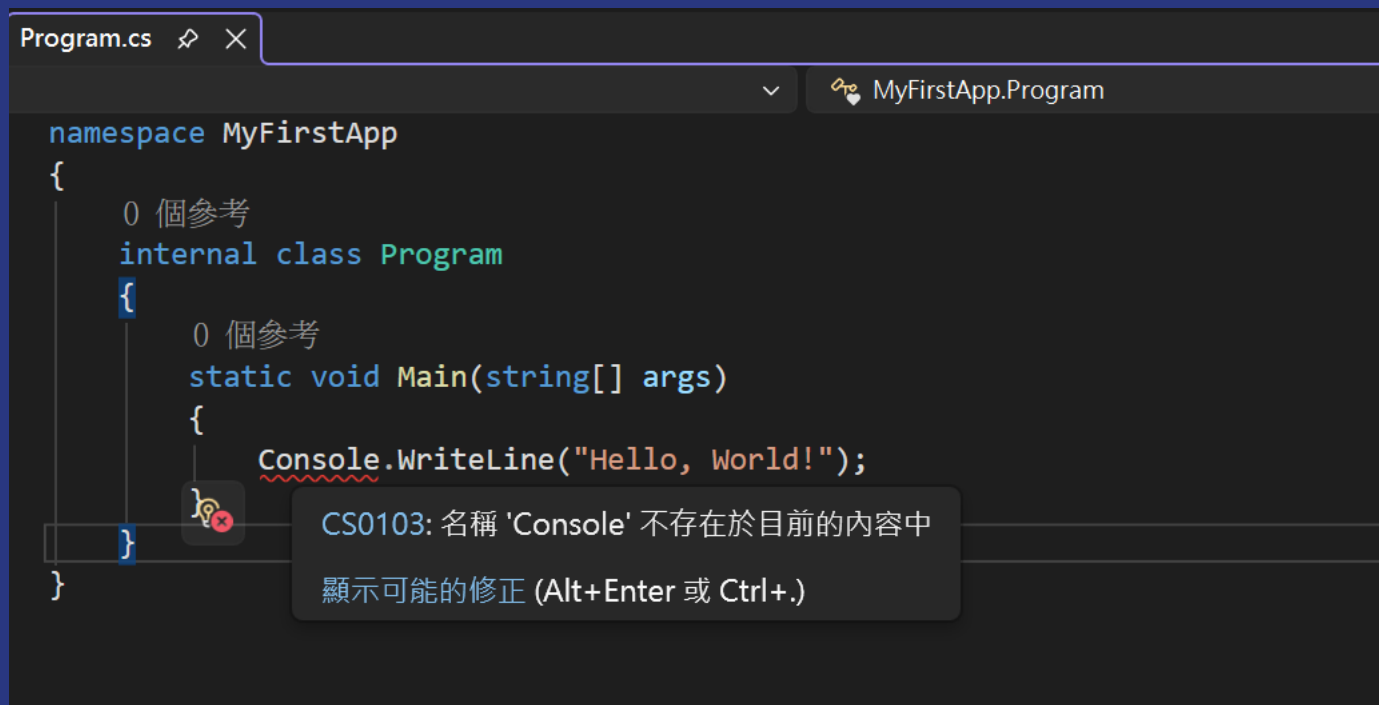
```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>disable</ImplicitUsings>
    <Nullable>disable</Nullable>
  </PropertyGroup>

</Project>
```

然後程式碼就出問題了

- 因為此時編譯器不知道 `Console` 是甚麼



```
Program.cs  MyFirstApp.Program

namespace MyFirstApp
{
    0 個參考
    internal class Program
    {
        0 個參考
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

CS0103: 名稱 'Console' 不存在於目前的內容中
顯示可能的修正 (Alt+Enter 或 Ctrl+.)

有兩個選擇

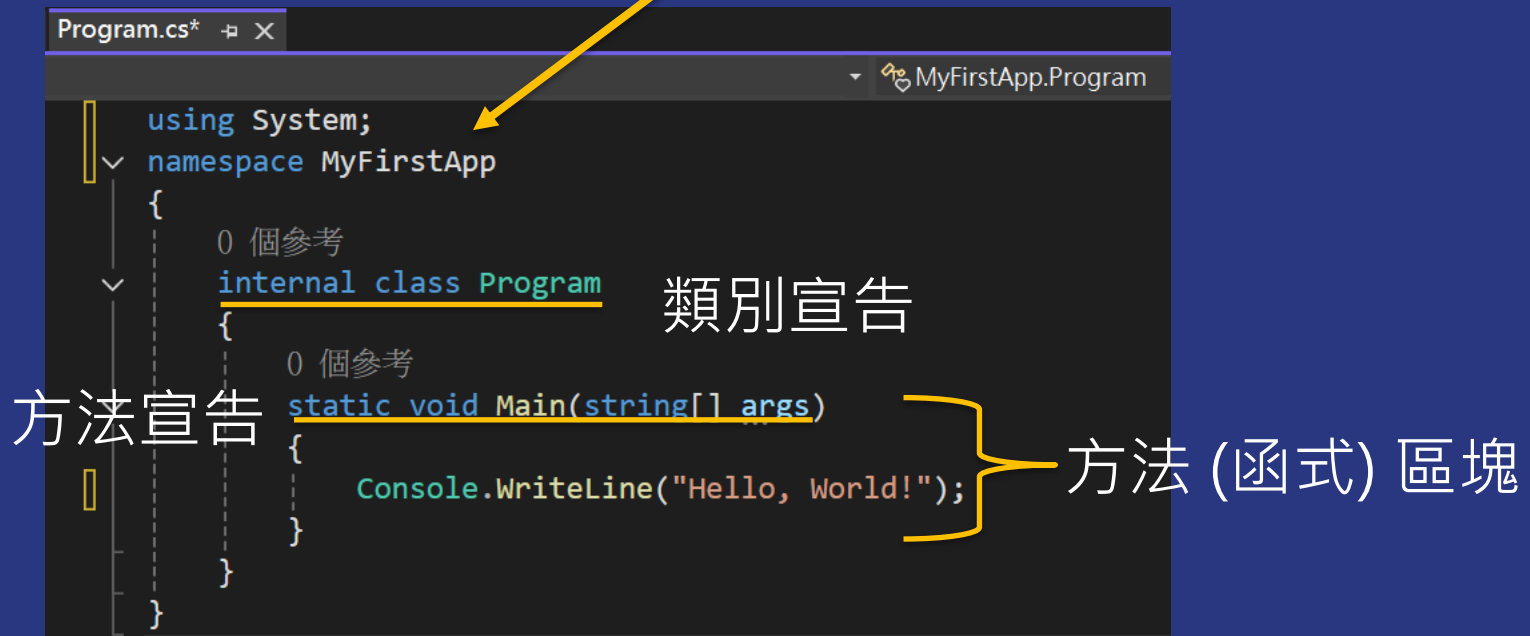
```
Program.cs*  X
MyFirstApp.Program

namespace MyFirstApp
{
    0 個參考
    internal class Program
    {
        0 個參考
        static void Main(string[] args)
        {
            System.Console.WriteLine("Hello, World!");
        }
    }
}
```

```
Program.cs*  X
MyFirstApp.Program

using System;
namespace MyFirstApp
{
    0 個參考
    internal class Program
    {
        0 個參考
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

此類別所在的命名空間



C# 的區塊

- 在 C# 中 { } 所在的區域代表一個區塊。
- 這個區塊有可能是一個
 - 命名空間區塊
 - 型別區塊
 - 方法區塊
 - 判斷式或流程敘述區塊
 - 其他敘述區塊

LAB

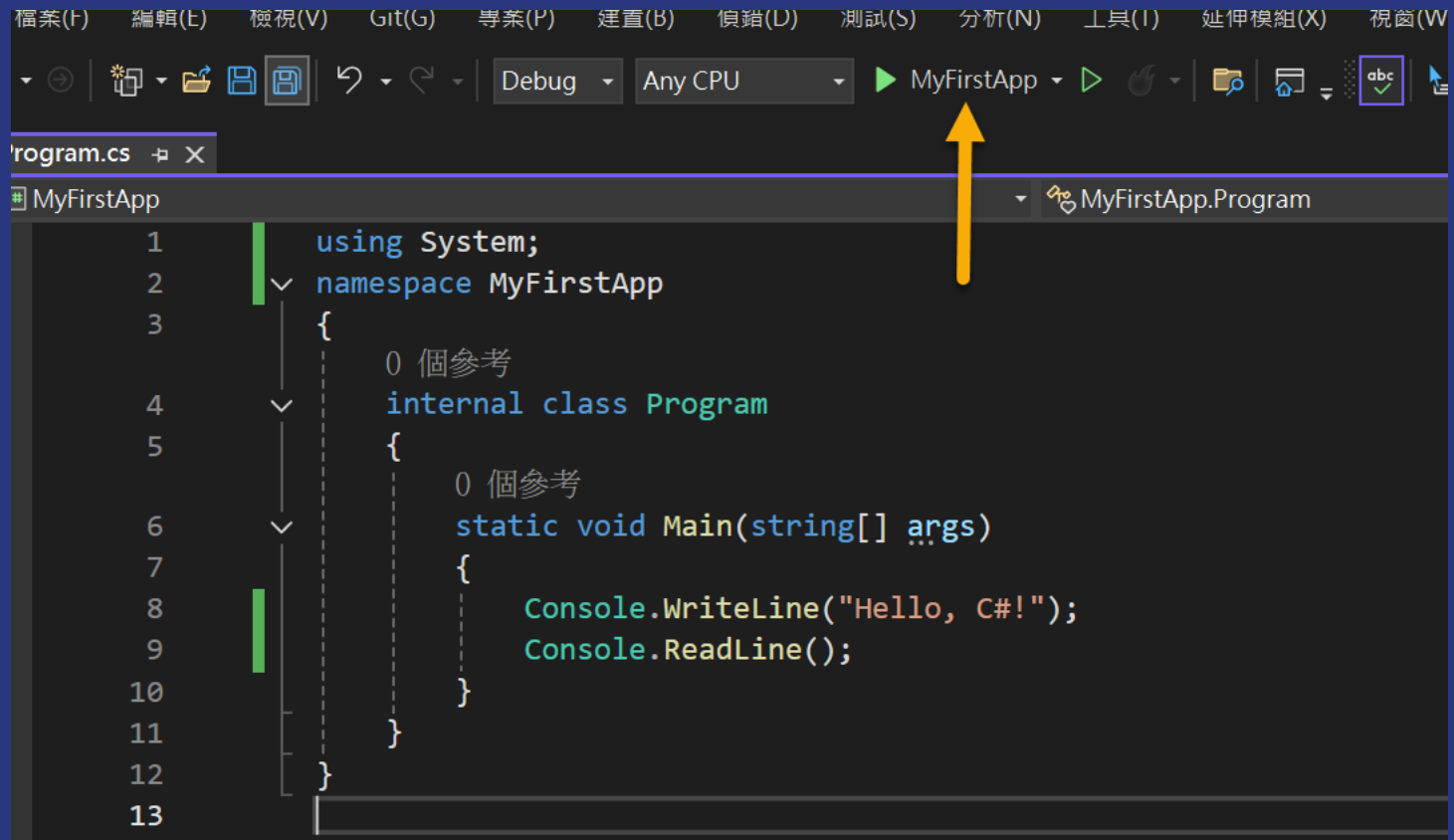
建立第一個程式

```
namespace MyFirstApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello C#");
            Console.ReadLine();
        }
    }
}
```



把 Main 內容改成這兩行程式碼

執行偵錯



執行偵錯的提示

- Debug / Release
- 輸出視窗
- 編譯後的檔案輸出位置

關於 Console 類別

- 它是一個內建於 .Net 裡的類別。
- 位在 System 命名空間內。
- 表示主控台應用程式 (Console Application) 的標準輸入、輸出和錯誤資料流。
- Microsoft Learn 連結 – [Console 類別](#)

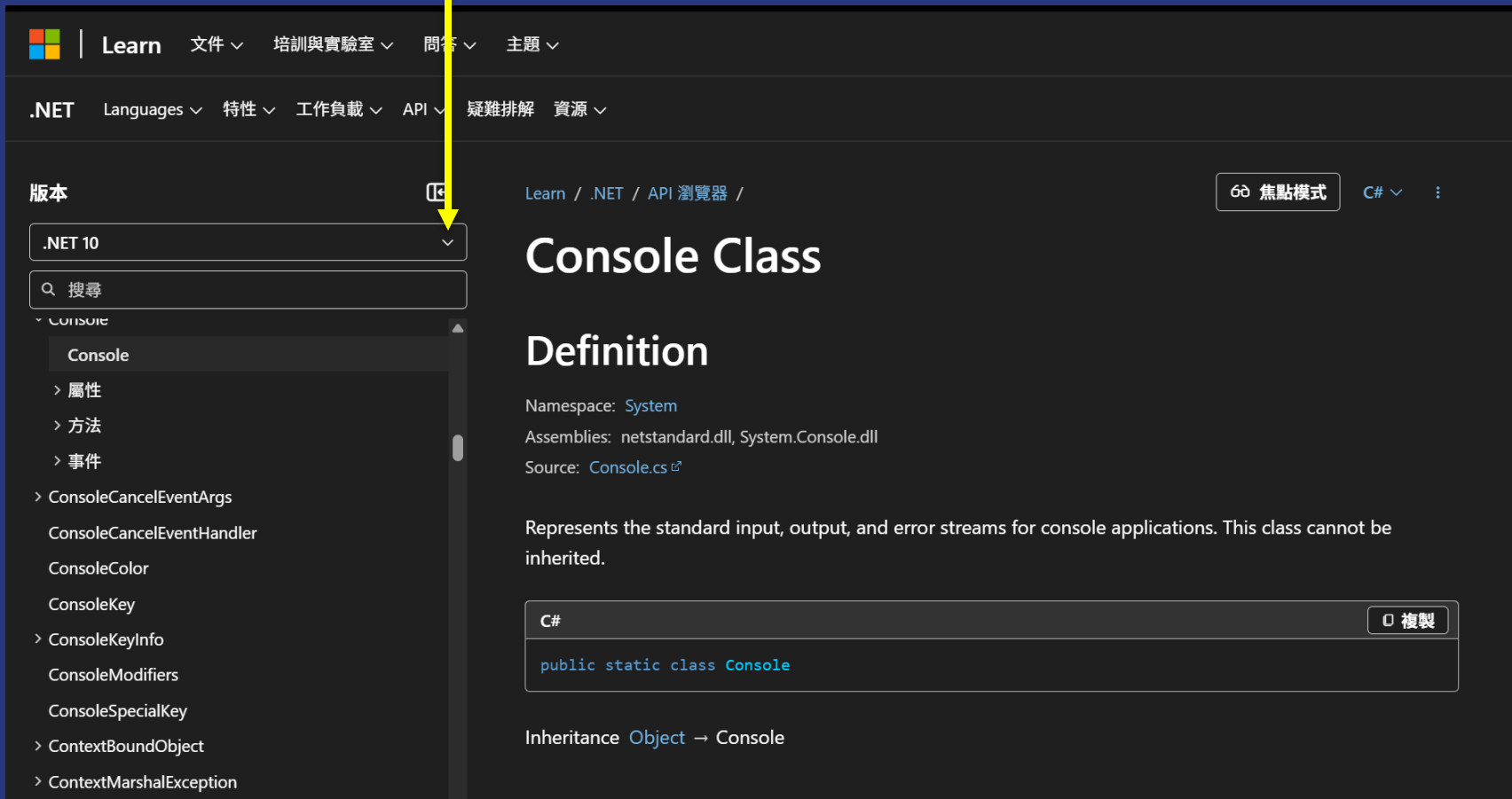
LAB

利用 Console 類別
練習閱讀 Microsoft learn 文件

前置步驟

- 開啟任何一種網路瀏覽器。
- 進到 `google.com`
- 在搜尋欄位中輸入『Console .NET』
- 在搜尋結果中選『Console 類別』或『Console Class』
- [Microsoft learn - .NET API browser](#)

選擇正確的框架版本



The screenshot shows the .NET API Browser interface. At the top, there is a navigation bar with the .NET logo and links for Learn, 文件 (Files), 培訓與實驗室 (Training and Labs), 問答 (Q&A), and 主題 (Topics). Below this is a secondary navigation bar with links for .NET, Languages, 特性 (Features), 工作負載 (Workloads), API, 疑難排解 (Troubleshooting), and 資源 (Resources).

On the left side, there is a sidebar titled "版本" (Version) with a dropdown menu currently set to ".NET 10". Below the dropdown is a search bar labeled "搜尋" (Search). The sidebar also contains a list of items under the "Console" category, including "Console", "屬性" (Properties), "方法" (Methods), "事件" (Events), "ConsoleCancelEventArgs", "ConsoleCancelEventHandler", "ConsoleColor", "ConsoleKey", "ConsoleKeyInfo", "ConsoleModifiers", "ConsoleSpecialKey", "ContextBoundObject", and "ContextMarshalException".

The main content area displays the "Console Class" definition. It includes the namespace "System", assemblies "netstandard.dll, System.Console.dll", and the source file "Console.cs". A description states: "Represents the standard input, output, and error streams for console applications. This class cannot be inherited." Below the description is a code block showing the C# definition:

```
public static class Console
```

. A "複製" (Copy) button is next to the code block. At the bottom, the inheritance path is shown: "Inheritance Object → Console".

修改語言代碼可以看到不同語言的內容，
例如 en-us 或是其他

The screenshot shows the Microsoft Learn .NET API browser interface. The browser tabs include "新分頁", ".NET API browser | Microsoft", and "Console Class (System) | Microsoft". The address bar shows the URL: `learn.microsoft.com/zh-tw/dotnet/api/system.console?view=net-10.0`. The page header features the "Microsoft Build 2026" banner with the dates "2026 年 6 月 2-3 日" and a "深入了解 >" button. Below the banner is a navigation bar with "Learn" and various categories like "文件", "培訓與實驗室", "問答", and "主題". The main navigation bar includes ".NET" and subcategories like "Languages", "特性", "工作負載", "API", "疑難排解", and "資源". The left sidebar shows the "版本" (Versions) section with ".NET 10" selected and a search bar. The main content area displays the "Console Class Definition" for the "System" namespace, including assemblies and source code. The right sidebar contains a "6d 焦點模式" button, a language dropdown menu (currently showing "C#"), and a list of actions: "閱讀英文", "新增至集合", "新增至計劃", "編輯", "共用方式為", "Facebook", "x.com", "LinkedIn", "電子郵件", "新增 Markdown", and "列印". A yellow arrow points to the language dropdown menu in the top right corner.

Microsoft Build 2026
2026 年 6 月 2-3 日

Learn 文件 培訓與實驗室 問答 主題

.NET Languages 特性 工作負載 API 疑難排解 資源

版本 .NET 10 搜尋

Console

Console

> 屬性

> 方法

> 事件

> ConsoleCancelEventArgs

ConsoleCancelEventHandler

ConsoleColor

ConsoleKey

> ConsoleKeyInfo

ConsoleModifiers

ConsoleSpecialKey

Learn / .NET / API 瀏覽器 /

Console Class

Definition

Namespace: `System`

Assemblies: `netstandard.dll`, `System.Console.dll`

Source: [Console.cs](#)

Represents the standard input, output, and error streams for console applications inherited.

C#

```
public static class Console
```

6d 焦點模式 C#

- 閱讀英文
- 新增至集合
- 新增至計劃
- 編輯

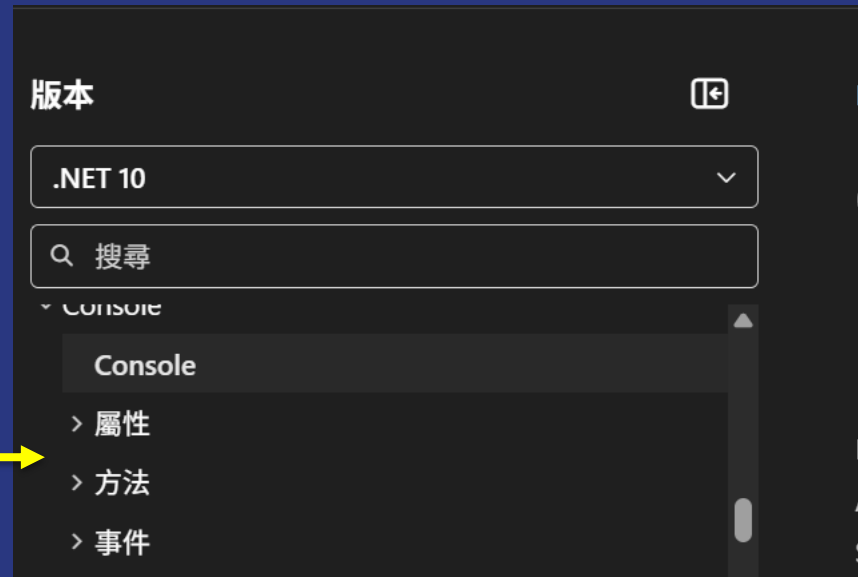
共用方式為

- Facebook
- x.com
- LinkedIn
- 電子郵件

<> 新增 Markdown

🖨 列印

側邊欄可以
展開它的成員



所在的命名空間與組件名稱

The screenshot shows the .NET API Browser for the **Console Class**. Annotations include:

- A yellow arrow pointing from the top-left to the **Definition** section.
- A yellow arrow pointing from the top-right to the **C#** dropdown menu, with the text "指定範例程式語言" (Specify example programming language).
- A yellow arrow pointing from the right to the `public static class Console` line in the code block, with the text "宣告" (Declaration).
- A yellow arrow pointing from the right to the inheritance path `Object → Console`, with the text "繼承樹" (Inheritance tree).
- A yellow rectangular box around the **Remarks** section.

Console Class

Definition

Namespace: [System](#)
Assemblies: [netstandard.dll](#), [System.Console.dll](#)
Source: [Console.cs](#)

Represents the standard input, output, and error streams for console applications. This class cannot be inherited.

C# 複製

```
public static class Console
```

Inheritance [Object](#) → [Console](#)

Remarks

For more information about this API, see [Supplemental API remarks for Console](#).

註：切換到 .NET 9 中文會比較多

右側欄可以選擇內容標籤

Properties

展開資料表

Name	Description
Background Color	Gets or sets the background color of the console.
BufferHeight	Gets or sets the height of the buffer area.
BufferWidth	Gets or sets the width of the buffer area.
CapsLock	Gets a value indicating whether the CAPS LOCK keyboard toggle is turned on or turned off.
CursorLeft	Gets or sets the column position of the cursor within the buffer area.
CursorSize	Gets or sets the height of the cursor within a character cell.
CursorTop	Gets or sets the row position of the cursor within the buffer area.
CursorVisible	Gets or sets a value indicating whether the cursor is visible.
Error	Gets the standard error output stream.
ForegroundColor	Gets or sets the foreground color of the console.
In	Gets the standard input stream.
InputEncoding	Gets or sets the encoding the console uses to read input.
IsError Redirected	Gets a value that indicates whether the error output stream has been redirected from the standard error stream.
IsInput Redirected	Gets a value that indicates whether input has been redirected from the standard input stream.
IsOutput Redirected	Gets a value that indicates whether output has been redirected from the standard output stream.

本文內容

- Definition
- Remarks
- Properties**
- Methods
- Events
- Applies to
- Thread Safety

此頁面對您有幫助嗎？

選擇右側欄的「方法 (Methods)」， 找到 WriteLine(string) 並點選

WriteLine(Double)	Writes the text representation of the specified double-precision floating-point value, followed by the current line terminator, to the standard output stream.
WriteLine(Int32)	Writes the text representation of the specified 32-bit signed integer value, followed by the current line terminator, to the standard output stream.
WriteLine(Int64)	Writes the text representation of the specified 64-bit signed integer value, followed by the current line terminator, to the standard output stream.
WriteLine(Object)	Writes the text representation of the specified object, followed by the current line terminator, to the standard output stream.
WriteLine(ReadOnlySpan<Char>)	
WriteLine(Single)	Writes the text representation of the specified single-precision floating-point value, followed by the current line terminator, to the standard output stream.
WriteLine(String, Object, Object, Object)	Writes the text representation of the specified objects, followed by the current line terminator, to the standard output stream using the specified format information.
WriteLine(String, Object, Object)	Writes the text representation of the specified objects, followed by the current line terminator, to the standard output stream using the specified format information.
WriteLine(String, Object)	Writes the text representation of the specified object, followed by the current line terminator, to the standard output stream using the specified format information.
WriteLine(String, Object[])	Writes the text representation of the specified array of objects, followed by the current line terminator, to the standard output stream using the specified format information.
WriteLine(String, ReadOnlySpan<Object>)	Writes the text representation of the specified span of objects, followed by the current line terminator, to the standard output stream using the specified format information.
WriteLine(String) 2	Writes the specified string value, followed by the current line terminator, to the standard output stream.
WriteLine(UInt32)	Writes the text representation of the specified 32-bit unsigned integer value, followed by the current line terminator, to the standard output stream.

本文內容

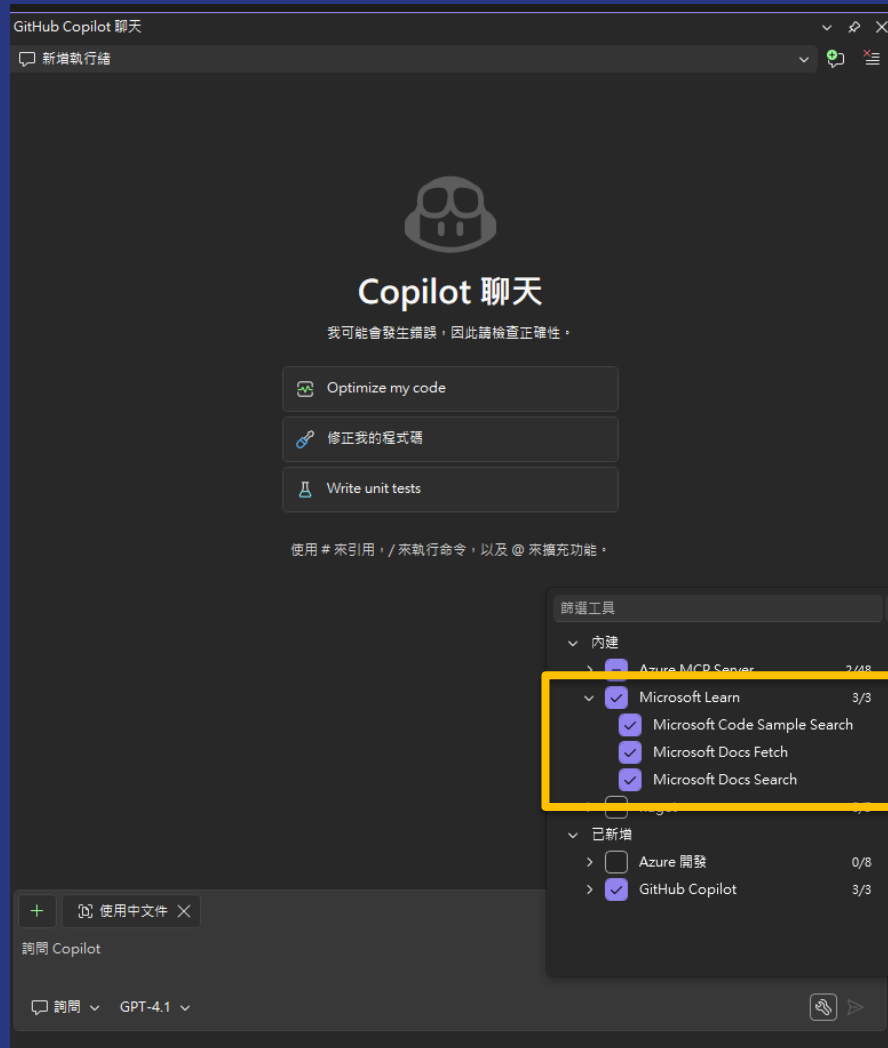
- Definition
- Remarks
- Properties
- 1 | Methods**
- Events
- Applies to
- Thread Safety

此頁面對您有幫助嗎？

LAB

利用 Console 類別
練習 github copilot chat 的
Microsoft Learn agents

啟用 agent



代理功能

- **Microsoft Code Sample Search**

- 尋找官方程式碼範例

- **Microsoft Docs Fetch**

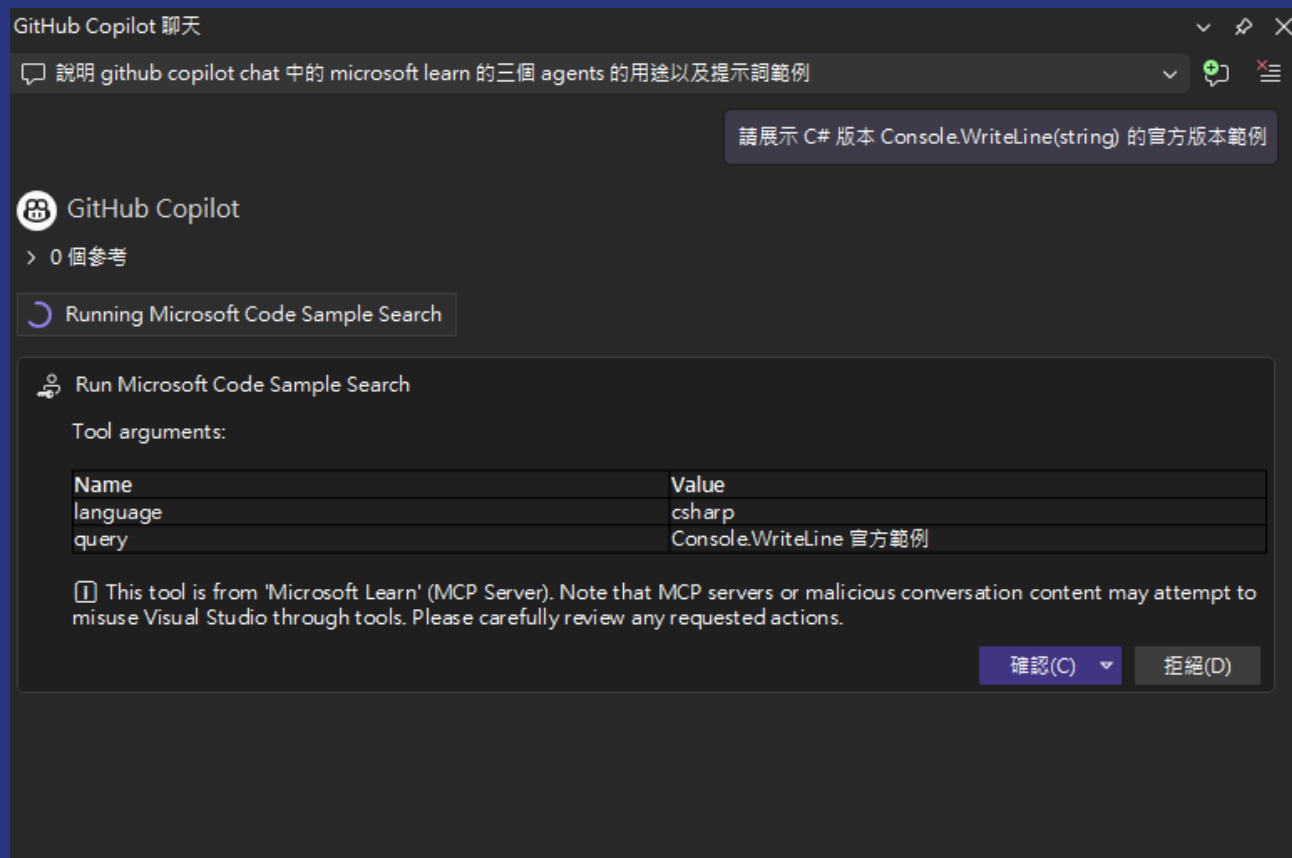
- 取得官方文件的完整內容（需提供明確的官方文件網址）

- **Microsoft Docs Search**

- 搜尋官方文件的相關內容（非程式碼，偏重於說明、步驟、概念等）

Microsoft Code Sample Search

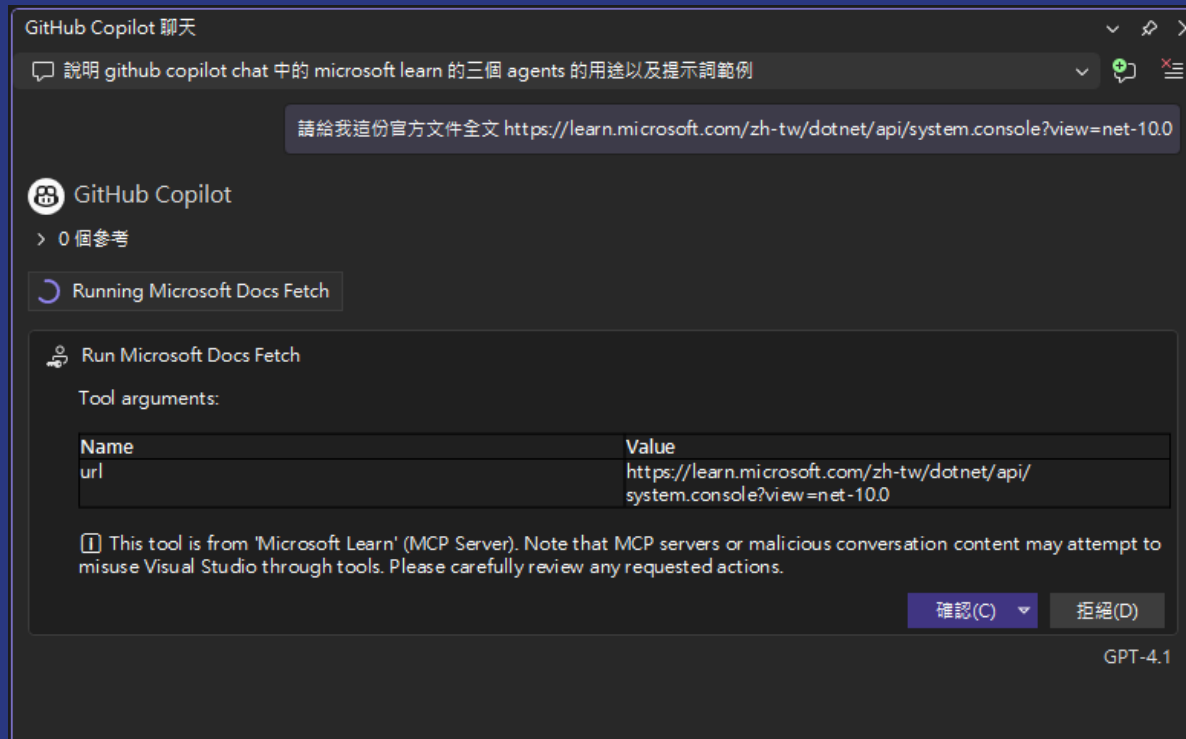
- 「請展示 C# 版本 Console.WriteLine(string) 的官方版本範例」



Microsoft Docs Fetch

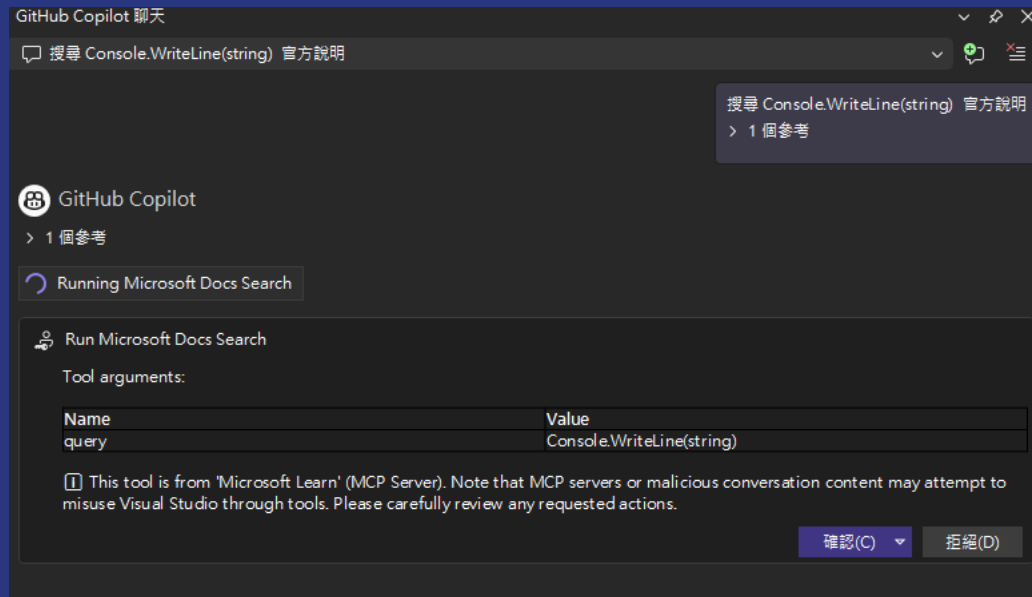
- 「請給我這份官方文件全文

<https://learn.microsoft.com/zh-tw/dotnet/api/system.console?view=net-10.0>」



Microsoft Docs Search

- 「搜尋 Console.WriteLine(string) 官方說明」



試試看

- copilot 建立一個新的對話，提示詞「請給我官方關於 Console 類別文件全文」

進入 C#

變數宣告

- 變數表示某個數值、字串值或其他型別的物件指向。
變數所儲存的值可能會變更，但名稱仍會相同。
- 宣告方式：*變數型別 變數名稱;*

變數宣告的範例

```
int i;  
string s;  
byte b = 12;  
string v = "abc";
```



這個等號被稱為
指派運算子

C# 內建型別

C# 型別	.NET Framework 型別
<u>bool</u>	System.Boolean
<u>byte</u>	System.Byte
<u>sbyte</u>	System.SByte
<u>char</u>	System.Char
<u>decimal</u>	System.Decimal
<u>double</u>	System.Double
<u>float</u>	System.Single
<u>int</u>	System.Int32
<u>uint</u>	System.UInt32
<u>long</u>	System.Int64
<u>ulong</u>	System.UInt64
<u>object</u>	System.Object
<u>short</u>	System.Int16
<u>ushort</u>	System.UInt16
<u>string</u>	System.String

Signed
與
Unsigned

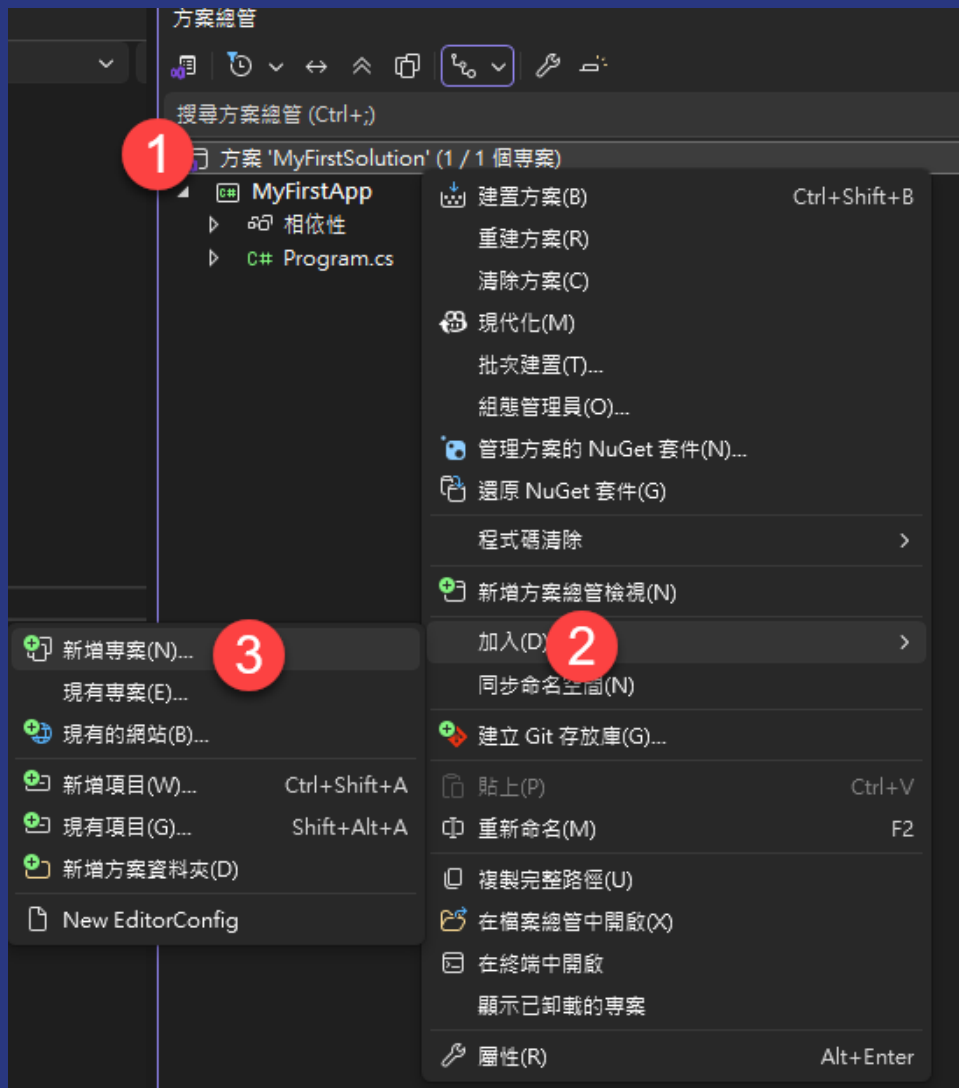
參考：[Microsoft learn-內建類型資料表 \(C# 參考\)](#)

提示詞：抓取官方 "C# 內建型別" 文件

LAB

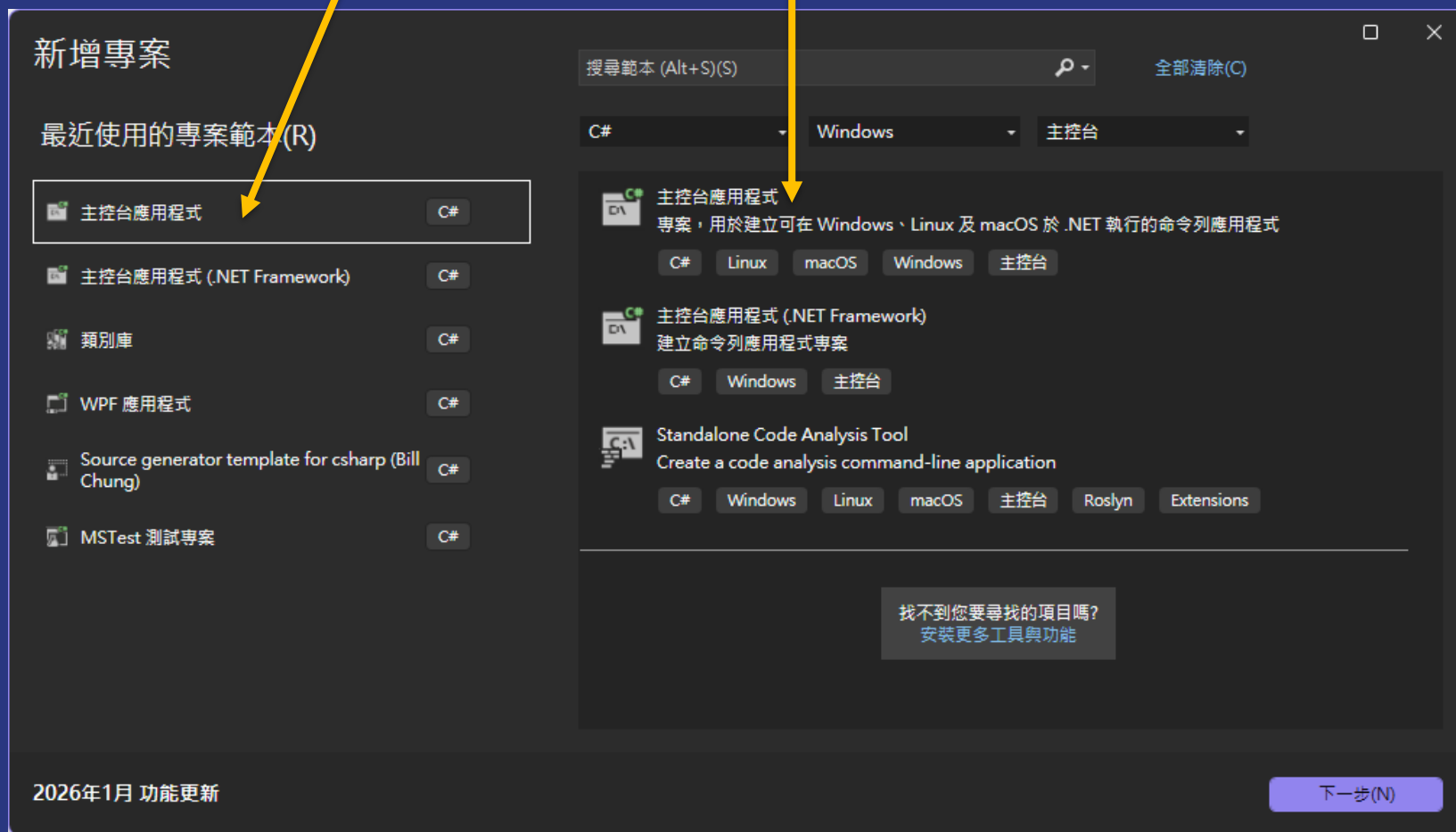
讀取並顯示

在MyFirstSolution 方案中加入一個新的 Console 專案 名稱就叫 ReadThenDisplay



選擇方案名稱
按下【滑鼠右鍵】
→ 【加入】
→ 【新增專案】

可以從左邊「最近使用」或右邊的全部範本搜尋
選擇範本



設定新的專案

主控台應用程式

C#

Linux

macOS

Windows

主控台

專案名稱(N)

ReadThenDisplay

位置(L)

C:\BuildSchool\台北\TPE2026_summer\samples\CH1\MyFirstSolution

專案 將在「C:\BuildSchool\台北\TPE2026_summer\samples\CH1\MyFirstSolution
\ReadThenDisplay\」中建立

輸入專案名稱「ReadThenDisplay」
按下【下一步】

2026年1月 功能更新

上一步(B)

下一步(N)

其他資訊

主控台應用程式

C#

Linux

macOS

Windows

主控台

架構(F) ⓘ

.NET 10.0 (長期支援)

☐ 啟用容器支援(E) ⓘ

容器作業系統(O) ⓘ

Linux

容器組建類型 ⓘ

Dockerfile

☒ 不要使用最上層陳述式(T) ⓘ

☐ 啟用 native AOT 發佈 ⓘ

(1) 選擇架構版本 .NET 10.0

(2) 請勿勾選

(3) 勾選「不要使用最上層陳述式」(通常第一次勾選後就會預設)

(4) 請勿勾選

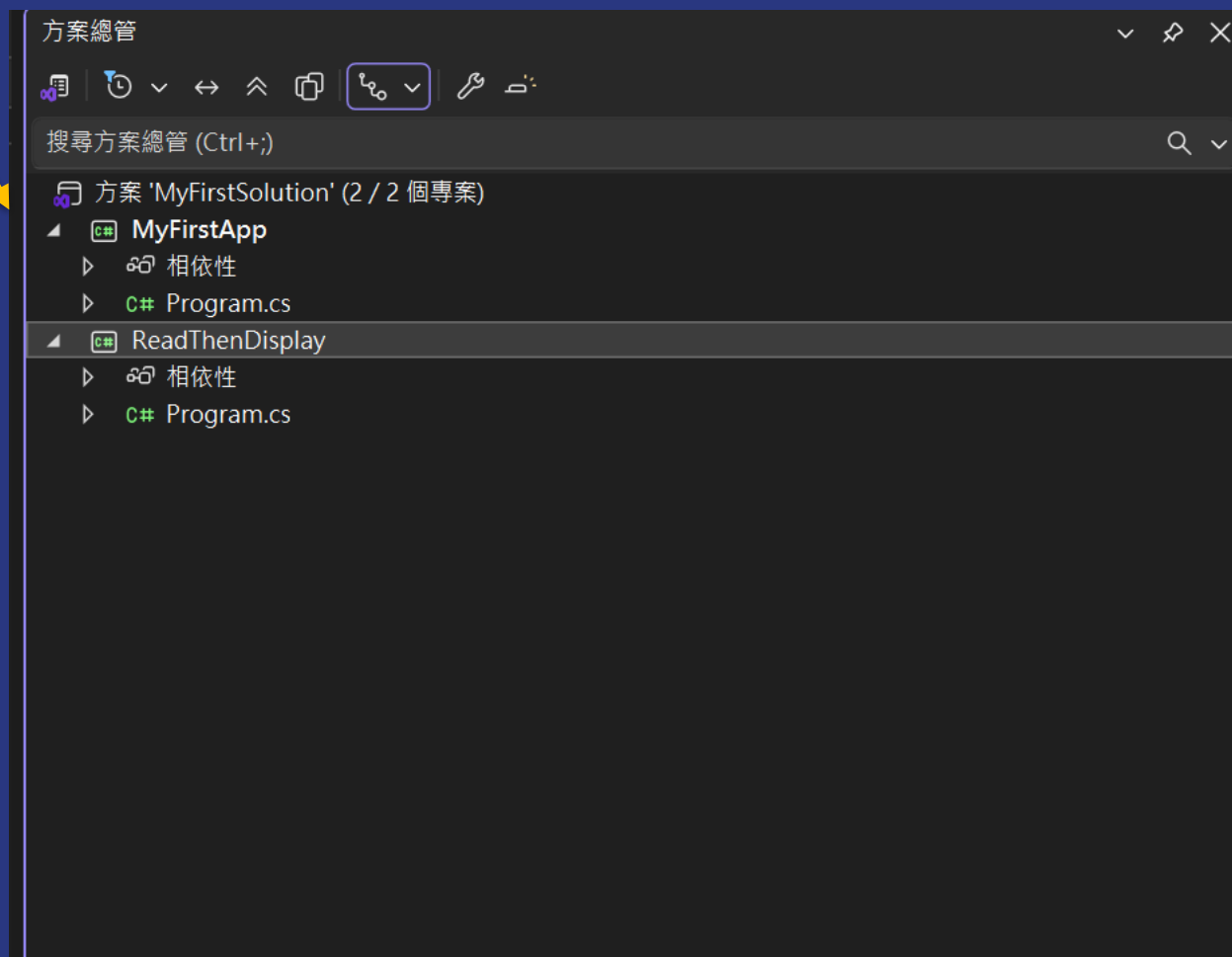
-- 確認好資訊後按下「建立」

2026年1月 功能更新

上一步(B)

建立(C)

觀察方案總管下現在是不是有兩個專案



開啟 csproj，將 nullable 設定為 disable

```
<Project Sdk="Microsoft.NET.Sdk">

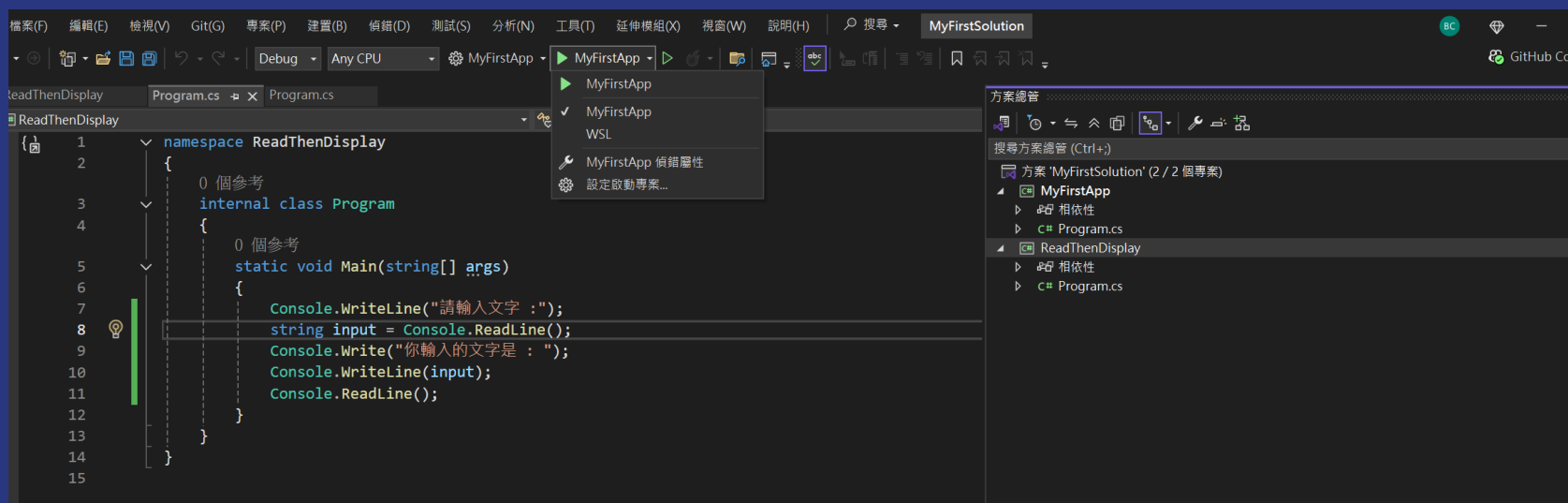
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>disable</Nullable>
  </PropertyGroup>

</Project>
```

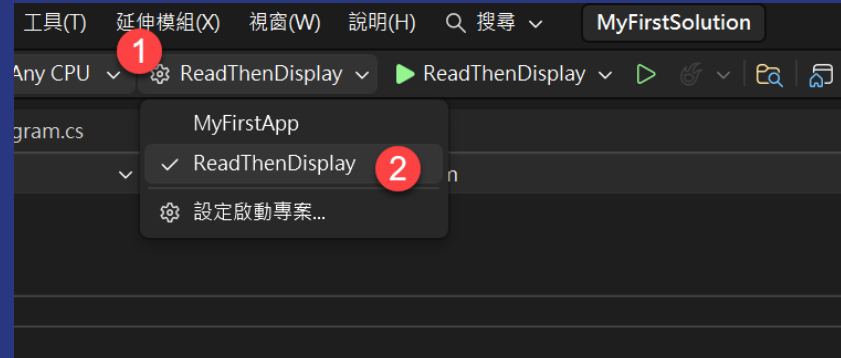
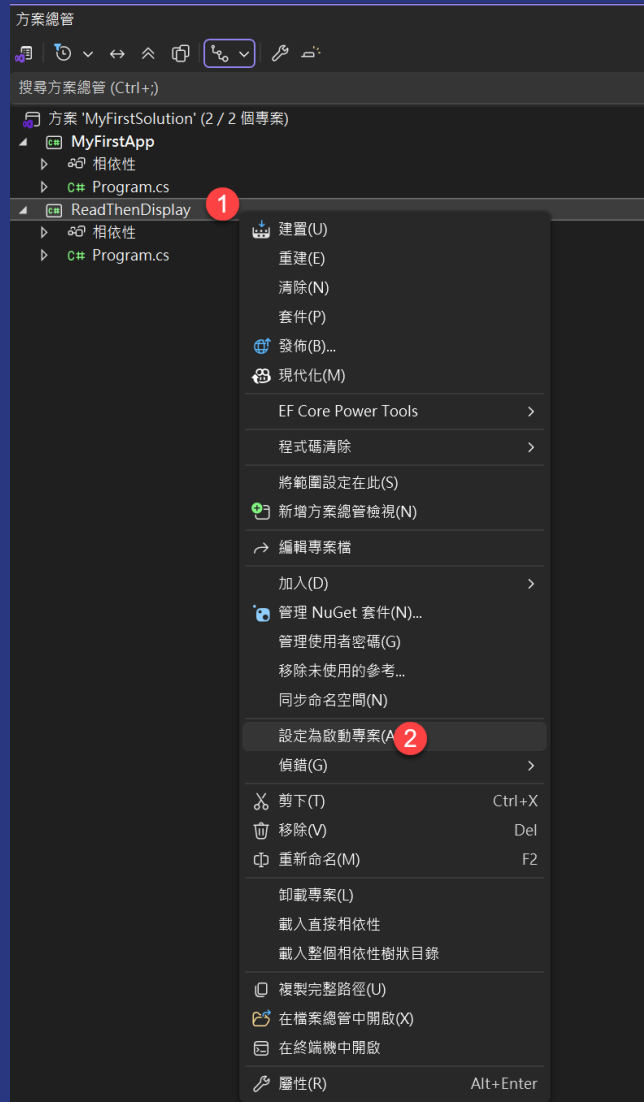
修改 Main 方法區塊內的程式碼

```
namespace ReadThenDisplay
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("請輸入文字 :");
            string input = Console.ReadLine();
            Console.Write("你輸入的文字是 : ");
            Console.WriteLine(input);
            Console.ReadLine();
        }
    }
}
```

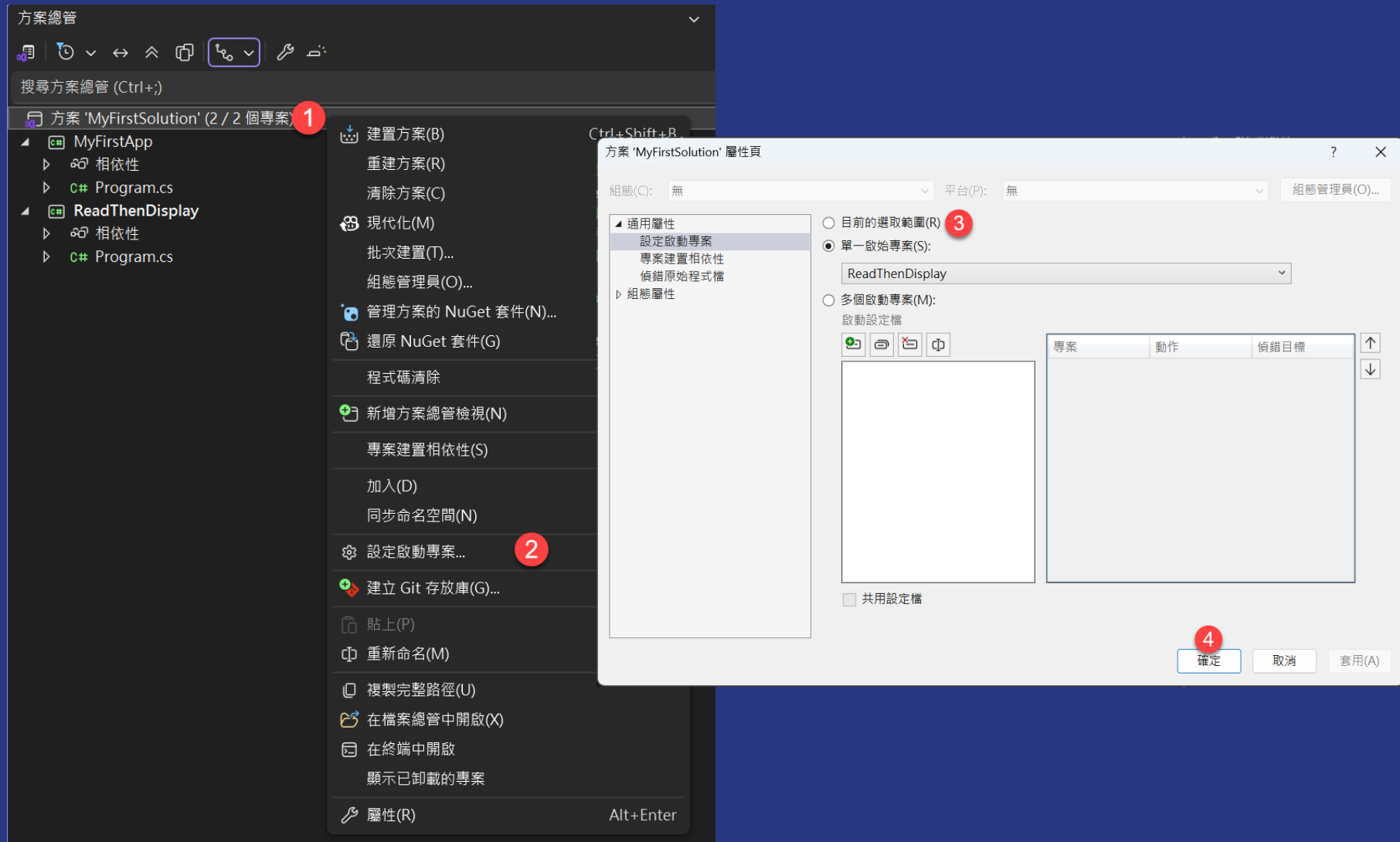
偵錯選項目前還是 MyFirstApp，
這是因為 MyFirstApp 是目前方案的「啟動專案」



解決方式 1: 強制設定 ReadThenDisplay 為啟動專案



解決方式 2： 設定 「目前的選取範圍」為啟動專案



程式碼註解

三種註解

- 單行註解 `//`
 - 適合簡短的說明或臨時註解
- 多行註解 `/* */`
 - 可以跨越多行，適合大段落的說明或臨時註解掉大塊程式碼
- XML 註解 `///`
 - 使用 XML 標籤格式
 - 可以被 Visual Studio 等 IDE 解析，提供智能提示
 - 可以生成 API 文件
 - 通常用於類別、方法、屬性等成員的正式文件

```
/// <summary>
/// 這是讀取輸入並顯示的範例程式的主要類別。
/// </summary>
internal class Program
{
    /// <summary>
    /// 這個 Main 方法是程式的入口點。
    /// </summary>
    /// <param name="args">程式啟動參數</param>
    static void Main(string[] args)
    {
        // 顯示提示訊息，要求使用者輸入文字
        Console.WriteLine("請輸入文字 :");
        /* 這裡使用 Console.ReadLine() 方法來讀取使用者輸入的文字。
           使用者輸入的文字會被存儲在 input 變數中。
           然後使用 Console.WriteLine() 方法來顯示使用者輸入的文字。
        */
        string input = Console.ReadLine();
        Console.Write("你輸入的文字是 : ");
        Console.WriteLine(input);
        Console.ReadLine();
    }
}
```

選擇陳述式、關係和比較運算子

C# 中的關係運算子

- 比較運算式的結果是 **bool** (**System.Boolean**) 型別
- **X > Y**
 - 表示左邊的 X 若大於 Y 則為 true (真)，反之為 false (假)。
- **X < Y**
 - 表示左邊的 X 若小於 Y 則為 true (真)，反之為 false (假)。
- **X >= Y**
 - 表示左邊的 X 若大於或等於 Y 則為 true (真)，反之為 false (假)。
- **X <= Y**
 - 表示左邊的 X 若小於 Y 則為 true (真)，反之為 false (假)。

C# 中的相等比較運算子

- `X == Y`

- 表示左邊的 `X` 若等於 `Y` 則為 `true` (真), 反之為 `false` (假)。

- `X != Y`

- 表示左邊的 `X` 若不等於 `Y` 則為 `true` (真), 反之為 `false` (假)。

選擇陳述式

- if - else
- switch - case - default

if - else

- `if` 陳述式會根據運算式的 `Boolean` 值識別要執行的陳述式。

```
static void Main(string[] args)
{
    int x = 1;
    if (x > 10)
    {
        Console.WriteLine("X 大於 10");
    }
    else
    {
        Console.WriteLine("X 小於或等於 10");
    }
}
```

如果要 else 個沒完呢？

```
static void Main(string[] args)
{
    int x = 1;
    if (x > 10)
    {
        Console.WriteLine("X 大於 10");
    }
    else
    {
        if (x < 10)
        {
            Console.WriteLine ("X 小於 10");
        }
        else
        {
            Console.WriteLine("X 等於 10");
        }
    }
}
```

也可以簡寫成這樣

```
static void Main(string[] args)
{
    int x = 1;
    if (x > 10)
    {
        Console.WriteLine("X 大於 10");
    }
    else if (x < 10)
    {
        Console.WriteLine("X 小於 10");
    }
    else
    {
        Console.WriteLine("X 等於 10");
    }
}
```

LAB

使用 if - else

在 MyFirstSolution 加入一個新專案

- 方案名稱：MyFirstSolution
- 專案名稱：ConditionDisplay
- 範本：Console Application
- csproj nullable disable

```
static void Main(string[] args)
{
    int condition = 10;
    int value;
    Console.WriteLine("請輸入一個數字 :");
    string input = Console.ReadLine();
    value = int.Parse(input);
    if (value > condition)
    {
        Console.WriteLine($"輸入的數字 {value} 大於 {condition}");
    }
    else if (value < condition)
    {
        Console.WriteLine($"輸入的數字 {value} 小於 {condition}");
    }
    else
    {
        Console.WriteLine($"輸入的數字 {value} 等於 {condition}");
    }
}
```

三種結合字串與變數的方式

- 字串插補 (String Interpolation) - C# 6.0+
- 複合格式化 (Composite Formatting)
- 字串串接 (String Concatenation)

```
Console.WriteLine($"輸入的數字 {value} 大於 {condition}");  
Console.WriteLine(string.Format("輸入的數字 {0} 大於 {1}", value, condition));  
Console.WriteLine("輸入的數字 " + value + " 大於 " + condition);
```

為什麼剛剛會需要轉換

- 重要的觀念：C# 是一個強型別的程式語言
- 強型別語言中的操作都需要符合其型別要求
- 型別要求可能是相容的，而非相同的。
- 例外
 - `dynamic`

執行

switch – case – default

- **switch** 陳述式會從候選項清單中選取要執行的「參數區段」
- **break** 陳述式會終止它所在之最靠近的封閉式迴圈或 **switch** 陳述式。 程式控制權轉移到終止陳述式之後的陳述式（如果有的話）。
- **C# 7** 以後的 **switch case** 有重大改變

```
class Program
{
    static void Main(string[] args)
    {
        int x = 1;
        switch (x)
        {
            case 1:
                Console.WriteLine("X 是 1");
                break;
            case 2:
                Console.WriteLine("X 是 2");
                break;
            default:
                Console.WriteLine("X 在條件之外");
                break;
        }

        Console.ReadLine();
    }
}
```

← 跳離區段

LAB

使用 switch - case

在 MyFirstSolution 加入一個新專案

- 方案名稱：MyFirstSolution
- 專案名稱：SwitchDisplay
- 範本：Console Application
- csproj nullable disable

```
static void Main(string[] args)
{
    int value;
    Console.WriteLine("請輸入一個數字 :");
    string input = Console.ReadLine();
    value = int.Parse(input);
    switch (value)
    {
        case 1:
            Console.WriteLine("數字是 1");
            break;
        case 2:
            Console.WriteLine("數字是 2");
            break;
        default:
            Console.WriteLine("數字不在條件內");
            break;
    }
    Console.ReadLine();
}
```

執行

若是 1, 2 要執行同樣的行為 (SwitchDisplay2)

```
static void Main(string[] args)
{
    int value;
    Console.WriteLine("請輸入一個數字 :");
    string input = Console.ReadLine();
    value = int.Parse(input);
    switch (value)
    {
        case 1:

        case 2:
            Console.WriteLine("數字是 1 或 2");
            break;
        default:
            Console.WriteLine("數字不在條件內");
            break;
    }
    Console.ReadLine();
}
```



執行

陣列

概觀

- 在陣列資料結構中儲存多個相同類型的變數
- 陣列可以是一維、多維或不規則。
- 維度的數目和每個維度的長度會在建立陣列執行個體時確立。 在執行個體的存留期中，這些值無法變更。
- 實值型別陣列元素的預設值會設定為零，而參考型別元素則設定為 `null`。
- 陣列本身是參考型別。

- 不規則陣列是指包含陣列的陣列，因此其元素為參考類型，而且會初始化為 `null`。
- C#中的陣列是以零為起始索引，即包含 `n` 個元素的陣列建立索引時，會從 `0` 開始，一直到 `n-1` 為止。
- 陣列元素可以是任何類型，包括陣列類型。
- 在我們的經驗裡，很少使用多維或不規則陣列。
- [參考 Microsoft learn - .NET C# 陣列](#)

陣列的宣告

```
// 定義一個尚未初始化的一維陣列 (這會一直出現警告，直到你初始化陣列為止)
int[] array0;

//定義一個有五個元素一維陣列
int[] array1 = new int[5];

// 定義一個一維陣列，同時給予陣列的內容
int[] array2 = new int[] { 1, 3, 5, 7, 9 };
/* 後面的 { 1, 3, 5, 7, 9 } 被稱為陣列初始化設定式 (array initializer) */

// 同上，語法糖的寫法
int[] array3 = { 1, 2, 3, 4, 5, 6 };

// 定義一個 2X3 的二維陣列
int[,] multiDimensionalArray1 = new int[2, 3];

// 定義一個二維陣列，同時給予陣列的內容
int[,] multiDimensionalArray2 = { { 1, 2, 3 }, { 4, 5, 6 } };

// 定義一個不規則陣列
int[][] jaggedArray = new int[6][];

// 設定不規則陣列的內容
jaggedArray[0] = new int[4] { 1, 2, 3, 4 };

// C# 12 開始支援集合運算式
int[] array4 = [1, 2, 3, 4, 5];
```

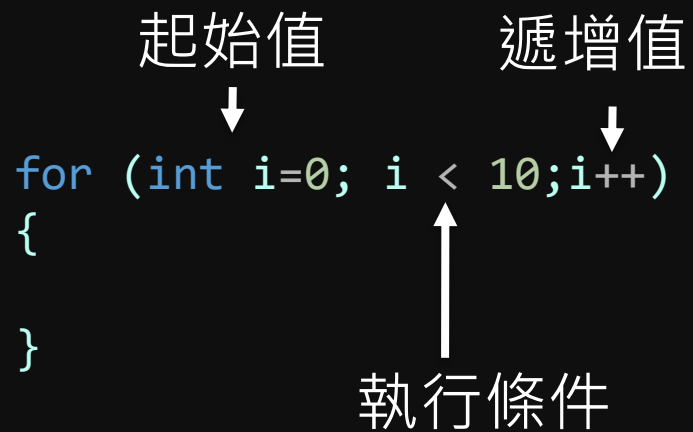
反覆運算陳述式 (迴圈)

反覆運算陳述式

- for
- while
- do while
- foreach in

for

- 基本語法



The diagram illustrates the syntax of a C-style for loop. It shows the code `for (int i=0; i < 10; i++)` with three annotations: '起始值' (Initial Value) with a downward arrow pointing to `i=0`, '遞增值' (Increment Value) with a downward arrow pointing to `i++`, and '執行條件' (Execution Condition) with an upward arrow pointing to `i < 10`. The code is followed by an opening curly brace `{` and a closing curly brace `}`.

```
          起始值          遞增值  
          ↓              ↓  
for (int i=0; i < 10; i++)  
{  
    
}
```

執行條件

LAB

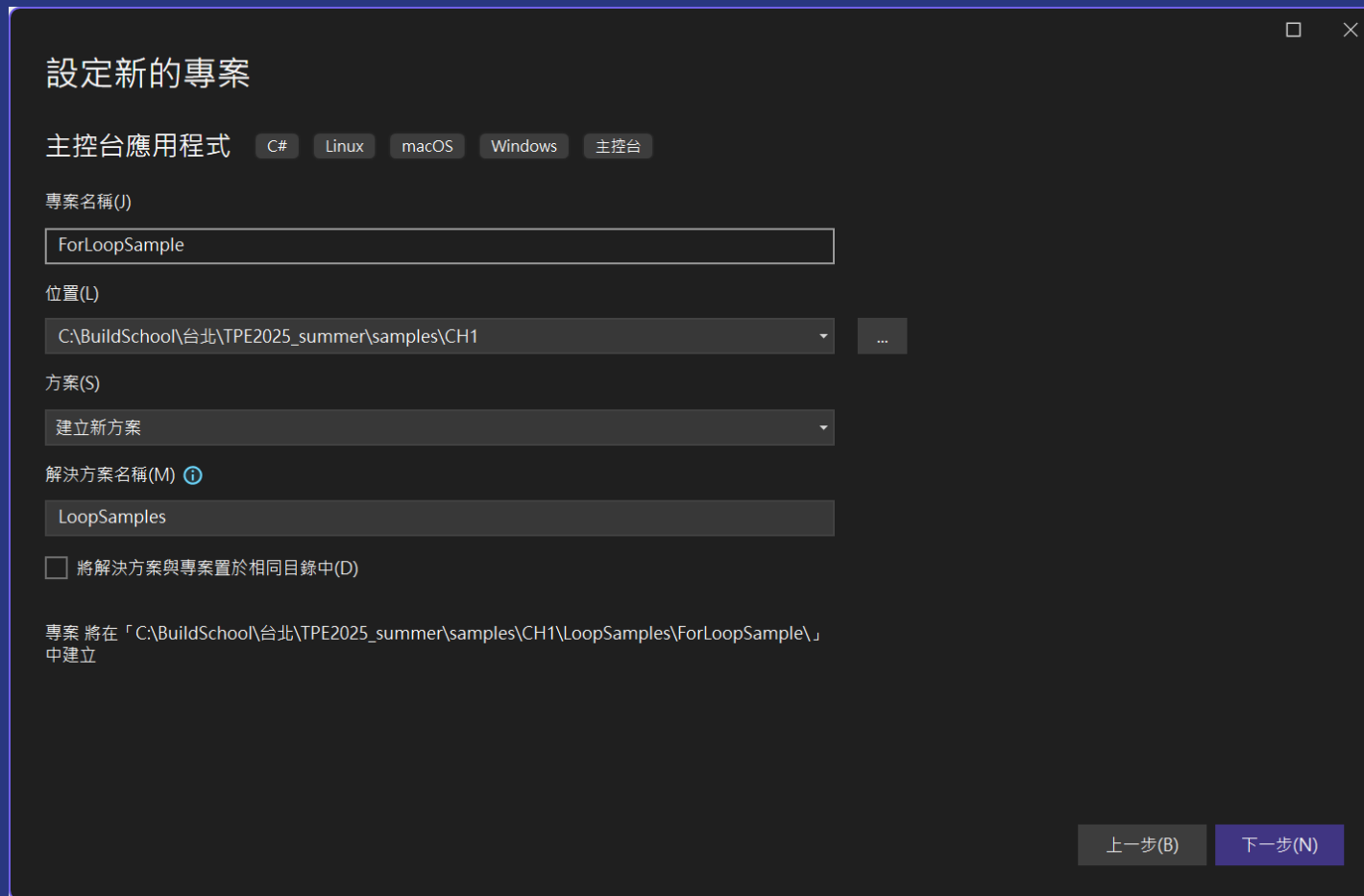
使用 for 迴圈做 1~10 加總

新增一個方案及專案

方案名稱：LoopSamples

專案名稱：ForLoopSample

csproj nullable disable



設定新的專案

主控台應用程式 C# Linux macOS Windows 主控台

專案名稱(I)

ForLoopSample

位置(L)

C:\BuildSchool\台北\TPE2025_summer\samples\CH1

方案(S)

建立新方案

解決方案名稱(M) ⓘ

LoopSamples

☐ 將解決方案與專案置於相同目錄中(D)

專案 將在「C:\BuildSchool\台北\TPE2025_summer\samples\CH1\LoopSamples\ForLoopSample\」中建立

上一步(B) 下一步(N)

```
static void Main(string[] args)
{
    int result = 0;
    for (int i = 1; i < 11; i++)
    {
        result = result + i;
        // 也可以寫成 result +=i;
    }
    Console.WriteLine($"加總結果 : {result}");
    Console.ReadLine();
}
```

執行

人腦編譯器時間



- 用紙和筆紀錄剛剛程式碼中執行過程的變數 `result` 和 `i` 的變化

while

- 基本語法

執行條件



```
while (i < 10)
{
}
```

LAB

使用 while 迴圈做 1~10 加總

在 LoopSamples 加入一個新專案

- 方案名稱： LoopSamples
- 專案名稱： WhileLoopSample
- 範本：Console Application
- csproj nullable disable

```
static void Main(string[] args)
{
    int result = 0;
    int i = 1;
    while (i < 11)
    {
        result = result + i;
        i = i + 1;
        /* 也可以寫成
           i++ 或 ++i 或 i+=1
        */
    }
    Console.WriteLine($"加總結果 : {result}");
    Console.ReadLine();
}
```



執行

人腦編譯器時間



- 用紙和筆紀錄剛剛程式碼中執行過程的變數 `result` 和 `i` 的變化

do while

- 基本語法

```
do  
{  
  
}  
while (i < 10);
```



執行條件

LAB

使用 do while 迴圈做 1~10 加總

在 LoopSamples 加入一個新專案

- 方案名稱： LoopSamples
- 專案名稱： DoWhileLoopSample
- 範本： Console Application
- csproj nullable disable

```
static void Main(string[] args)
{
    int result = 0;
    int i = 1;
    do
    {
        result = result + i;
        i = i + 1;
    }
    while (i < 11);
    Console.WriteLine($"加總結果 : {result}");
    Console.ReadLine();
}
```



執行

人腦編譯器時間



- 比較 while 和 do while
- 剛剛的兩個範例
 - WhileLoopSample
 - DoWhileLoopSample
- 如果把 i 的初始值改成 12 會是甚麼結果？

foreach

- **foreach** 陳述式是用來逐一查看集合，以取得所需的資訊，但是不能用來加入或移除來源集合的項目，以避免無法預期的副作用。
- 嚴格來說他是一種迭代模式。
- 基本語法

```
int[] array = new int[] { 0, 1, 2, 3, 4, 5 };  
foreach(int i in array)  
{  
  
}
```

LAB

使用 foreach 逐一顯示集合內資料

在 LoopSamples 加入一個新專案

- 方案名稱： LoopSamples
- 專案名稱： ForeachLoopSample
- 範本： Console Application
- csproj nullable disable

```
static void Main(string[] args)
{
    int[] array = new int[] { 0, 1, 2, 3, 4, 5 };
    foreach (int i in array)
    {
        Console.WriteLine(i);
    }
    Console.ReadLine();
}
```

執行

跳躍陳述式

跳躍陳述式

- 所謂跳躍，就是移轉程式的控制權（執行路徑）
- `break`：還算常用
- `continue`：不太常用
- `goto`：這是個廢物，不用理他
- `return`：你會常常用

break

- `break` 陳述式會終止它所在之最靠近的封閉式迴圈或 `switch` 陳述式。 程式控制權轉移到終止陳述式之後的陳述式（如果有的話）。

LAB

在迴圈內使用 break

新增一個方案及專案

- 方案名稱：JumpSamples
- 專案名稱：BreakSample
- 範本：Console Application
- csproj nullable disable

```
static void Main(string[] args)
{
    int condition = 15;
    for (int i = 0; i < 3; i++)
    {
        Console.WriteLine($"i = {i}");
        for (int j = 11; j < 20; j++)
        {
            if (j == condition)
            {
                break;
            }
            Console.WriteLine($"j = {j}");
        }
    }
    Console.ReadLine();
}
```

執行

continue

- `continue` 陳述式將控制封入 `while`、`do - while`、為或它所在的 `foreach` 陳述式的下一個反覆項目。
- 意思就是說，當條件符合的時候，`continue` 會直接進到下一輪迴圈，而忽略後面的程式碼。

LAB

在迴圈內使用 `continue`

在 JumpSamples 中加入新專案

- 方案名稱：JumpSamples
- 專案名稱：ContinueSample
- 範本：Console Application
- csproj nullable disable

```
static void Main(string[] args)
{
    int condition = 15;
    for (int i = 0; i < 3; i++)
    {
        Console.WriteLine($"i = {i}");
        for (int j = 11; j < 20; j++)
        {
            if (j == condition)
            {
                continue;
            }
            Console.WriteLine($"j = {j}");
        }
    }
    Console.ReadLine();
}
```



執行

人腦編譯器時間



- 不用電腦，比較出 break 和 continue 的差異

return

- **return** 陳述式終止其所在處之方法的執行，並且轉移程式控制權至呼叫的方法。它也可以傳回選擇性值。如果方法屬於 **void** 型別，那麼可以省略 **return** 陳述式。
- 如果 **return** 陳述式位於 **try** 區塊內，則會先執行 **finally** 區塊（如果有的話），控制權才會回到呼叫的方法。

LAB

使用 return

在 JumpSamples 中加入新專案

- 方案名稱：JumpSamples
- 專案名稱： ReturnSample001
- 範本：Console Application
- csproj nullable disable

先加入兩個分開的方法(函式)

```
static int ReturnInt()
{
    Console.WriteLine("進入 ReturnInt");

    // 回傳值要與宣告的型別相同
    return 10;
    Console.WriteLine("這行不會被執行到");
}

static void ReturnVoid()
{
    Console.WriteLine("進入 ReturnVoid");
    // 沒有回傳值時，return 若在整個方法的最後一行可以省略
    return;
    Console.WriteLine("這行不會被執行到");
}
```

在 Main 方法呼叫 ReturnVoid 和 ReturnInt

```
static void Main(string[] args)
{
    ReturnVoid();
    int i = ReturnInt();
    Console.ReadLine();
}
```

條件運算子

條件邏輯運算子

- `x && y`
 - And
 - 當 `x` 與 `y` 皆為 `true`，回傳結果才為 `true`。
 - 當 `x` 為 `false`，直接回傳 `false`，不會再判斷 `y`。
- `x || y`
 - Or
 - 當 `x` 或 `y` 其中一個為 `true`，回傳結果即為 `true`。
 - 當 `x` 為 `true`，直接回傳 `true`，不會再判斷 `y`。
- `!x`（其實不算條件運算子，但一起講比較方便）
 - Not
 - `x` 為 `true` 則結果為 `false`，`x` 為 `false` 則結果為 `true`。

LAB

使用條件邏輯運算子

新增一個方案及專案

- 方案名稱：ConditionalOperatorSamples
- 專案名稱：And_Or_OperatorSample
- 範本：Console Application
- csproj nullable disable

```
static void Main(string[] args)
{
    int x1 = 10;
    string y1 = "abc";

    int x2 = 10;
    string y2 = "abc";

    if (x1 == x2 && y1 == y2)
    {
        Console.WriteLine("都正確");
    }
    else
    {
        Console.WriteLine("至少有一個不正確");
    }

    if (x1 == x2 || y1 == y2)
    {
        Console.WriteLine("至少有一個正確");
    }
    else
    {
        Console.WriteLine("兩個都不正確");
    }
}
```



執行

討論

- 隨意改變前面程式碼的 $x2$ 與 $y2$ 變數的值，並觀察結果將會有甚麼樣的變化。



寫個方法 (Method)

初步了解 Method

- 方法是包含一系列陳述式的程式碼區塊。 程式會呼叫方法並指定任何所需的方法引數來執行陳述式。
- 其他的稱呼
 - 函式
 - 副程式
- 從現在開始我們主要會以『方法』或『Method』來稱呼他
- 方法是型別的成員，會存在於類別、結構或介面中

為什麼要寫成特定的 `method`

- 某些一連串的流程可能會被重複使用，為了避免程式碼重複撰寫，會將這些流程封裝在一個方法內，這樣就可以重複使用同樣的流程而不需撰寫重複的程式碼。
- 使用方法封裝流程的另一個好處是可以讓每個方法只執行特定的任務，能有效縮短單一方法的程式碼長度，對於簡化程式碼維護有幫助。

方法宣告三要素

- 回傳值
- 方法名稱
- 參數列表

```
static int CalculateSumOfPositives(int[] numbers)
```

計算陣列內正整數總和（無獨立方法）

```
static void Main(string[] args)
{
    int[] numbers1 = { 1, -2, 3, -4, 5, 6 };
    int[] numbers2 = { 10, 20, -30, 40, -50 };

    int sumOfPositives1 = 0;
    foreach (int num in numbers1)
    {
        if (num > 0)
        {
            sumOfPositives1 += num;
        }
    }
    Console.WriteLine($"numbers1 中的正數總和: {sumOfPositives1}");

    int sumOfPositives2 = 0;
    foreach (int num in numbers2)
    {
        if (num > 0)
        {
            sumOfPositives2 += num;
        }
    }
    Console.WriteLine($"numbers2 中的正數總和: {sumOfPositives2}");

    Console.ReadLine();
}
```

LAB

讓咱們來寫個 Method 吧

新增一個方案及專案

- 方案名稱：MethodSamples
- 專案名稱：MethodSample001_R1
- 範本：Console Application
- csproj nullable disable

先宣告一個方法 CalculateSumOfPositives

```
internal class Program
{
    static void Main(string[] args)
    {

    }

    static int CalculateSumOfPositives(int[] numbers)
    {

    }

}
```

完成方法内容

```
static int CalculateSumOfPositives(int[] numbers)
{
    int sum = 0;
    foreach (int num in numbers)
    {
        if (num > 0)
        {
            sum += num;
        }
    }
    return sum;
}
```

在 Main 中呼叫 CalculateSumOfPositives

```
static void Main(string[] args)
{
    int[] numbers1 = { 1, -2, 3, -4, 5, 6 };
    int[] numbers2 = { 10, 20, -30, 40, -50 };

    // 使用提取出的方法處理 numbers1
    int sumOfPositives1 = CalculateSumOfPositives(numbers1);
    Console.WriteLine($"numbers1 中的正數總和: {sumOfPositives1}");

    // 使用提取出的方法處理 numbers2
    int sumOfPositives2 = CalculateSumOfPositives(numbers2);
    Console.WriteLine($"numbers2 中的正數總和: {sumOfPositives2}");

    Console.ReadLine();
}
```

執行

人腦編譯器時間



- 畫出程式呼叫的路徑
- 標示傳入值與回傳值

LAB

練習 Visual Studio 的重構功能

新增一個方案及專案

- 方案名稱：MethodSamples
- 專案名稱：MethodSample001
- 範本：Console Application
- csproj nullable disable

先完成 Main 方法，接著跟我操作

```
static void Main(string[] args)
{
    int[] numbers1 = { 1, -2, 3, -4, 5, 6 };
    int[] numbers2 = { 10, 20, -30, 40, -50 };

    int sumOfPositives1 = 0;
    foreach (int num in numbers1)
    {
        if (num > 0)
        {
            sumOfPositives1 += num;
        }
    }
    Console.WriteLine($"numbers1 中的正數總和: {sumOfPositives1}");

    int sumOfPositives2 = 0;
    foreach (int num in numbers2)
    {
        if (num > 0)
        {
            sumOfPositives2 += num;
        }
    }
    Console.WriteLine($"numbers2 中的正數總和: {sumOfPositives2}");

    Console.ReadLine();
}
```

隨堂練習

最大值判斷

- 說明

- 使用者輸入三個整數，判斷並輸出其中最大值

- 流程

- 輸入三個整數（使用 , 分隔）(例如 11,22,99)
- 呼叫 String.Split 將輸入的字串分割為 string[]
 - `string[] content = Console.ReadLine().Split(',');`
- 把陣列的每一個元素 parse 為 int
- 使用 if 判斷哪一個最大
- 輸出「最大值為 : X」

Windows Forms Application

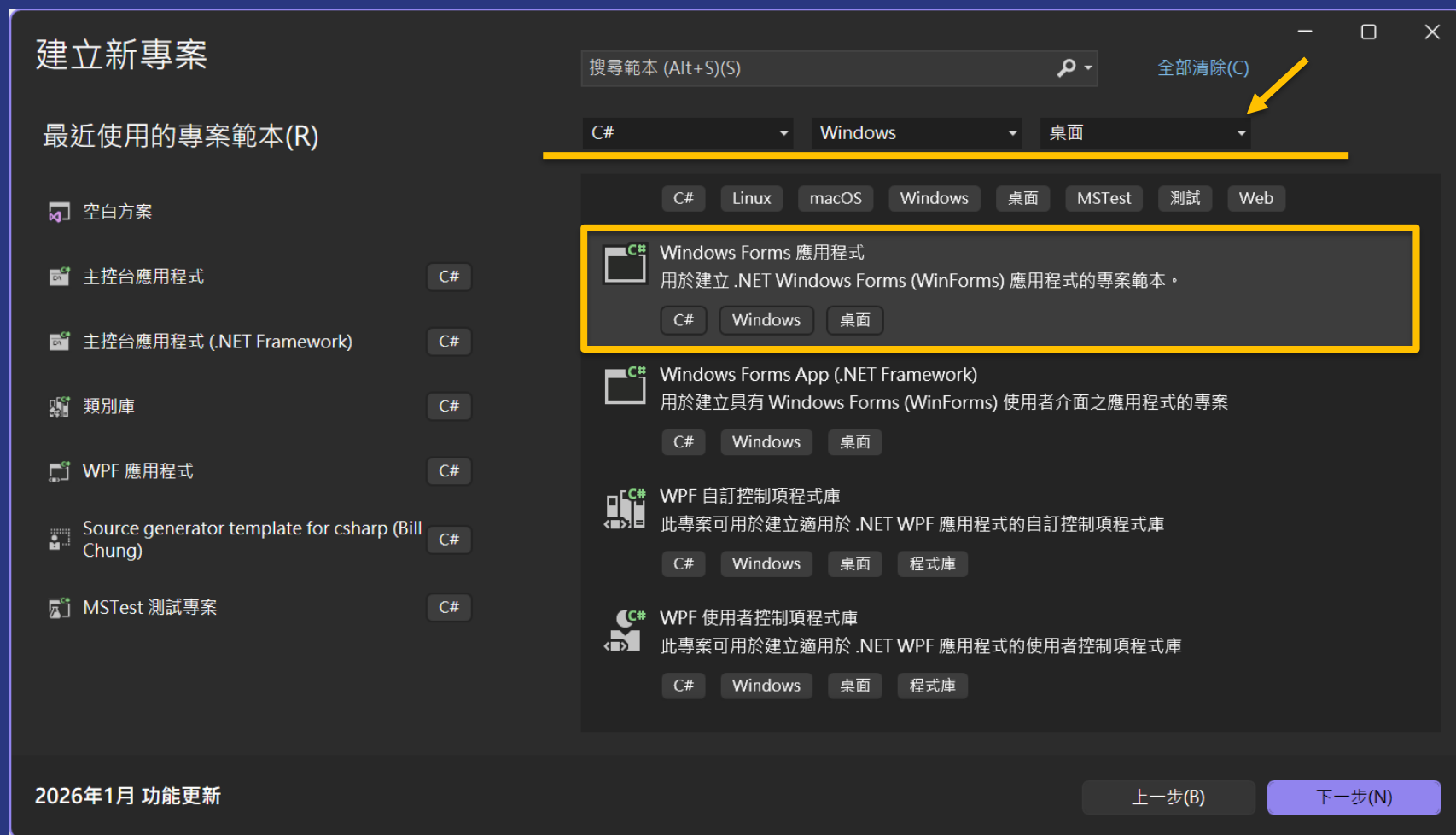
Windows Forms Application

- 用來建立桌面應用程式。
- 有許多既有個控制項可以使用。

LAB

建立 Windows Forms Application

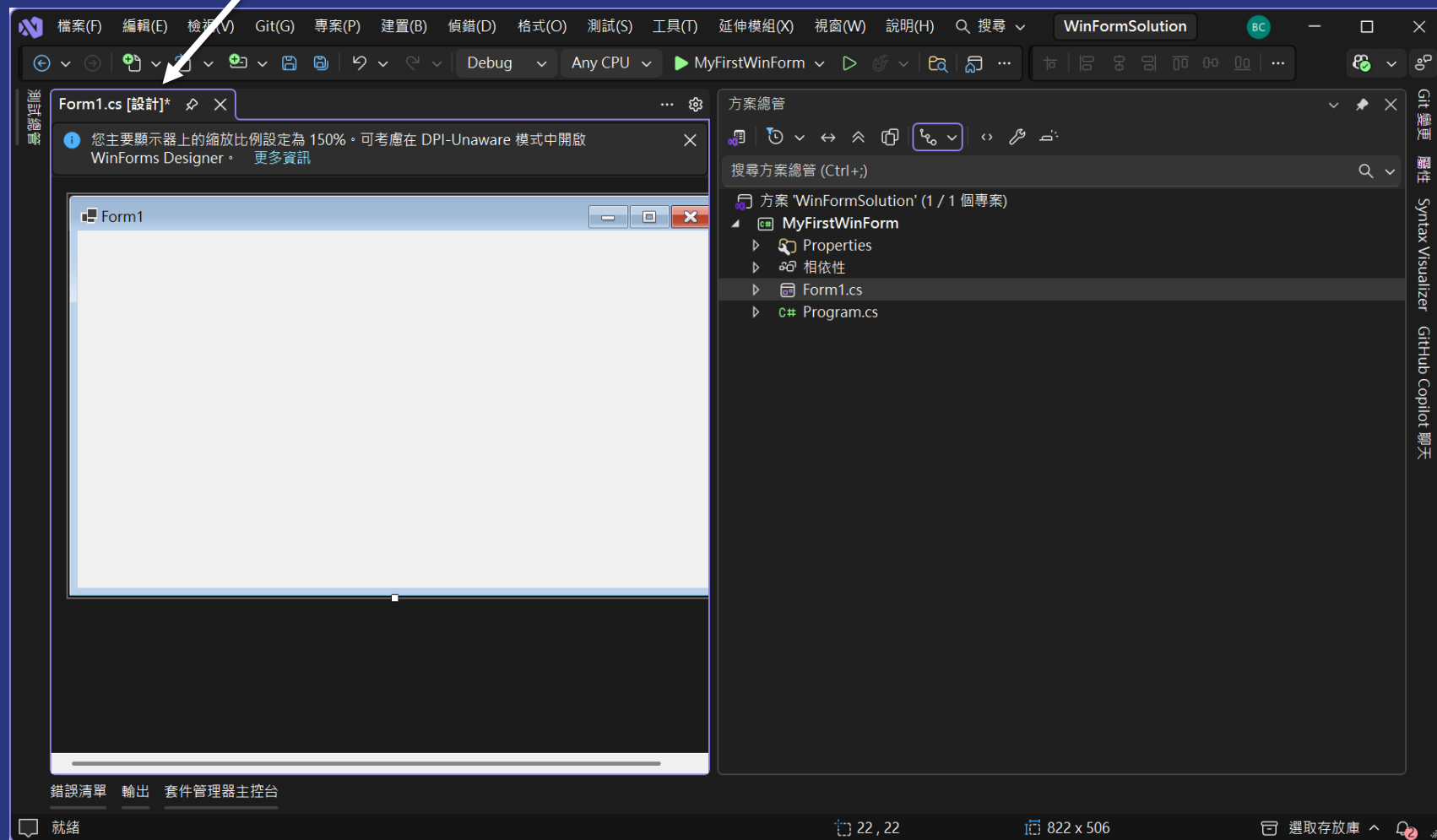
專案範本選【Windows Forms App】



新增一個方案及專案

- 方案名稱：WinFormSolution
- 專案名稱：MyFirstWinForm
- 範本：Windows Forms Application
- csproj nullable disable

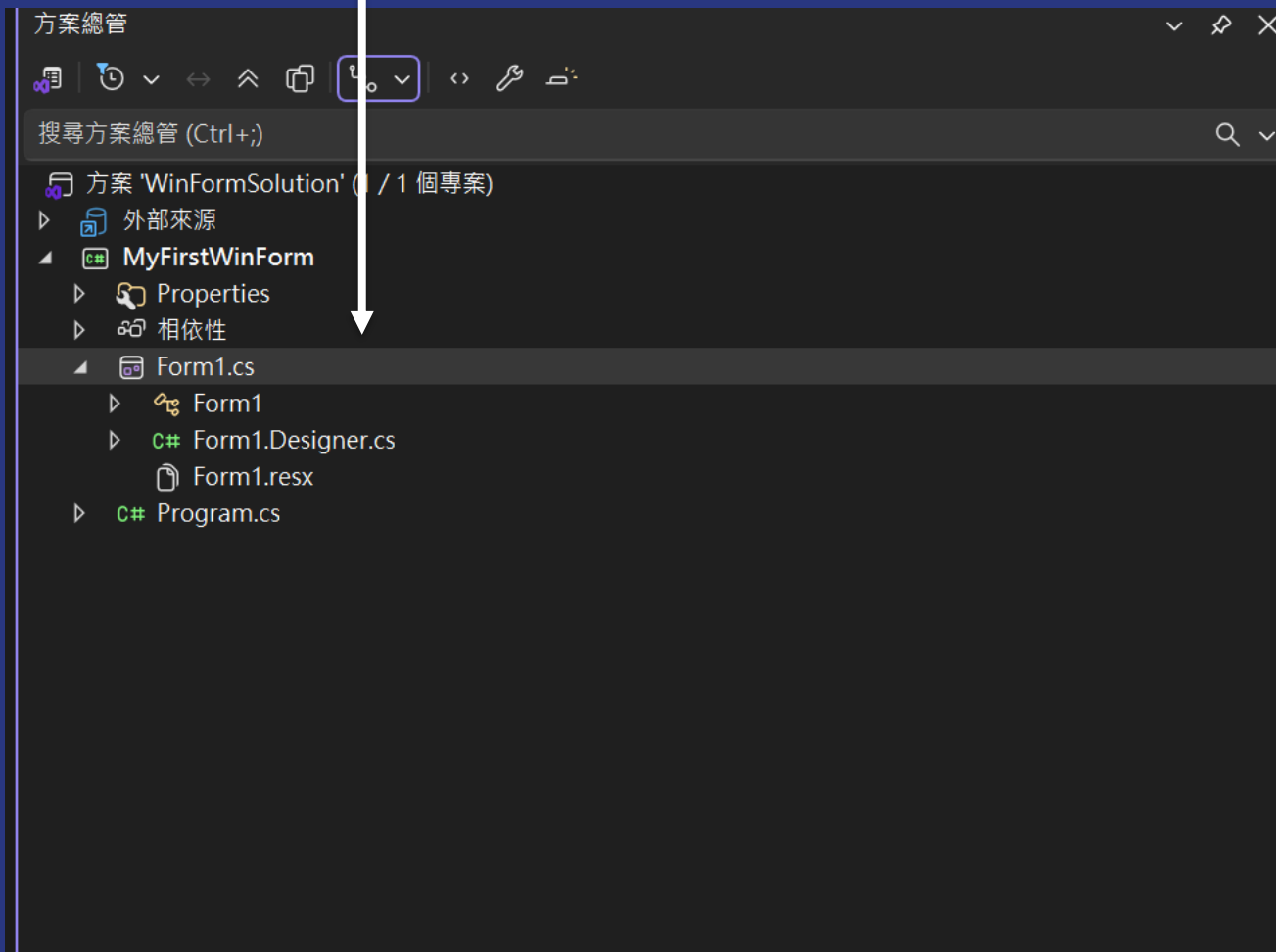
Windows Form 的設計畫面 (頁籤上有「設計」二字)



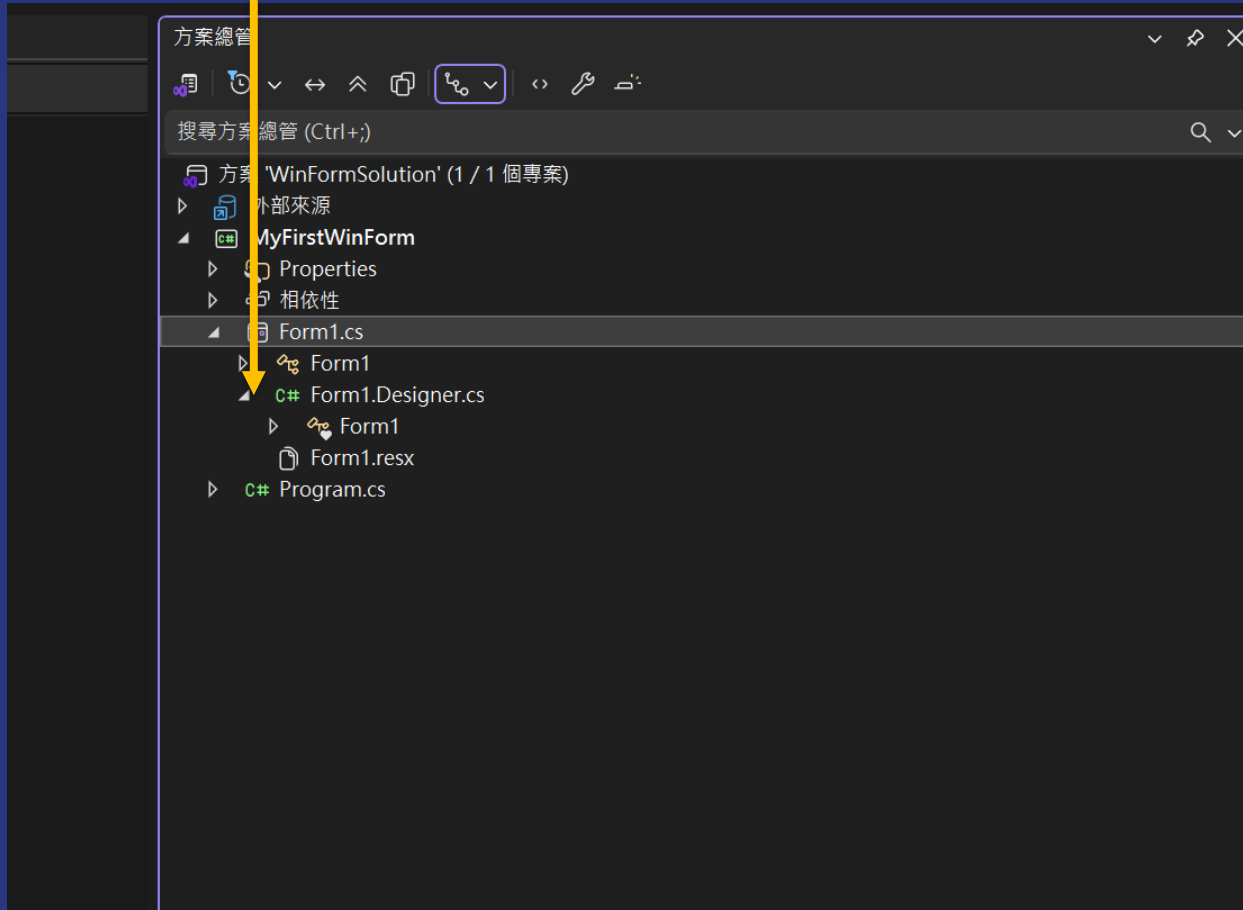
WYSIWYG -- What You See Is What You Get

- 在編輯或設計介面時所看到的畫面，就是使用者實際執行時會看到的結果。讓開發者或設計者能夠「**所見即所得**」，不需要預先想像程式執行後的樣子。
- 在 **Visual Studio** 中建立 **Windows Forms** 專案
 - 設計階段 (**Design View**)
 - 可以直接拖曳按鈕、文字框、標籤等元件到表單上。
 - 調整位置、大小、顏色、字型等屬性。
 - 所有變更都立即反映在設計介面上。
 - 執行階段 (**Run-Time View**)
 - 當你按下「開始執行」時，程式外觀與你在設計介面中看到的幾乎相同。
 - 使用者看到的 **UI** 就是你設計時所見的結果。

如果你把設計畫面關掉了，
在Form1.cs上雙擊滑鼠左鍵



展開 Form1.Designer.cs



Form 的檔案組成

- **Form1.cs** — 表單的主程式邏輯
 - 用途：這是你撰寫程式邏輯的地方。
 - 開發者互動最多的檔案：你會在這裡加入控制流程、資料處理、與其他元件互動的程式碼。
- **Form1.designer.cs** — UI 元件的初始化程式碼
 - 用途：由 Visual Studio 自動產生（當你在設計畫面拖拉會改變這個檔案內容），用來定義和初始化 UI 元件。
 - 不建議手動編輯：因為這部分通常由設計工具維護，手動修改可能導致設計器錯誤（只要熟悉還是可以的）。
- **Form1.resx**— 表單的資源檔案
 - 用途：儲存與表單相關的非程式資源。

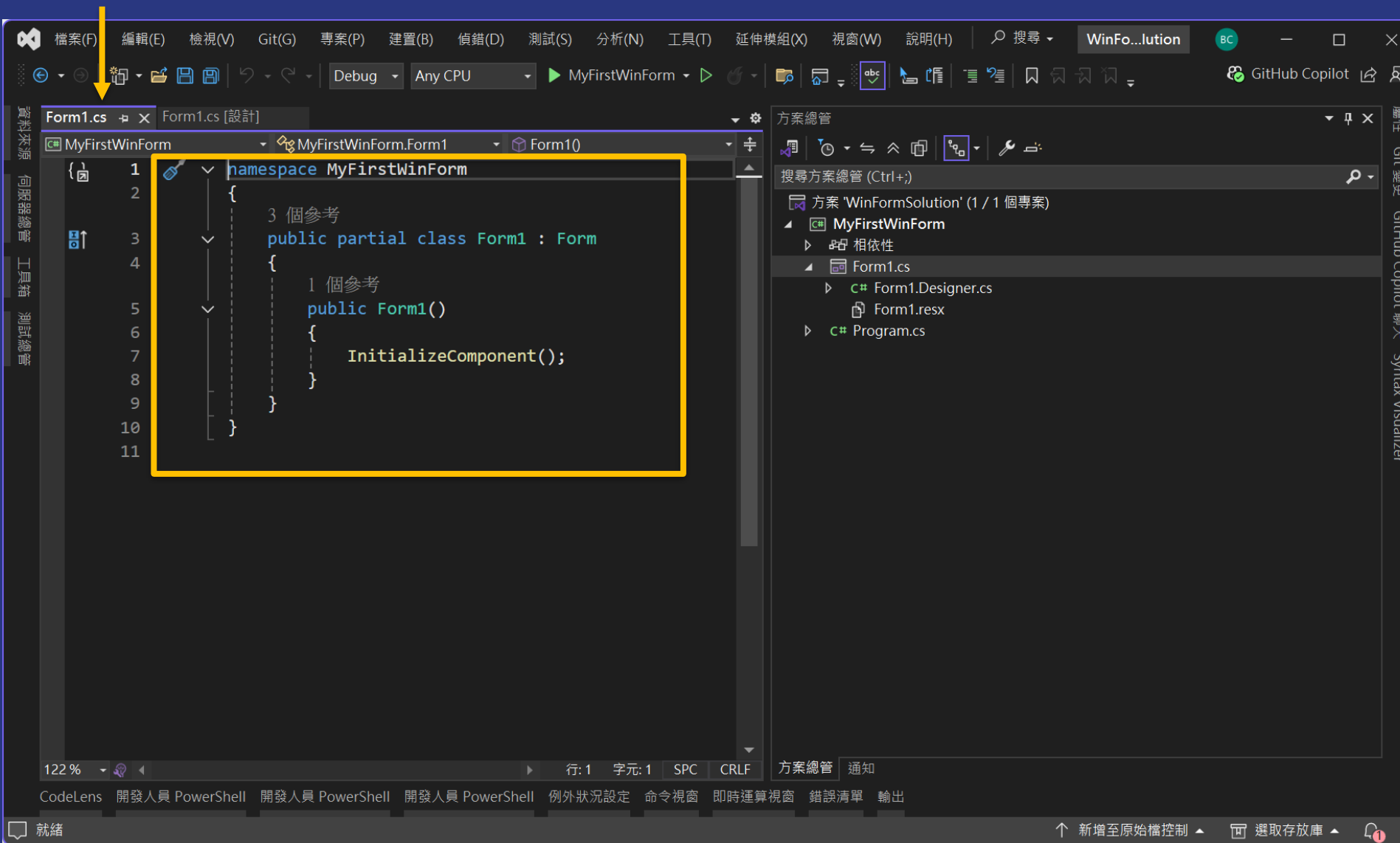
使用檔案總管觀察

名稱	修改日期	類型	大小
bin	2025/8/18 下午 04:20	檔案資料夾	
obj	2025/8/18 下午 04:20	檔案資料夾	
Form1.cs	2025/8/18 下午 04:20	C# Source File	1 KB
Form1.Designer.cs	2025/8/18 下午 04:20	C# Source File	2 KB
Form1.resx	2025/8/18 下午 04:20	VisualStudio.resx.c8...	6 KB
MyFirstWinForm.csproj	2025/8/18 下午 04:20	C# Project File	1 KB
MyFirstWinForm.csproj.user	2025/8/18 下午 04:20	Per-User Project Op...	1 KB
Program.cs	2025/8/18 下午 04:20	C# Source File	1 KB

將滑鼠移到設計畫面，按右鍵，
選擇【檢視程式碼】



這就是讓你寫程式的地方



- (1) class 宣告代表這是一個類別
- (2) Form1 : Form 代表這個類別的名稱是 Form1 ，他的父類別是 Form (也就是繼承自 Form Class)
- (3) partial 代表這是一個部分類別

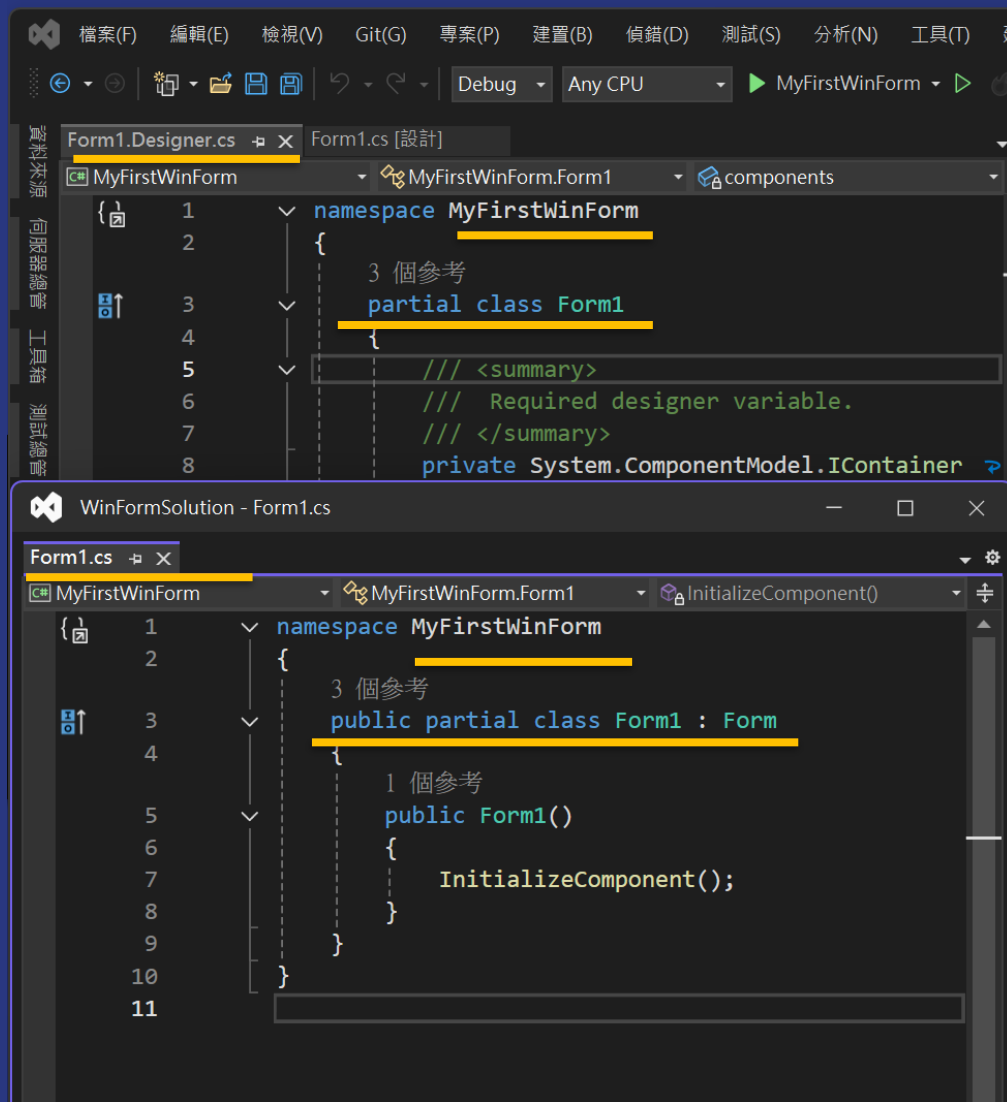
```
public partial class Form1 : Form
```

右側畫面可以看出兩個檔案的類別名稱都一樣，並且都帶有 `partial`

代表這兩份程式碼是屬於 `MyFirstWinForm.Form1` 的一部分

編譯時會將兩者合併成單一個 `MyFirstWinForm.Form1` class

參考 [部分類別和成員](#)



各位會看到一個和類別同名的方法
這是一個特殊的方法
被稱為建構式

```
public Form1()  
{  
    InitializeComponent();  
}
```

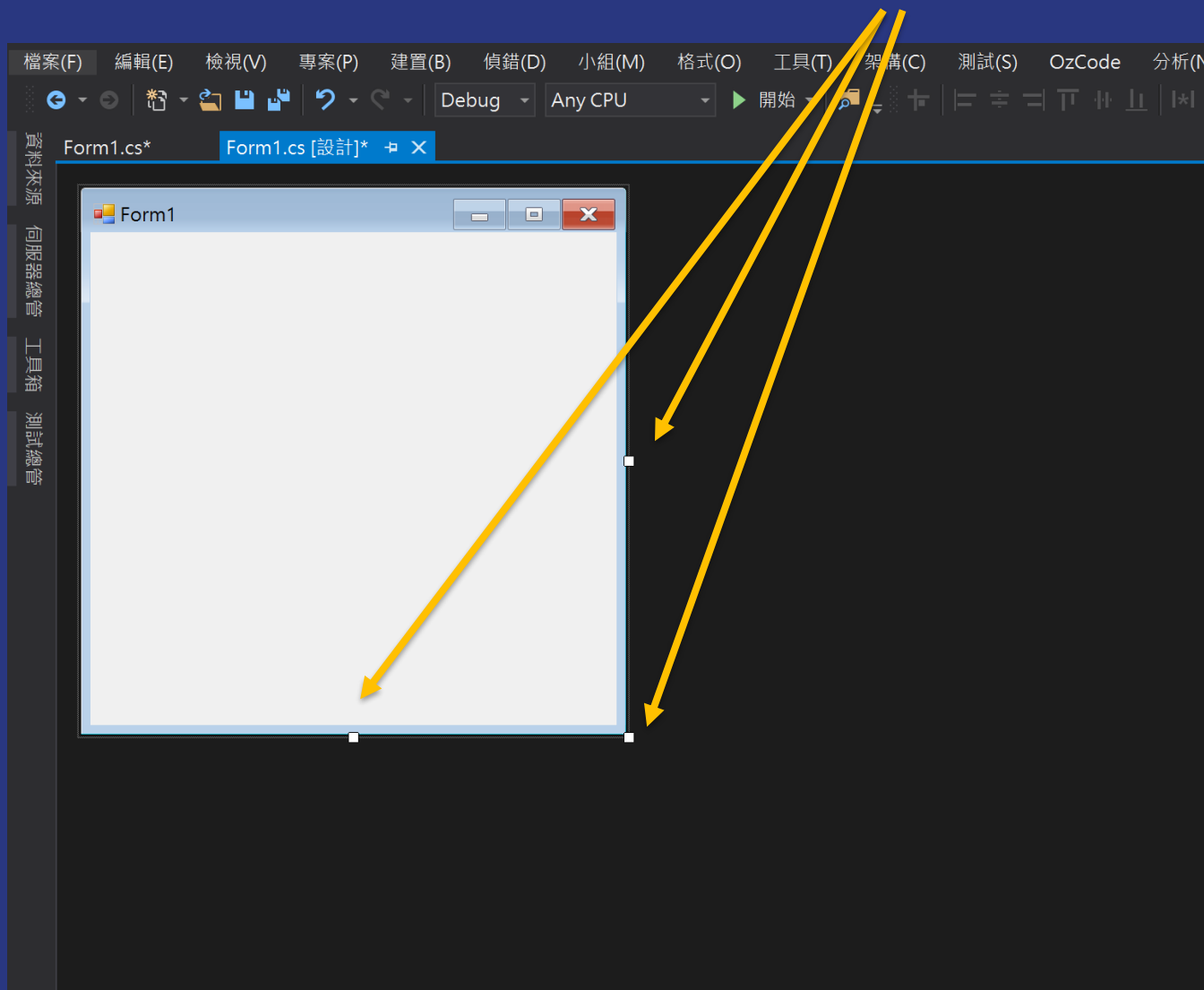
註：建構式是用來初始化類別欄位成員

討論

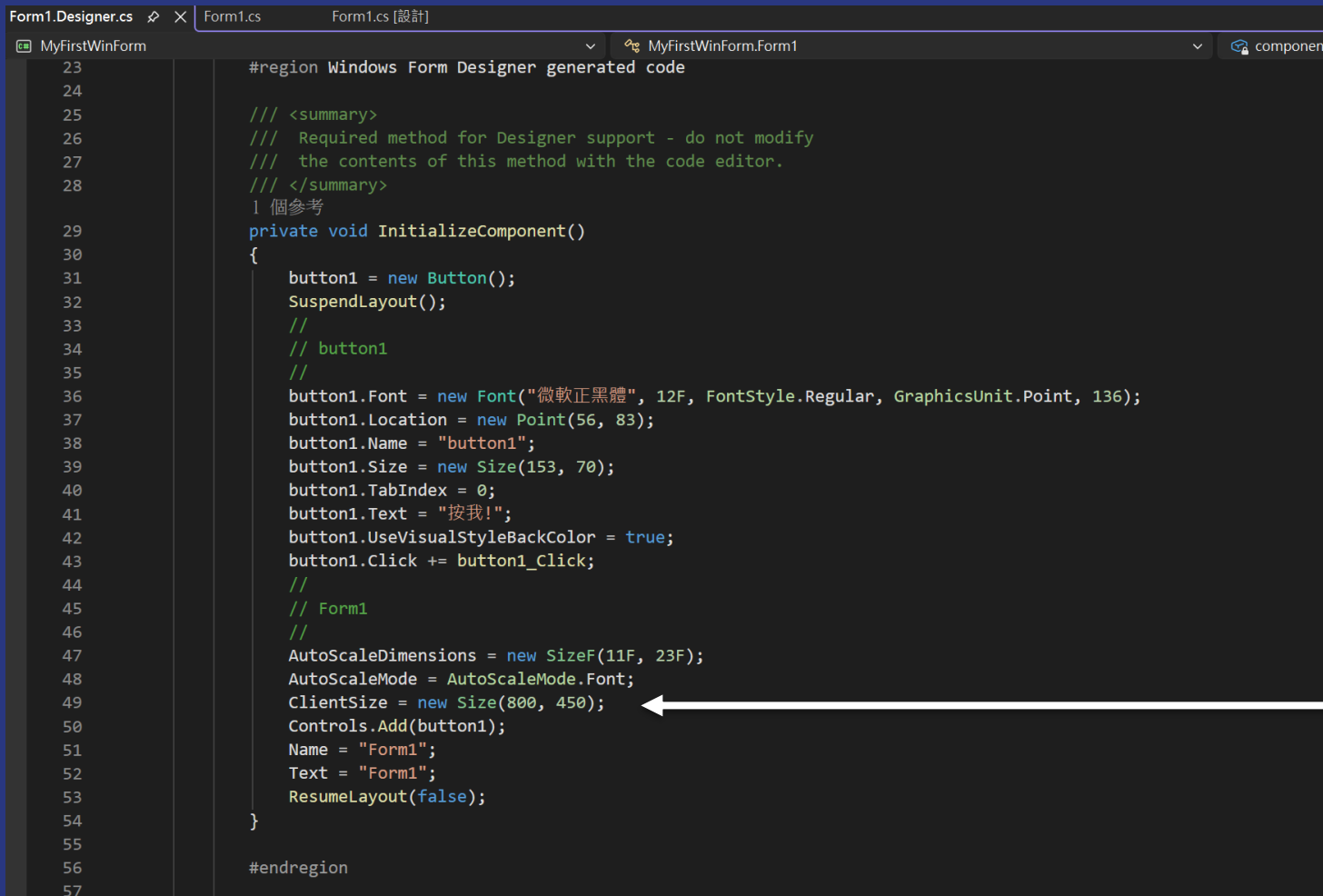
- 進行到這邊，了解建立 Windows Forms Application 的步驟了嗎？



回到設計畫面，拖動視窗的大小



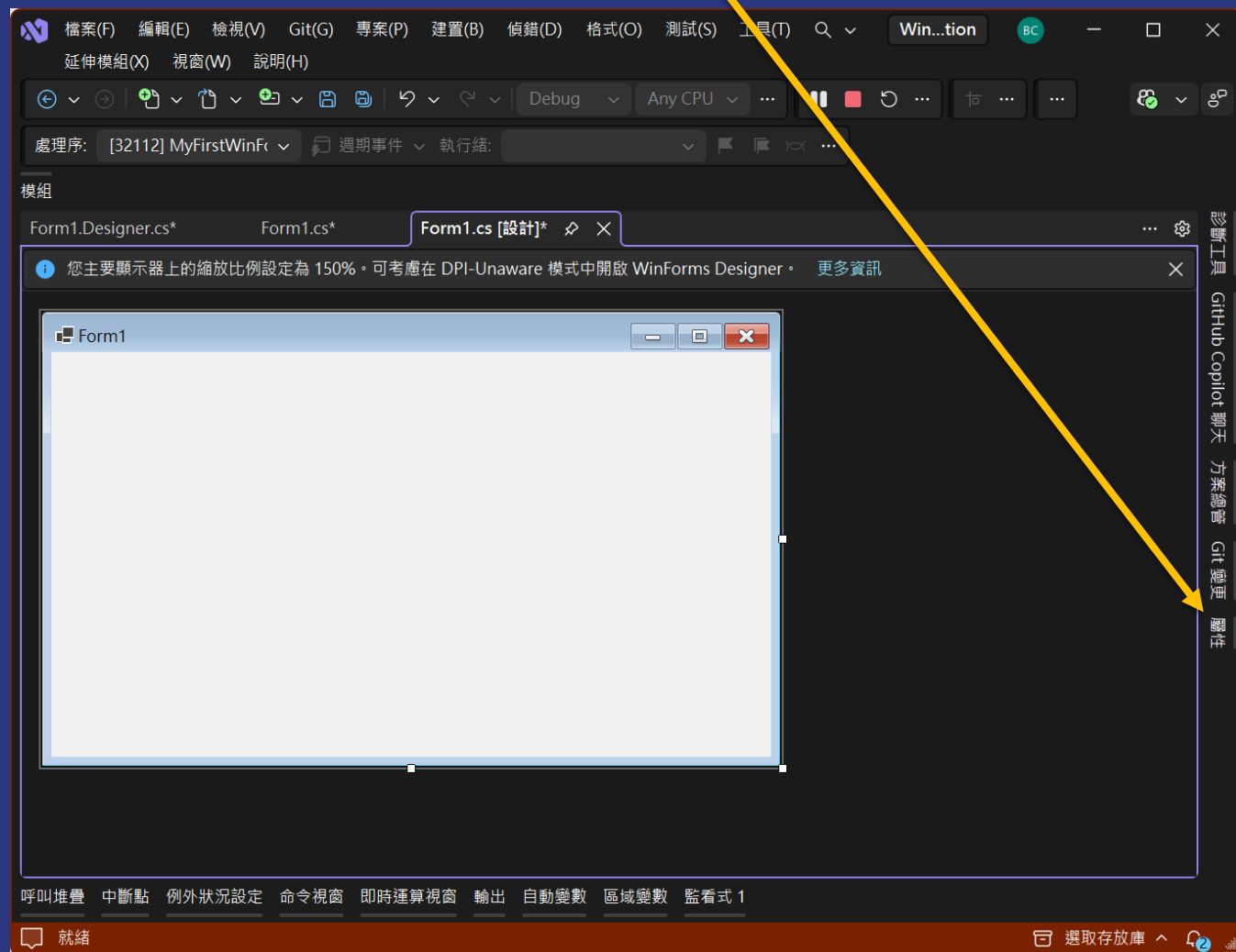
開啟 Form1.Designer.cs，找到這一行
記下他的數字，回去設計畫面再改變一次
大小，回來看數字的變化

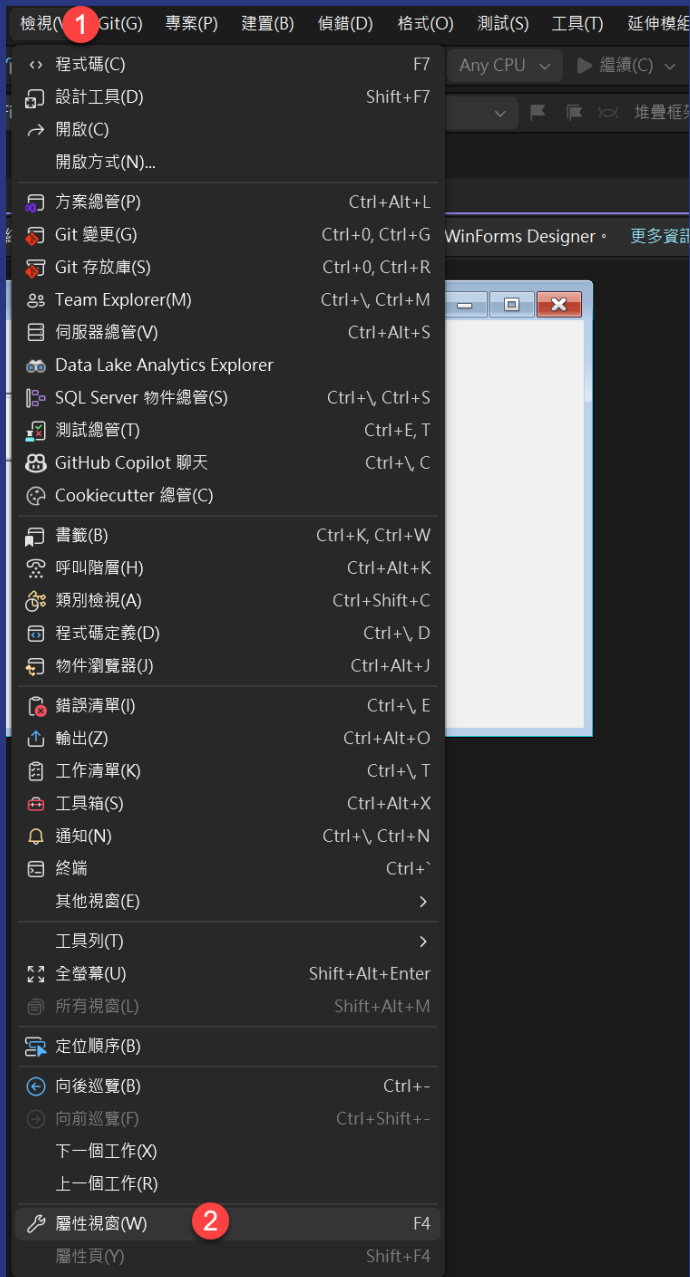


```
Form1.Designer.cs  ✕  Form1.cs  Form1.cs [設計]
MyFirstWinForm  MyFirstWinForm.Form1  componen

23  #region Windows Form Designer generated code
24
25  /// <summary>
26  /// Required method for Designer support - do not modify
27  /// the contents of this method with the code editor.
28  /// </summary>
29  1 個參考
30  private void InitializeComponent()
31  {
32      button1 = new Button();
33      SuspendLayout();
34      //
35      // button1
36      //
37      button1.Font = new Font("微軟正黑體", 12F, FontStyle.Regular, GraphicsUnit.Point, 136);
38      button1.Location = new Point(56, 83);
39      button1.Name = "button1";
40      button1.Size = new Size(153, 70);
41      button1.TabIndex = 0;
42      button1.Text = "按我!";
43      button1.UseVisualStyleBackColor = true;
44      button1.Click += button1_Click;
45      //
46      // Form1
47      //
48      AutoScaleDimensions = new.SizeF(11F, 23F);
49      AutoScaleMode = AutoScaleMode.Font;
50      ClientSize = new Size(800, 450);
51      Controls.Add(button1);
52      Name = "Form1";
53      Text = "Form1";
54      ResumeLayout(false);
55  }
56
57  #endregion
```

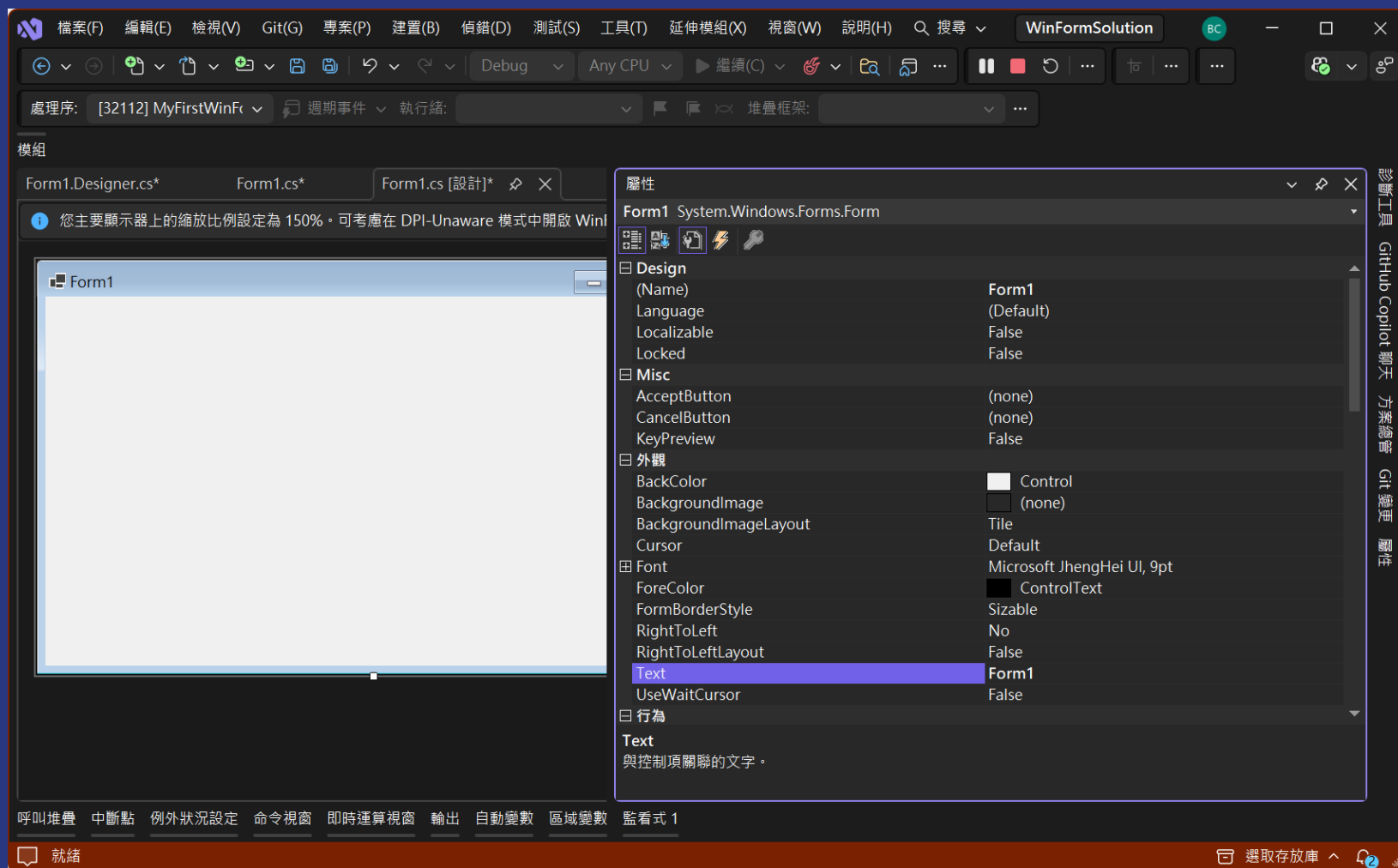
回到設計畫面，開啟屬性



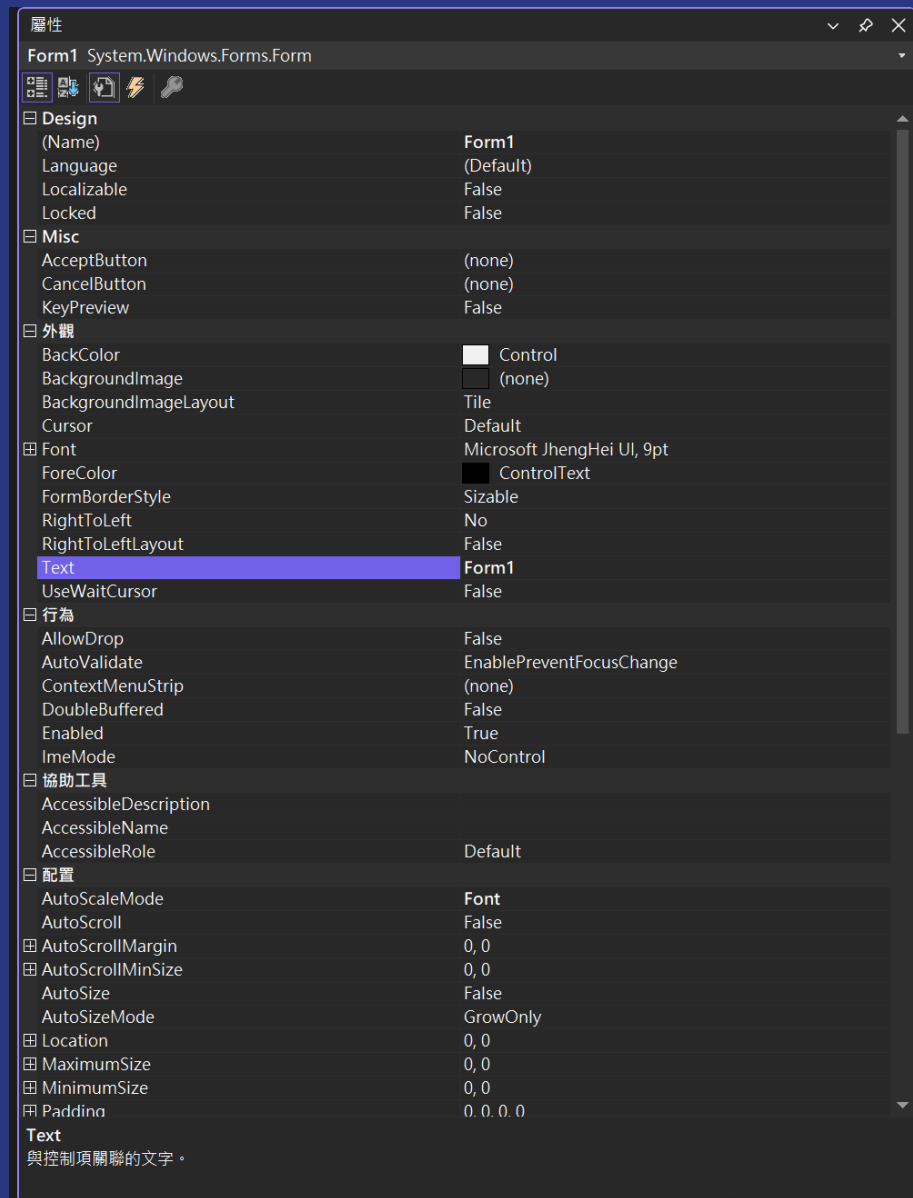


如果找不到屬性頁籤
【檢視】→【屬性視窗】

屬性頁籤內容畫面



簡單認識一下關於 Form 的屬性



外觀

- BackColor
- BackgroundImage
- BackgroundImageLayout
- ForeColor
- FormBorderStyle
- Text

討論

- 請改變 BackColor 與 Text
- 和你的同學討論這些改變造成甚麼影響



配置

- Location
- MaximumSize
- MinimumSize
- Size
- StartPosition
- WindowState

討論

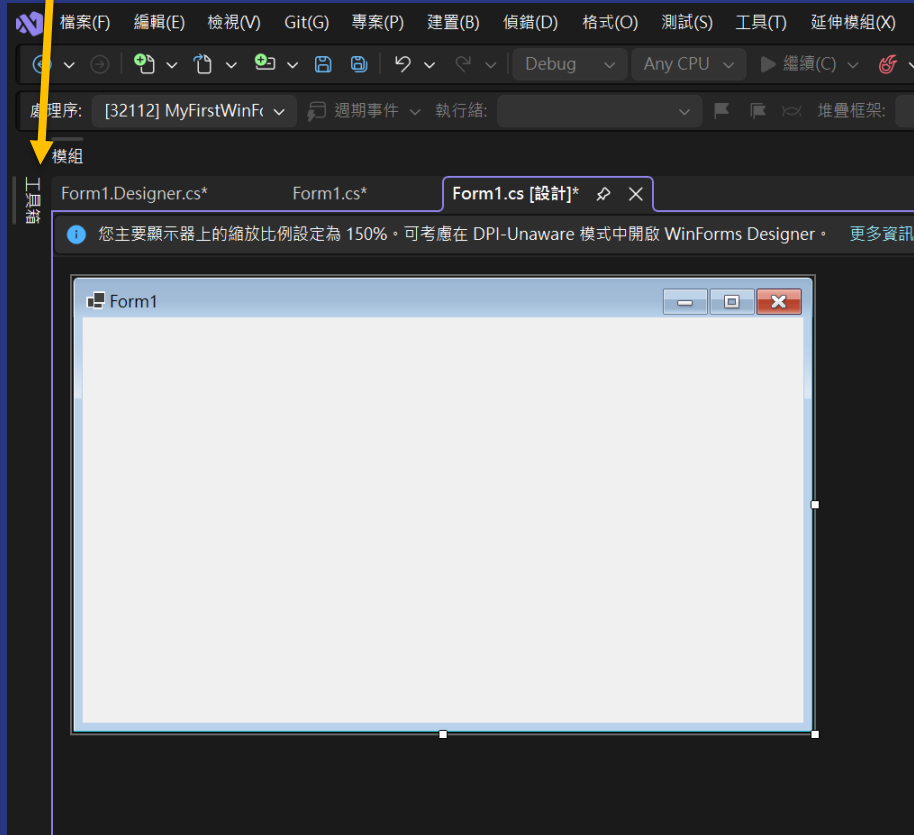
- 請改變 WindowState
- 和你的同學討論這些改變造成甚麼影響



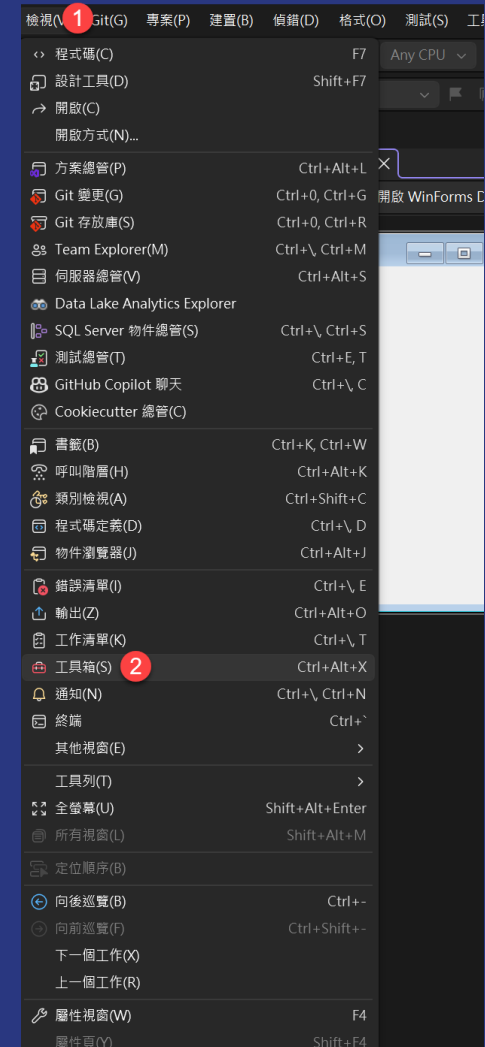
LAB

加個 Button

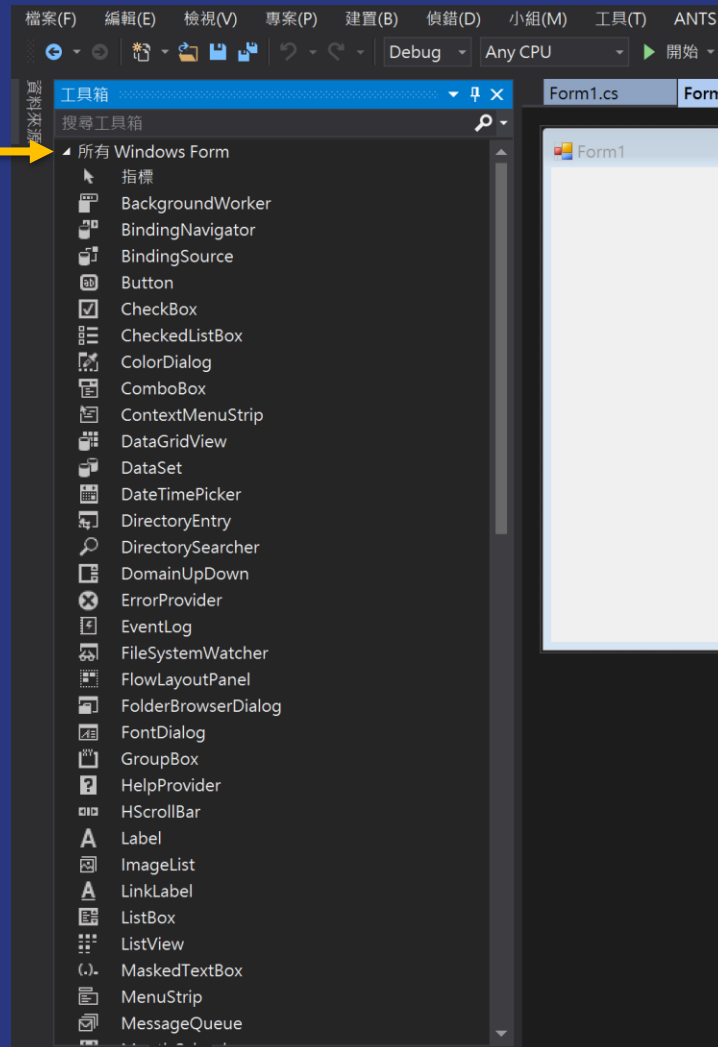
回到設計畫面，開啟工具箱



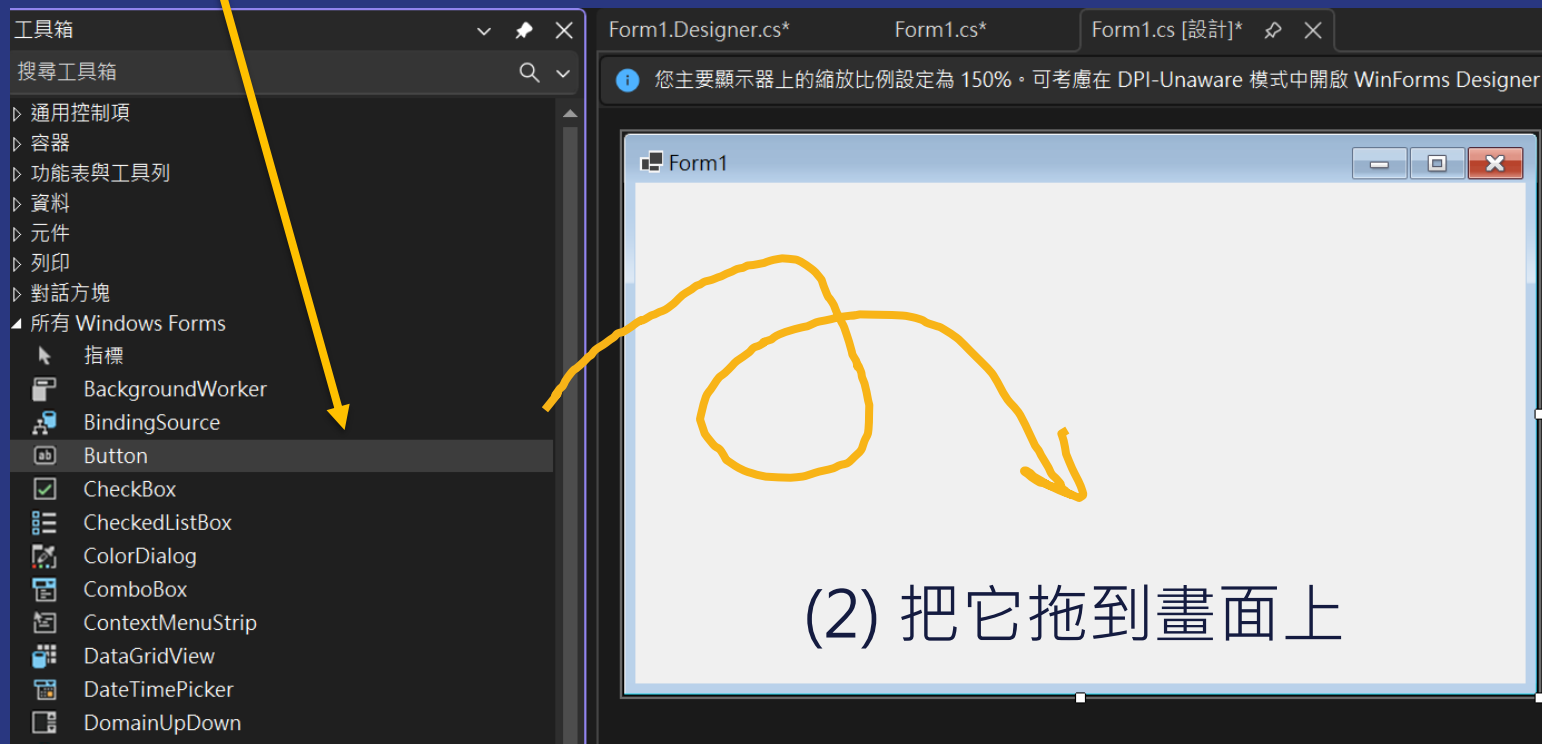
如果找不到呢？



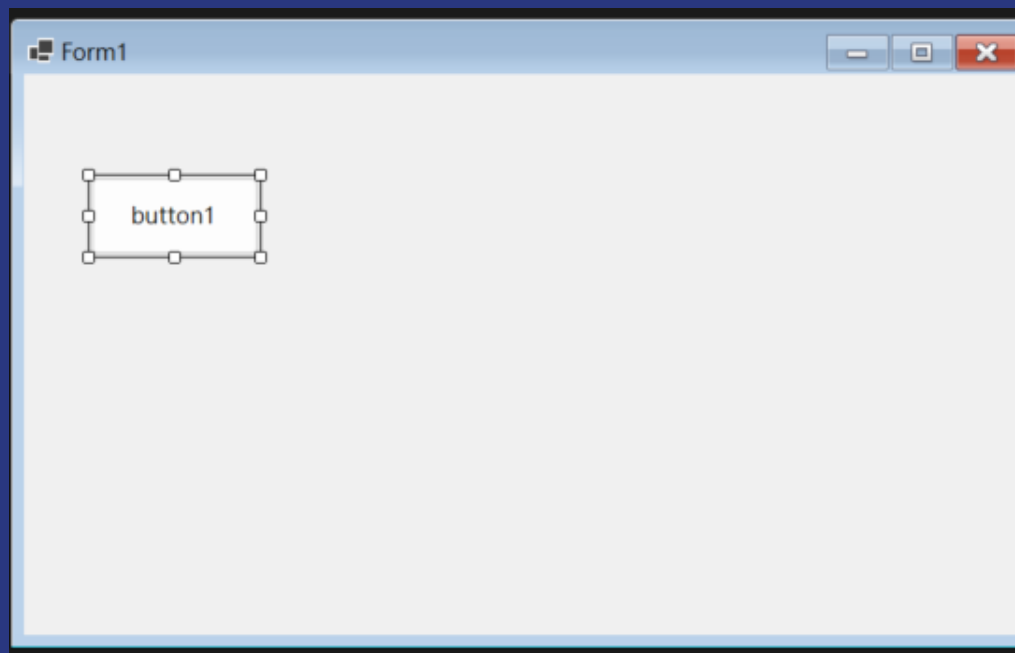
工具箱



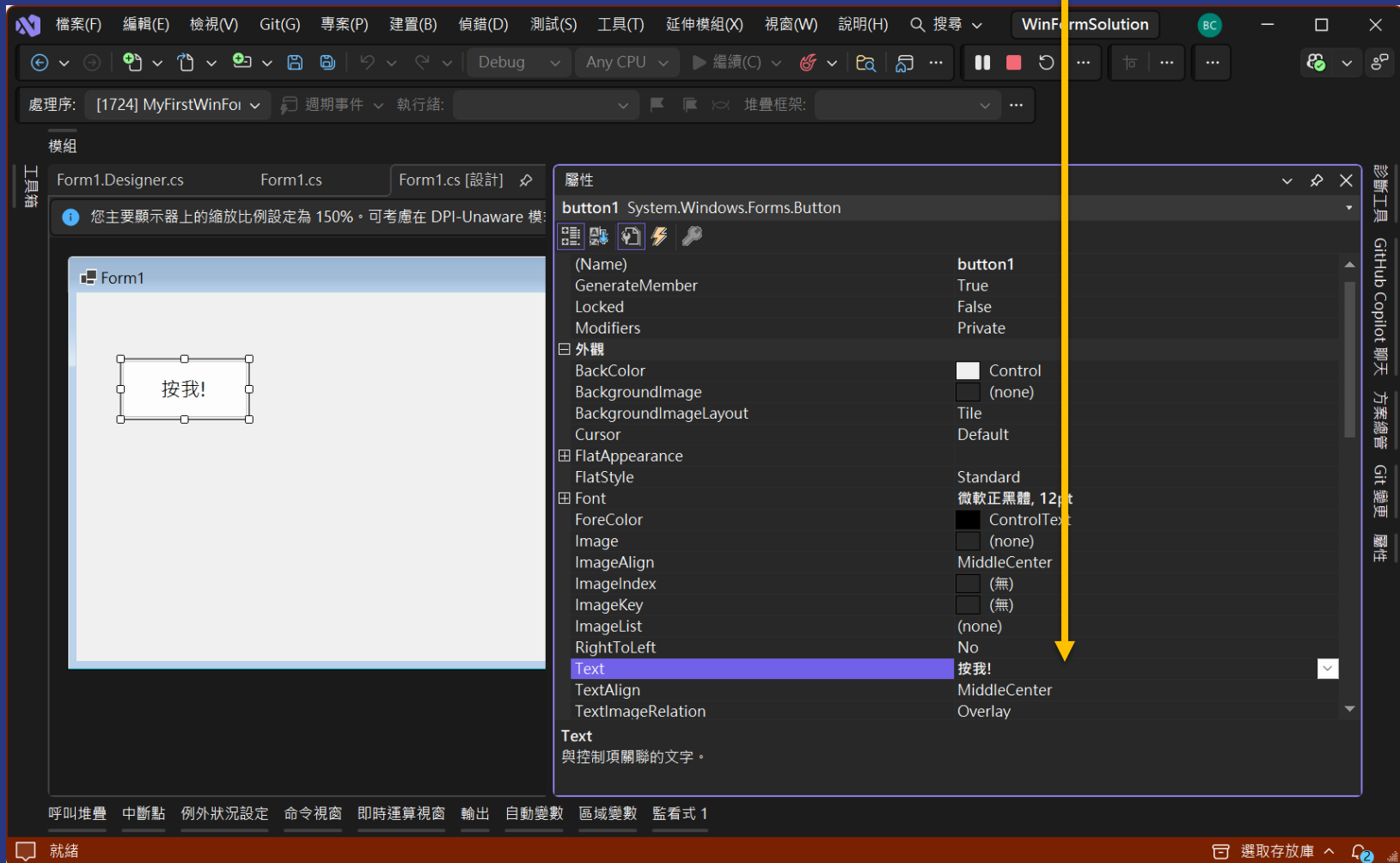
(1) 滑鼠左鍵按住【Button】



調整成適當的位置與大小

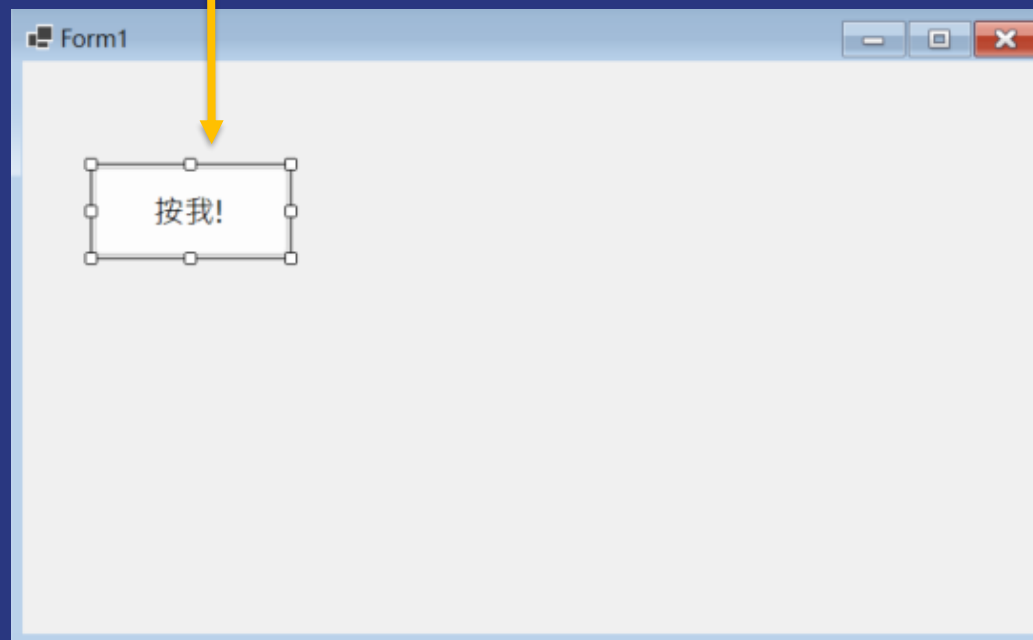


打開屬性頁籤，把 Text 的值改成【按我!】

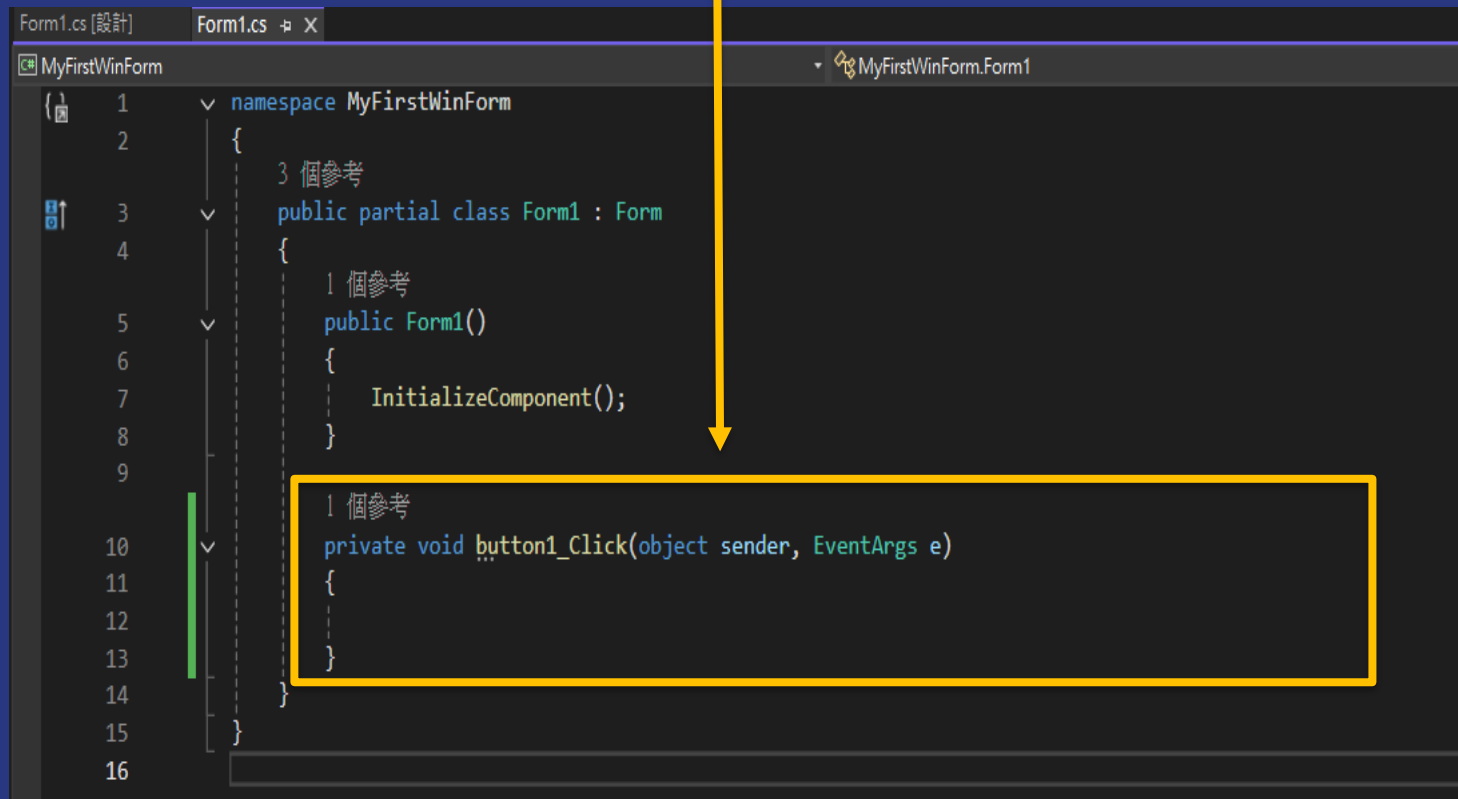




滑鼠雙擊這個按鈕



你的程式碼將會多出這段



```
1 namespace MyFirstWinForm
2 {
3     3 個參考
4     public partial class Form1 : Form
5     {
6         1 個參考
7         public Form1()
8         {
9             InitializeComponent();
10
11         1 個參考
12         private void button1_Click(object sender, EventArgs e)
13         {
14
15         }
16     }
17 }
```

事件委派方法

- 它其實就是一個方法 (Method / 函式)
- 這個方法特別的一點是『它和剛剛那個 Button 的 Click 事件有連結關係』

開啟 Form1.Designer.cs，找到這一段程式碼

```
private void InitializeComponent()
{
    button1 = new Button();
    SuspendLayout();
    //
    // button1
    //
    button1.Font = new Font("微軟正黑體", 12F, FontStyle.Regular, GraphicsUnit.Point, 136);
    button1.Location = new Point(56, 83);
    button1.Name = "button1";
    button1.Size = new Size(153, 70);
    button1.TabIndex = 0;
    button1.Text = "按我!";
    button1.UseVisualStyleBackColor = true;
    button1.Click += button1_Click;
    //
    // Form1
    //
    AutoScaleDimensions = new.SizeF(11F, 23F);
    AutoScaleMode = AutoScaleMode.Font;
    ClientSize = new Size(800, 450);
    Controls.Add(button1);
    Name = "Form1";
    Text = "Form1";
    ResumeLayout(false);
}
```

回到 Form1.cs 補上這段程式碼，然後執行它

```
private void button1_Click(object sender, EventArgs e)
{
    MessageBox.Show("Hello Windows Forms");
}
```



執行

LAB

簡單加減器

在 WinFormSolution 加入新專案

- 方案名稱：WinFormSolution
- 專案名稱：SimpleCalculator01
- 範本：Windows Forms
- csproj nullable disable

畫面配置



一些調整

- 兩個 Button 的 Size 改為 96,96
- button1 的 Text 屬性改為 加
- button2 的 Text 屬性改為 減
- label1 的 Text 屬性改為 0

接著，點選【加】的 Button，讓 Visual Studio 為這個 Button 的 Click 事件產生一個委派方法。
並加入以下程式碼

```
private void button1_Click(object sender, EventArgs e)
{
    int x = int.Parse(textBox1.Text);
    int y = int.Parse(textBox2.Text);
    label1.Text = (x + y).ToString();
}
```

然後，點選【減】的 Button，讓 Visual Studio 為這個 Button 的 Click 事件產生一個委派方法。
並加入以下程式碼

```
private void button2_Click(object sender, EventArgs e)
{
    int x = int.Parse(textBox1.Text);
    int y = int.Parse(textBox2.Text);
    label1.Text = (x - y).ToString();
}
```

執行

LAB

消除重複程式碼

看到了奇妙的重複嗎？

```
private void button1_Click(object sender, EventArgs e)
{
    int x = int.Parse(textBox1.Text);
    int y = int.Parse(textBox2.Text);
    label1.Text = (x + y).ToString();
}
```

```
private void button2_Click(object sender, EventArgs e)
{
    int x = int.Parse(textBox1.Text);
    int y = int.Parse(textBox2.Text);
    label1.Text = (x - y).ToString();
}
```

在 WinFormSolution 加入新專案

- 方案名稱：WinFormSolution
- 專案名稱：SimpleCalculator02
- 範本：Windows Forms
- csproj nullable disable
- 畫面配置同 SimpleCalculator01

討論

- 你們會如何消除這些重複？
- 請先不要偷看 SimpleCalculator02



討論

- 在 SimpleCalculator02 加入乘除的功能



學習自訂類別 (Class)

.NET 型別概觀

- 參考型別
 - class 類別 (包含 record)
 - interface 介面
 - delegate 委派
- 實值型別
 - structure 結構 (包含 record struct)
 - enum 列舉

先來點粗淺的認識

- 類別是一種參考型別
- 既然是型別，就可用來宣告變數
- 請先把類別想像成一個用來製造某種物品的藍圖或模具
- 我們通常把這個模具產生出來的物品稱為
 - 物件 (object)
 - 執行個體 (instance)
- 模具（類別）本身也是一個物件，由於其特殊性，我們通常會稱它為『型別物件 (Type Object)』

類別內的成員（結構通常也是這樣）

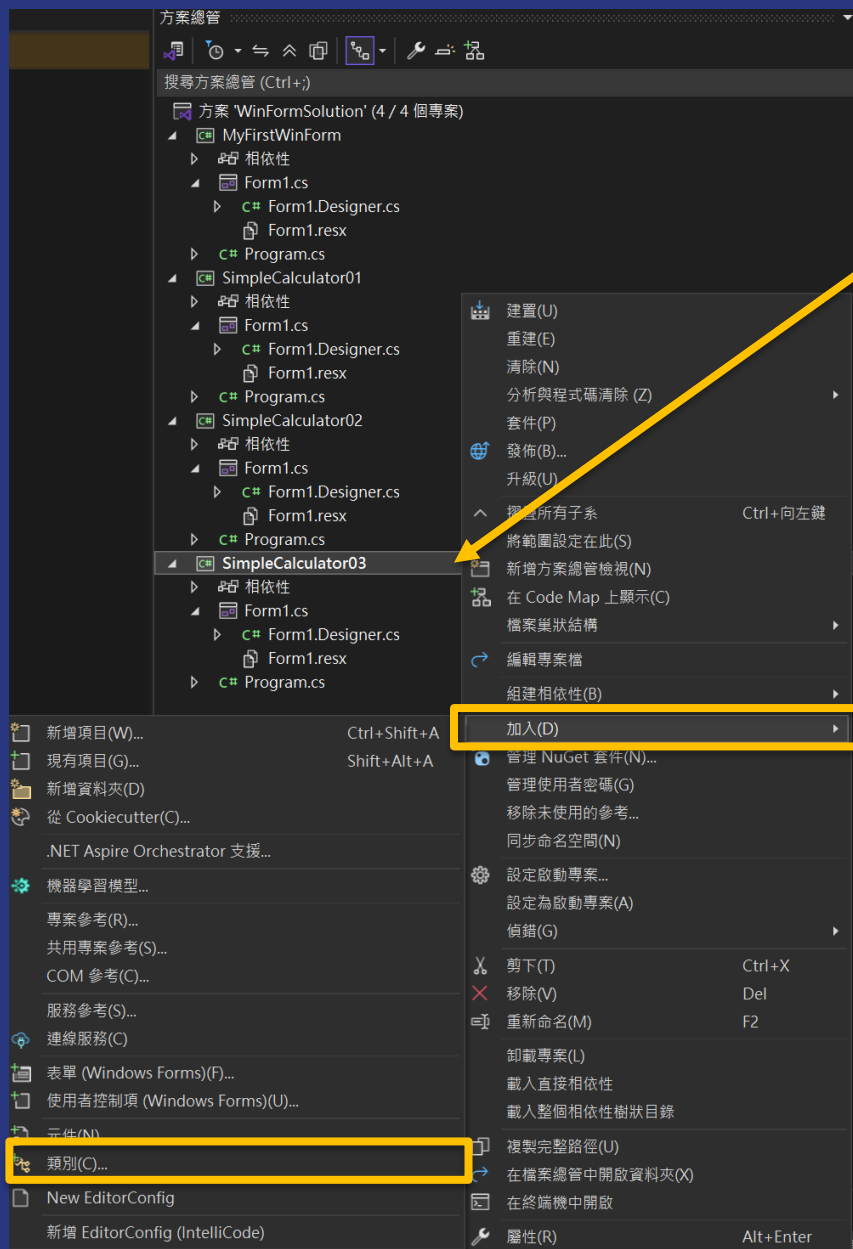
- 建構式 – Constructor
- 常數 – Constant
- 欄位 – Field
- 屬性 – Property
- 方法 – Method
- 事件 – Event
- 索引子 – Indexer
- 類別內也可以定義其他的型別，我們稱為巢狀型別

LAB

加入一個類別

在 WinFormSolution 加入新專案

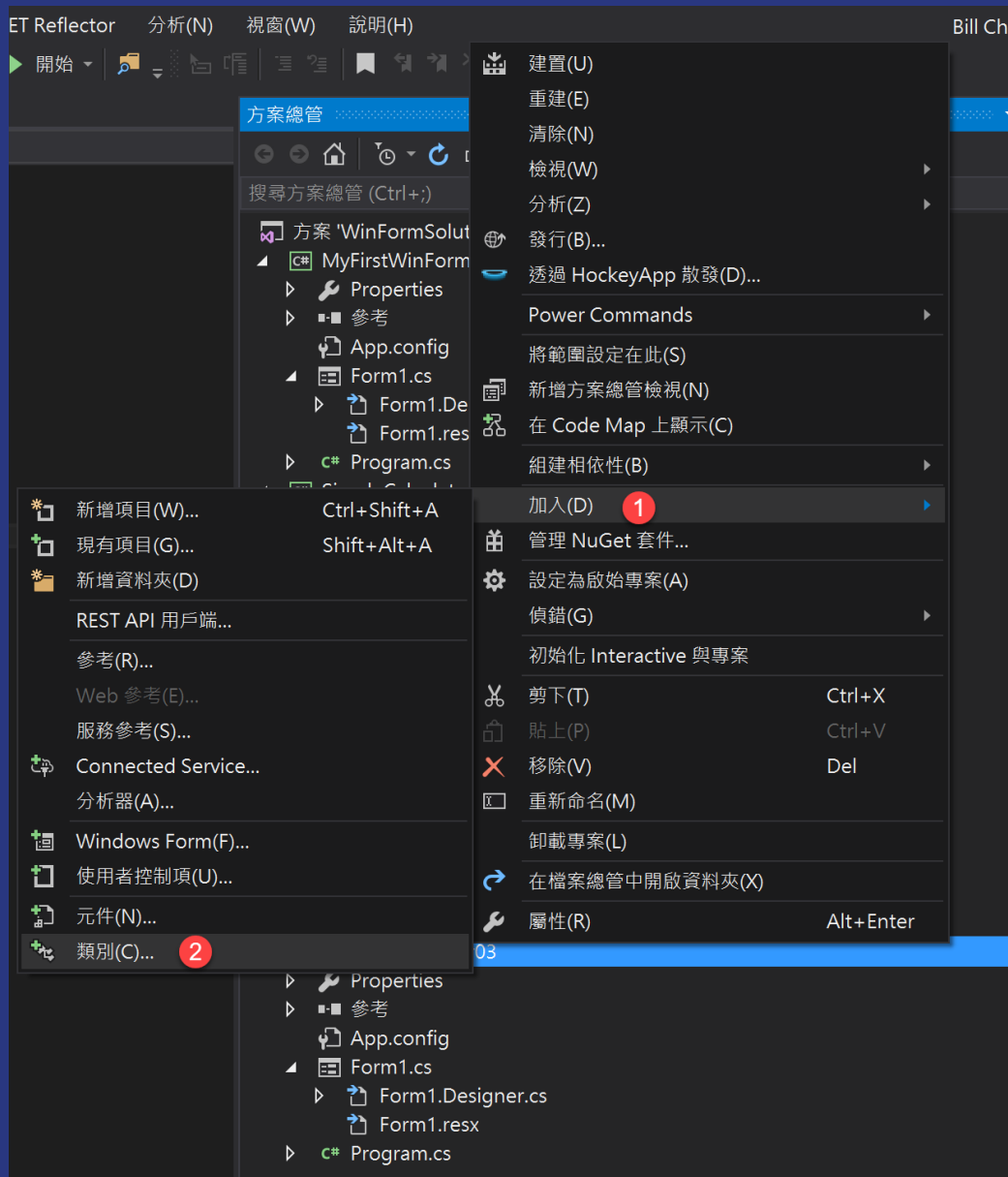
- 方案名稱：WinFormSolution
- 專案名稱：SimpleCalculator03
- 範本：Windows Forms
- csproj nullable disable
- 畫面配置同 SimpleCalculator01



選擇專案

SimpleCalculator03

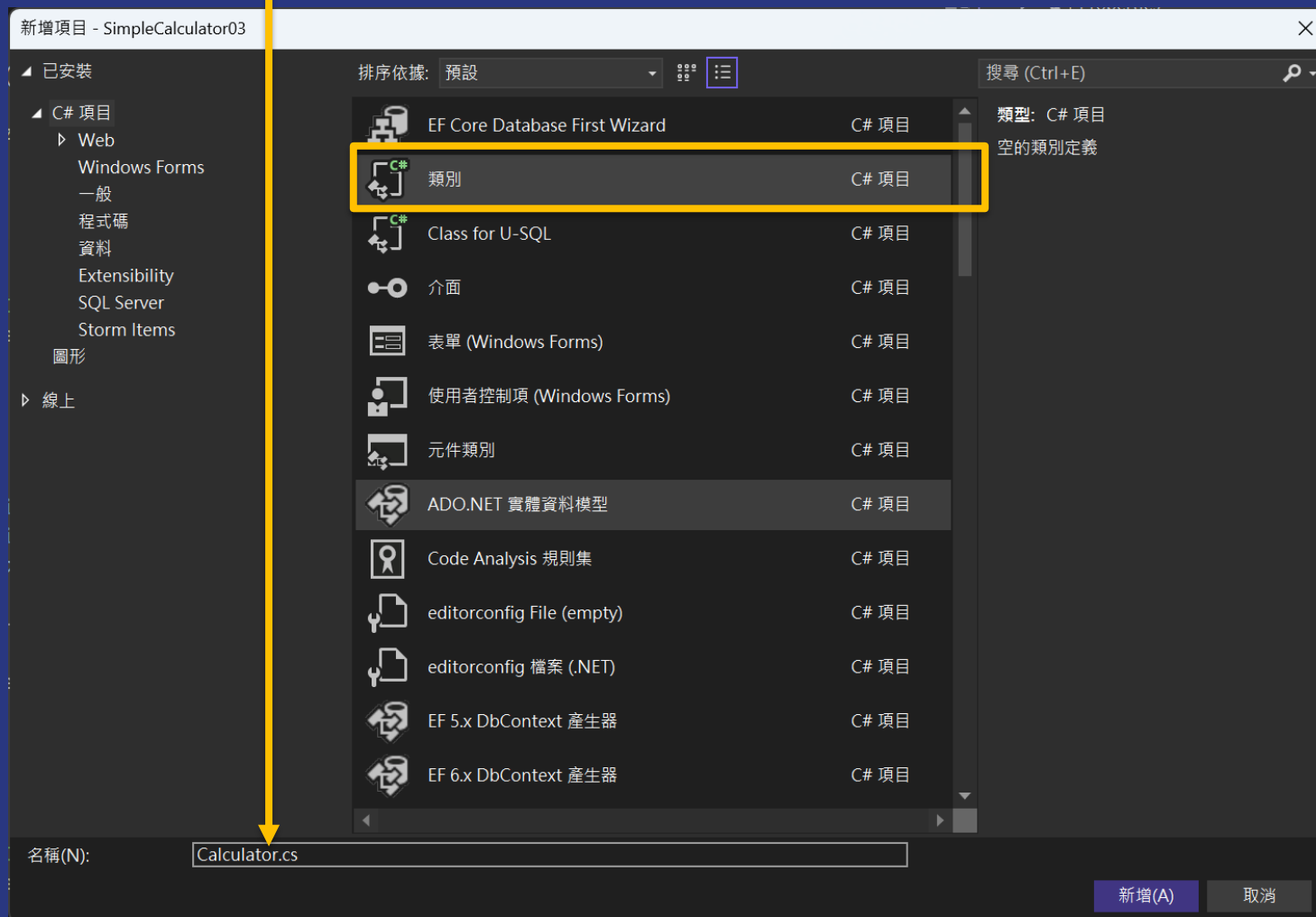
按下滑鼠右鍵



選擇
【加入】→
【類別】

註：
其實從
【加入】→
【新增項目】
也行。

將檔案名稱改為 Calculator.cs，然後按【新增】



出現 Calculator.cs 的程式碼編輯畫面

```
Calculator.cs [X]
SimpleCalculator03 SimpleCalculator03.Calculator
1  using System;
2      using System.Collections.Generic;
3      using System.Linq;
4      using System.Text;
5      using System.Threading.Tasks;
6
7  namespace SimpleCalculator03
8  {
9      0 個參考
10     internal class Calculator
11     {
12     }
13
```

欄位

- 「欄位」(Field) 是一個任意型別的變數，直接在類別或結構中宣告
- 類別 (Class) 或結構 (Struct) 可能會有執行個體 (Instance) 欄位或靜態 (Static) 欄位，或者兩個都有
- 先記得一件事，執行個體成員和靜態成員的存取路徑是不一樣的。
- 後面的課程會解釋執行個體成員與靜態成員的詳細差異。
- 先記得一件事，沒有 `static` 宣告的成員就是執行個體成員，但常數除外，常數永遠都是靜態的。

為 Calculator class 加入兩個欄位

```
internal class Calculator
{
    private int _x;
    private int _y;
}
```

屬性

- 屬性 (Property) 就是提供讀取、寫入或計算私用 (Private) 欄位值之彈性機制的成員
- 屬性是方法的變形
- 使用屬性取代欄位成為公開介面
- 自動實做屬性

為 Calculator class 加入兩個屬性

```
internal class Calculator
{
    private int _x;
    private int _y;

    public int X
    {
        get { return _x; }
        set { _x = value; }
    }

    public int Y
    {
        get { return _y; }
        set { _y = value; }
    }
}
```

屬性存取子 (accessor)

- `get` –用來讀取屬性所對應的值
- `set` –用來設定屬性所對應的值
- 兩者不一定需要同時存在
- 編譯器在建置程式碼的時候，會分別為 `get/set` 產生對應的 `method`

這種使用 {} 的存取子寫法稱為「區塊主體存取子」

```
get { return _x; }  
set { _x = value; }
```

上下寫法不同，但建置後結果是相同的

這種使用 => 的存取子寫法稱為「運算式主體存取子」

```
get => _x;  
set => _x = value;
```

運算式主體只允許一行程式碼，參考[運算式主體成員 \(C# 程式設計手冊\)](#)

提示詞：C# 運算式主體成員和區塊式主體成員的比較

屬性的真實樣貌

```
public int X
{
    get { return _x; }
    set { _x = value; }
}
```



```
public int GetX()
{
    return _x;
}

public void SetX(int value)
{
    _x = value;
}
```

自動實作屬性

當屬性沒有甚麼特別要求的時候，
可以採用『自動實作屬性』的寫法
省略掉其儲存欄位以及 **getter** 與 **setter** 的內容
C# 7 之後有更多關於屬性的奇妙寫法

```
internal class Calculator
{
    public int X { get; set; }
    public int Y { get; set; }
}
```

方法

- 方法」(Method) 是包含一系列陳述式 (Statement) 的程式碼區塊。 程式會「呼叫」(Calling) 方法並指定所有必要的方法引數，藉以執行陳述式

方法宣告

這個方法的名稱

這個方法的傳入參數

```
public int Add(int x, int y)
{
}
}
```

這個方法的回傳值
如果沒有回傳值則宣告為 **void**

為 Calculator class 加入 Add 與 Subtract 方法

```
internal class Calculator
{
    //.. 前略
    public int Add()
    {
        return X + Y;
    }

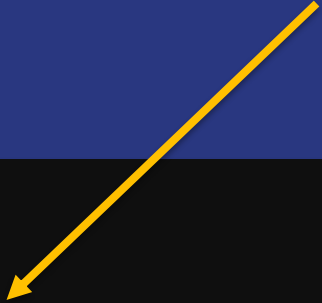
    public int Subtract()
    {
        return X - Y;
    }
}
```

LAB

使用 Calculator 類別

在 Form1 Class 宣告一個 Calculator 型別的欄位

```
public partial class Form1 : Form
{
    private Calculator _calculator;
    public Form1()
    {
        InitializeComponent();
    }
}
```



在 Form1 Class 建構式中
指派 Calculator 執行個體給 _calculator 變數

```
public partial class Form1 : Form
{
    private Calculator _calculator;
    public Form1()
    {
        InitializeComponent();
        _calculator = new Calculator();
    }
}
```



你也可以這樣寫
但目前不建議這樣用



```
public partial class Form1 : Form
{
    private Calculator _calculator = new Calculator();
    public Form1()
    {
        InitializeComponent();
    }
}
```

為 Form1 Class 補上其他程式碼

```
private void button1_Click(object sender, EventArgs e)
{
    label1.Text = CallCalculator("+");
}

private void button2_Click(object sender, EventArgs e)
{
    label1.Text = CallCalculator("-");
}

private string CallCalculator(string operation)
{
    _calculator.X = int.Parse(textBox1.Text);
    _calculator.Y = int.Parse(textBox2.Text);
    switch (operation)
    {
        case "+":
            return _calculator.Add().ToString();
        case "-":
            return _calculator.Subtract().ToString();
        default:
            return "Unknown operation";
    }
}
```



執行

為何要設計類別

- 塑造了良好且可維護的程式碼結構
- 職責觀念
 - 模組化職責
 - 將不同的功能分離到不同的類別中，避免一個類別承擔過多職責
 - 降低耦合度
 - 當功能需求改變時，只需修改對應的類別，不會影響其他不相關的功能
 - 提高可維護性
 - 每個類別的職責清晰，容易理解和修改

- 重用觀念

- 一次編寫，多處使用
 - 設計好的類別可以在專案的多個地方重複使用
- 標準化操作
 - 建立統一的操作介面和行為模式
- 減少錯誤
 - 重用經過測試的程式碼，降低新錯誤的產生機率

雞蛋糕的故事 – class and object

如果你曾買過雞蛋糕

- 老闆有個模具用來做雞蛋糕的
 - 這就是個類別 (`Class`)，類別是一種型別。
 - 而模具本身是一個存在的個體，所以它也是個物件，但因為它是用來產生另外一個物件(也就是雞蛋糕)的東西，所以我們稱它為 `Type Object` (型別物件)。
- 你會拿到一個雞蛋糕
 - 這個雞蛋糕就是 `Instance` (執行個體)，當然它也被稱為 `Object` (物件)

關於模具

- 模具由幾個部分組成
 - 上面的鐵板
 - 下面的鐵板
 - 一個把手
- 以上這些物品都是屬於模具的，所以會使用 `static` 宣告，`static` 在中文通常被稱為靜態。
- 也因此上述三樣被稱為 `static members`（靜態成員）。

範例

- 以下，我們使用雞蛋糕的範例，來解說剛剛的概念。
- 請打開以下範例。
- `CakeSample`

在這裏你將學到

Learn How to Learn

- 學新東西、新技術的能力
- 尋找解答的能力
- 隨時吸取新知識的能力

跟著你一輩子的能力 ...